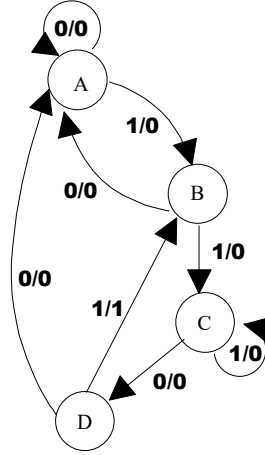


Örnek: Ard arda gelen clock saykılarında girişin iki tane 1, sonra 0 , daha sonra da 1 olması durumunda çıkışın 1 olmasını sağlayacak Mealy türü bir sistem tasarlanması isteniyor. Çıkışın 1 olmasına neden olan 1 girişi sonraki clock saykılarında kullanılacaktır (tekrar var).

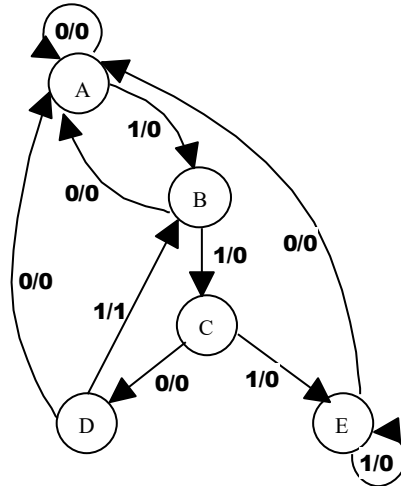
x 0 1 1 0 1 1 0 1 0 0 1 1 1 0 1 1 0 1
z 0 0 0 0 1 0 0 1 0 0 0 0 0 0 1 0 0 1



Soruya bir ilave yapalım:

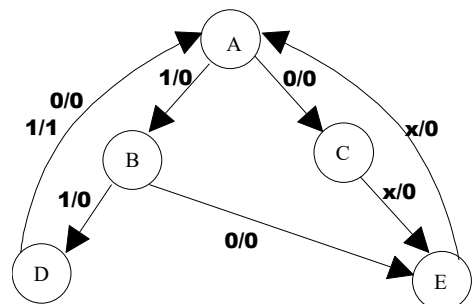
Ancak ard arda girişte 3 veya daha fazla sayıda 1 gelirse bu 1 'lerin dikkate alınmaması isteniyor.

x 0 1 1 0 1 1 0 1 0 0 1 1 1 1 0 1 1 0 1
z 0 0 0 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 1



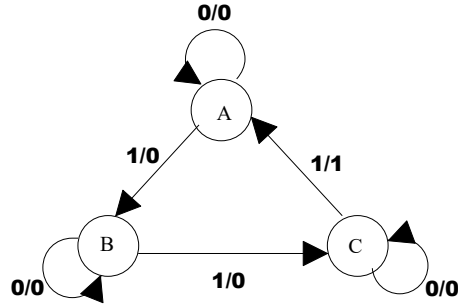
Örnek: Girişin üçlü grup olarak düşünüldüğü Mealy türü bir sistemde, grubun tüm elemanlarının 1 olması durumunda çıkışın 1 olması isteniyor.

x	1	0	1	1	1	0	1	1	1	0	1	1
z	0	0	0	0	0	0	0	0	1	0	0	0



Örnek: Girişe, her üçüncü defa 1 geldiğinde, çıkışın 1 olması isteniyor. Girişteki 1'lerin ard arda olma zorunluluğu yoktur. Ayrıca, çıkışın 1 olduğu durumda, girişteki 1 değeri bir sonraki işleme katılmayacaktır (tekrar yok, Non-overlapping).

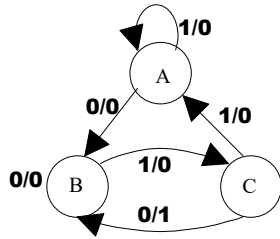
x 0 1 1 1 0 1 0 1 0 1 1 0 1 0
z 0 0 0 1 0 0 0 0 0 1 0 0 0 0



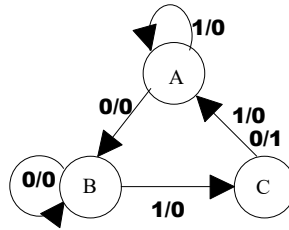
Örnek: Mealy türü bir sistemde, ard arda gelen clock saykılarında giriş 010 ise çıkışın 1 olması isteniyor.

a) Girişte tekrarlama olsun.

a) Girişte tekrarlama olmasın.

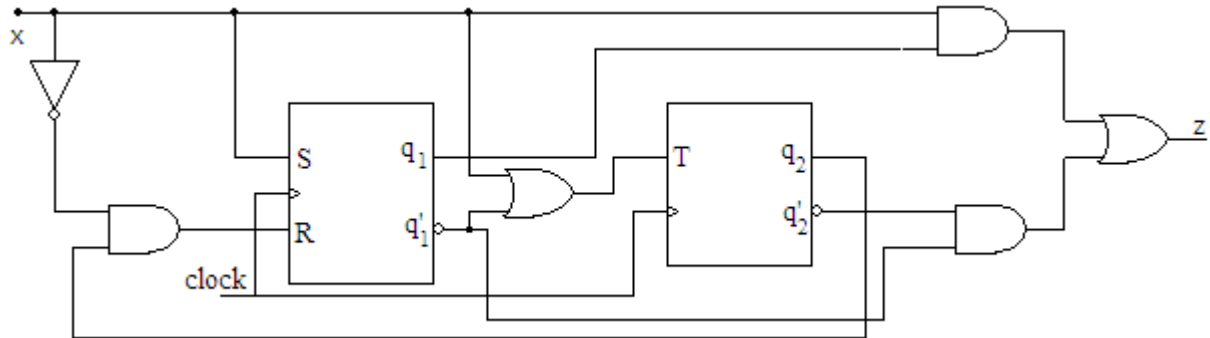


x 1 1 0 1 0 1 0 1
z 0 0 0 0 1 0 1 0



x 0 1 0 1 0 1 0
z 0 0 1 0 0 0 1

Örnek: Aşağıdaki devrenin durum diyagramını oluşturalım ve x'in 0 0 1 1 0 0 1 0 0 değerleri için çıkışı bulalım.



$$S=x \quad R=x'.q_2 \quad T=x+q_1' \quad z=x.q_1+q_1'.q_2'$$

x=0 için S=0 ve R=q₂. Dolayısıyla q₂=0 için SR flip flobu durumunu koruyacak aksi durumda çıkış 0 olacak. x=1 için SR flip flobunun çıkışı 1 olacak. x=1 veya q₁=0 için T flip flobu mevcut durumunun tersini alacak, aksi halde durumunu muhafaza edecek. Durum tablosu;

Şimdiki Durum q_1q_2	Sonraki Durum(Q_1Q_2)		Çıkış (z)	
	x=0	x=1	x=0	x=1
00	0 1	1 1	1	1
01	0 0	1 0	0	0
10	1 0	1 1	0	1
11	0 1	1 0	0	1

(Sistemin başlangıç durumu $q_1q_2=00$ olarak verilmiştir.)

x 0 0 1 1 0 0 1 0 0
 q_1 0 0 0 1 1 1 1 1 0 0
 q_2 0 1 0 1 0 0 0 1 1 0 1
z 1 0 1 1 0 0 1 0 0 1

Örnek: Aşağıda verilen durum tablosuna göre, verilen x girişi için flip flopların durumlarını ve çıkışı belirleyiniz.

Şimdiki Durum q_1q_2	Sonraki Durum(Q_1Q_2)		Çıkış (z)	
	x=0	x=1	x=0	x=1
00	00	10	0	1
01	00	00	0	0
10	11	01	1	1
11	10	10	1	0

(Sistemin başlangıç durumu $q_1q_2=00$ olarak verilmiştir.)

x 0 1 0 0 1 1 1 0
 q_1 0 0 1 1 1 1 0 0 1 1 1
 q_2 0 0 0 1 0 1 0 0 1 0 1 0
z 0 1 1 1 1 0 1 1 - 1

Örnek: Aşağıda verilen durum tablosuna göre, verilen x girişi için flip flopların durumlarını ve çıkışı belirleyiniz.

Şimdiki Durum q	Sonraki Durum(Q)		Çıkış (z)
	x=0	x=1	
A	A	B	1
B	D	C	1
C	D	C	0
D	A	B	0

x 0 1 0 1 0 1 1 1 0 1 0 0 0 0
q A A B D B D B C C D B D A A A
z 1 1 1 0 1 0 1 0 0 0 1 0 1 1 1 1

Sıralı olmayan sayıcı örneği ve don't care durumların incelenmesi

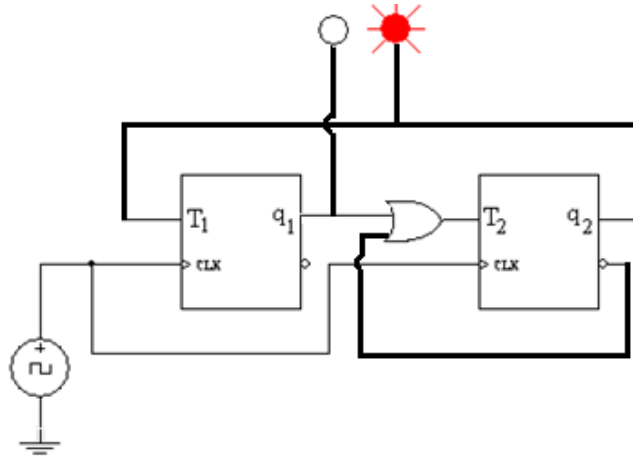
Örnek: $0 \rightarrow 1 \rightarrow 3$

3 durum olduğu için ve 3 sayısını görüntüleyebilmek için 2 flip flop kullanmak gerekir. T tipi flip floplar kullanarak gerçekleştirimi yapalım:

Şimdiki Durum $q_1 q_2$	Sonraki Durum $Q_1 Q_2$	$T_1 T_2$
0 0	0 1	0 1
0 1	1 1	1 0
1 1	0 0	1 1
1 0		x x

$$T_1 = q_2$$

$$T_2 = q_2' + q_1$$



Sistem şayet don't care durum olan 10 ile başlasaydı hangi duruma gideceğini bulalım:

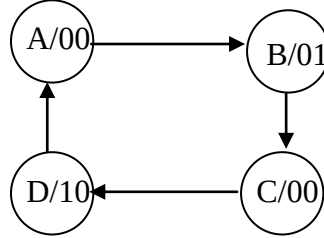
Karno haritası incelendiğinde T_1 için don't care durum 0, T_2 için ise 1 kabul edilmiş. O halde T_1 flip flopu durumunu koruyacak, T_2 flip flopu ise mevcut durumunun tersini alacaktır. Yani 10 durumundan 11 durumuna gidilecektir. Daha sonra da bizim belirlediğimiz sırada sayma işlemi gerçekleşecektir. Aşağıdaki tabloda bu durum gösterilmiştir.

Şimdiki Durum $q_1 q_2$	Sonraki Durum $Q_1 Q_2$	$T_1 T_2$
0 0	0 1	0 1
0 1	1 1	1 0
1 1	0 0	1 1
1 0	1 1	0 1

Aşağıda değişik bir sayıcı tasarımı vardır. Genellikle sayıcıların, giriş ve z çıkışına sahip olmadan tasarlandığını söylemiştik. Ders notlarında girişe bağlı olarak ileri ya da geri sayan sayıcı örneği vardır. burada ise ekstra z çıkışları kullanılmıştır. Çünkü direkt olarak durumlardan çıkış almak bu örnekte mümkün olmamaktadır.

Clock darbesiyle 0-1-0-2-0-1-0-2-... şeklinde sayan bir sayıcıyı D tipi flip floplar kullanarak tasarlayınız? (İpucu: Sistem 4 duruma sahiptir. Bu sayıları göstermek için de 2 çıkışı vardır. Moore tarzı devredir.)

q1q0	Q1Q0	z1	z0
A	B	0	0
B	C	0	1
C	D	0	0
D	A	1	0



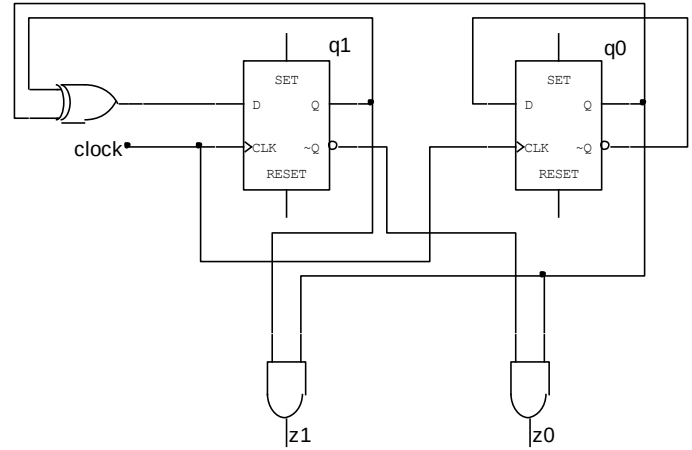
q1q0	Q1Q0	z1	z0	D1	D0
00	01	0	0	0	1
01	10	0	1	1	0
10	11	0	0	1	1
11	00	1	0	0	0

$$D1 = q1 \oplus q0$$

$$D0 = q0'$$

$$z1 = q1 \cdot q0$$

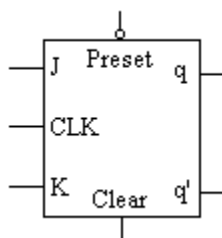
$$z0 = q1' \cdot q0$$



z1 ve z0 uçlarına 2 led bağlarsak önce 2 ledin de sönmük olduğunu, daha sonra sağdaki ledin yandığını, daha sonra 2 ledin de sönmük olduğunu ve son olarak da soldaki ledin yandığını görebiliriz. Bu işlem sürekli olarak devam edecektir. Bu tasarımda kullanılan mantıkla değişik led animasyonları (yürüyen ışık gibi) yapabiliriz.

Flip flopların başlangıç durumu nasıl ayarlanır?

Flip flopların *Preset* ve *Clear* girişleri ile ayarlama yapılır. Örnek olarak aşağıdaki flip flopbun çıkışını 0 yapmak istersek *Clear* ucunu aktif, *Preset* ucunu ise pasif yapmamız gerekir. Yani *Clear* ucuna lojik 1, *Preset* ucuna da lojik 1 (değilleme işleminden dolayı pasif durum 1'dir) uygulamamız gerekir. Fakat sistemin normal çalışmasına başlayabilmesi için tekrardan *Clear* ucunu pasif yani 0 durumuna getirmemiz gerekecektir. Bu işlem için bir butondan faydalanılabilir.



Bölüm 1. Giriş

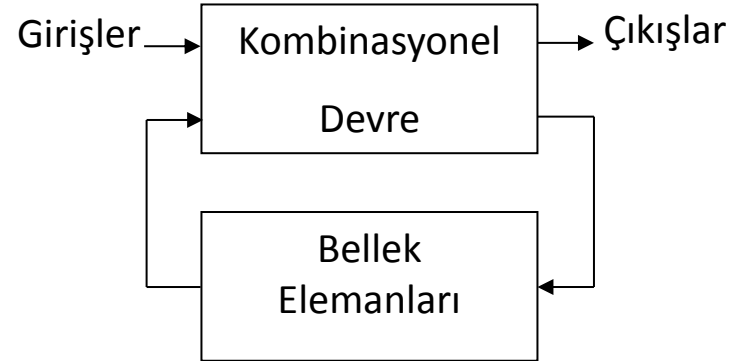
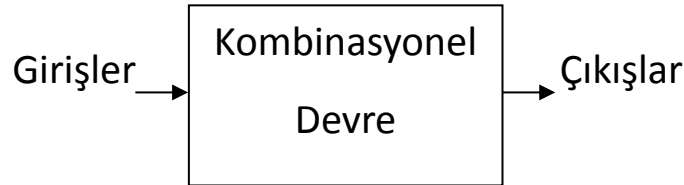
Latche ve Flip-Floplar

Flip-Flop Türleri

- SR tipi
- D tipi
- T tipi
- JK tipi

Giriş

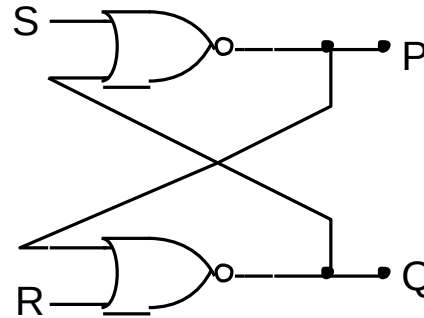
Lojik devreler kombinasyonel ve ardışıl devreler olmak üzere iki kategoride incelenir. Ardışıl devrelerin ise senkron ve asenkron formu vardır. Bilindiği gibi kombinasyonel devrelerin çıkışları, girişlerin şimdiki değerlerine bağlıyken, ardışıl devrelerinki şimdiki ve önceki değerlerine bağlıdır. Ardışıl devrelerde bellek elemanları olarak latche'ler, flip-floplar, register'lar, sayıcılar gibi depolama elemanları kullanılır.



Latche ve Flip Floplar

Latche'ler iki veya daha fazla kapının geri beslemeli olarak birbirine bağlanmasından oluşan ikili bilgiyi tutan bellek elemanlarıdır.

SR Tipi Latche:



$$P = (S+Q)'$$
$$Q = (R+P)'$$

- **Normal depolama durumu:** Her iki girişin de 0 olduğu durumdur.

$S = R = 0$ durumunda çıkışlar birbirinin tümleyenidir $P = Q'$ ve $Q = P'$.

- **Bilgi yükleme:** Bir latche ya 1 ya da 0 değerini depolar.

$\Rightarrow$ 1 yüklemek için $S = 1$ ve $R = 0$ yapılır. Bu durumda $Q = 1$ ve $P = 0$ 'dır.

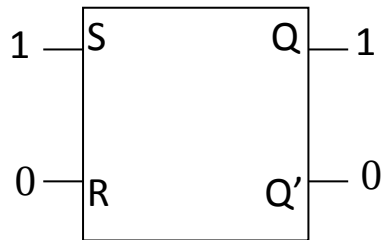
$\Rightarrow$ 0 yüklemek için $S = 0$ ve $R = 1$ yapılır. Bu durumda $Q = 0$ ve $P = 1$ 'dir.

P çıkışı genellikle Q' olarak ifade edilir. S girişi set ve R girişi de reset manası taşır.

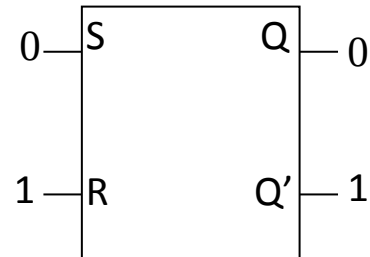
SR Tipi Latche

Anlatılanları şekil üzerinde özetleyecek olursak;

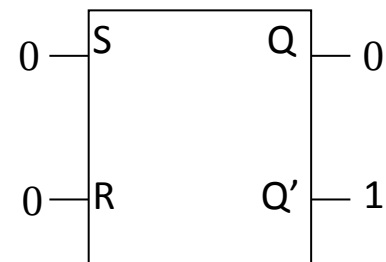
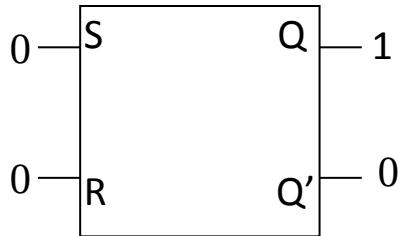
Set Durumu



Reset Durumu



Daha sonrasında $SR = 00$ yaparsak normal depolama durumu oluşur.



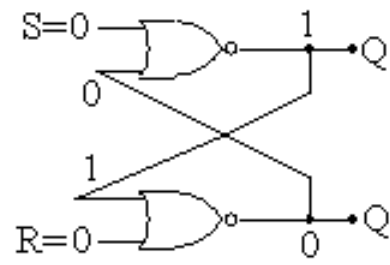
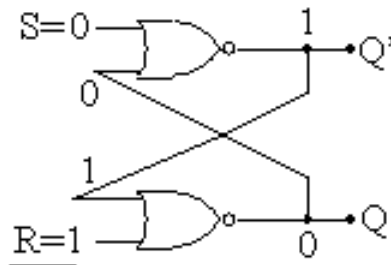
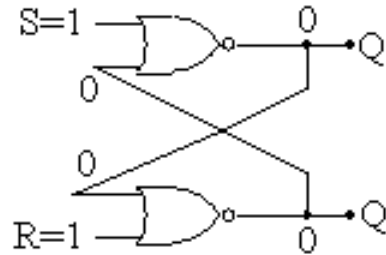
SR Tipi Latche

Şayet her iki girişe de 1 verilirse, tanım denklemlerinden

$$P = (S+Q)' \text{ ve } Q = (R+P)'$$

$$P = (1+Q)' = 0 \text{ ve } Q = (1+P)' = 0 \text{ olur.}$$

Çıkışlar birbirinin tümleyeni olmamıştır. SR türü latche için girişlerin her ikisinin de aynı anda 1 olmasına müsaade edilmez.



Flip-Floplar

Latche'lere göre daha güvenilir bir şekilde veri yüklemek için flip floplar geliştirilmiştir.

Flip floplar, clock girişine sahip olan depolama elemanlarıdır.

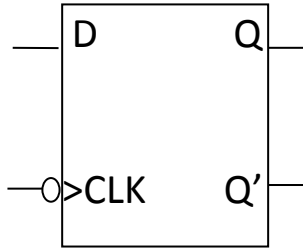
Flip flobun değeri ancak clock geçişiyle değişir. Clock girişinin 1'den 0'a geçişiyle tetiklenenler negatif kenar tetiklemeli, 0'dan 1'e geçişiyle tetiklenenler de pozitif kenar tetiklemeli (trailing-edge triggered) flip floplardır.

Clock geçişiyle flip floba ne yükleneceği, veri girişlerine ve bir önceki geçişte ne yüklendiğine bağlıdır.

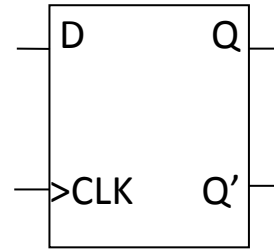
Temel olarak 4 tip flip flop vardır; D, JK, SR ve T.

D Tipi Flip Flop

D harfi gecikme (Delay) manası taşır; bir sonraki clock geçişine kadar giriş geciktirilir. D flip flobunun bir sonraki değeri clock geçişinden önceki D değerine bağlıdır.



Negatif kenar tetiklemeli



Pozitif kenar tetiklemeli

D flip flobunun doğruluk tablosu;

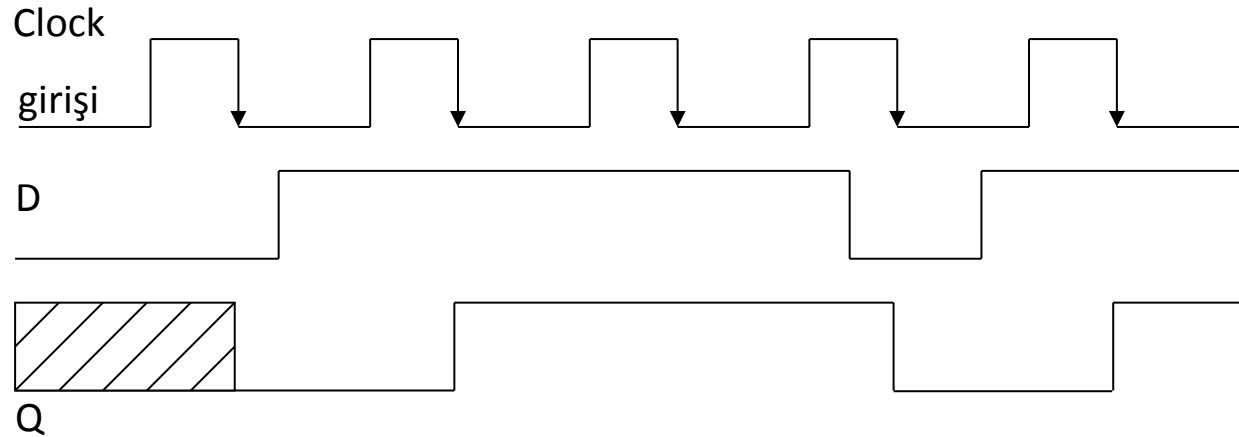
D	q	Q
0	0	0
0	1	0
1	0	1
1	1	1

D	Q
0	0
1	1

$$Q = D$$

D Tipi Flip Flop

Negatif kenar tetiklemeli D flip flobunun davranışını zaman ekseninde inceleyelim,



SR Tipi Flip Flop

SR flip flopbunun, SR latche gibi *set* ve *reset* manasına gelen iki girişı vardır.

Doğruluk tablosu;

S	R	q	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

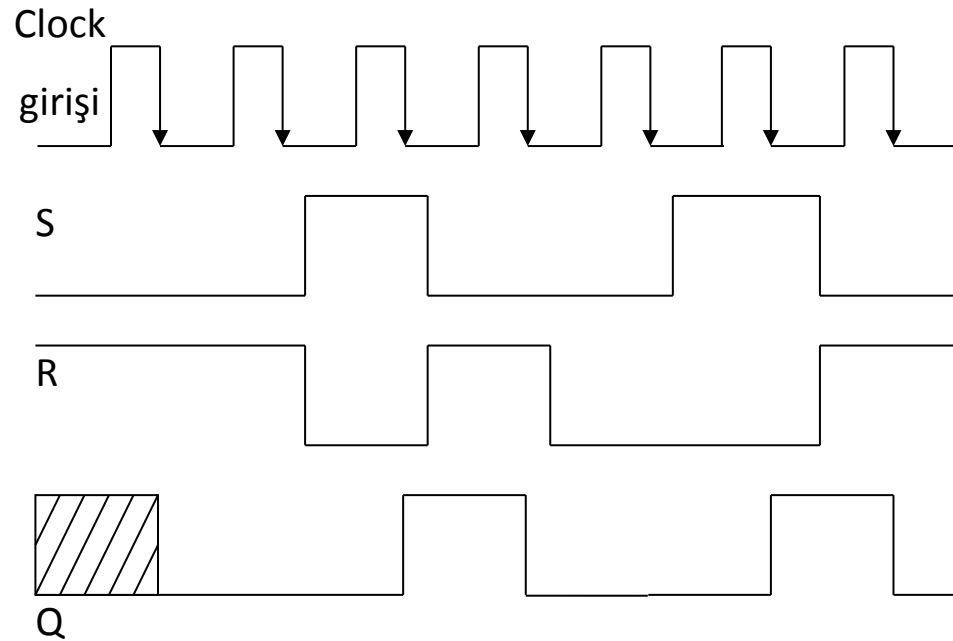
S	R	Q
0	0	q
0	1	0
1	0	1
1	1	-

		Q			
q	SR	00	01	11	10
	0			x	1
1		1		x	1

$$Q = S + R'.q$$

SR Tipi Flip Flop

Negatif kenar tetiklemeli SR flip flobunun davranışını zaman ekseninde inceleyelim,



T Tipi Flip Flop

T (Toggle) tipi flip flop, T girişine sahiptir. Şayet $T=1$ olursa flip flop durum değiştirir, $T=0$ olursa durumunu muhafaza eder.

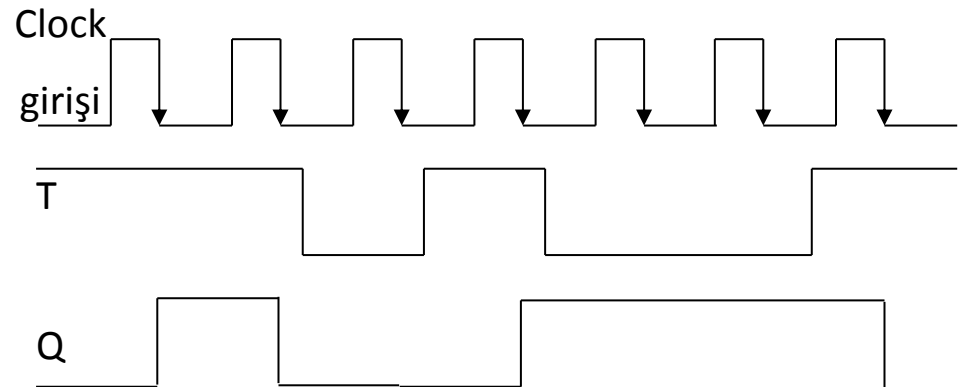
Doğruluk tablosu;

T	q	Q
0	0	0
0	1	1
1	0	1
1	1	0

T	Q
0	q
1	q'

$$Q = T \oplus q$$

Negatif kenar tetiklemeli T flip floğunun davranışını zaman ekseninde inceleyelim.



JK tipi flip flop

J ve K diye iki girişe sahiptir. SR ve T flip floplarının kombinasyonu olarak düşünülebilir; J=K=1 durumu dışında SR flip flobu gibi davranır, J=K=1 durumunda ise T flip flobu gibi davranır.

Doğruluk tablosu;

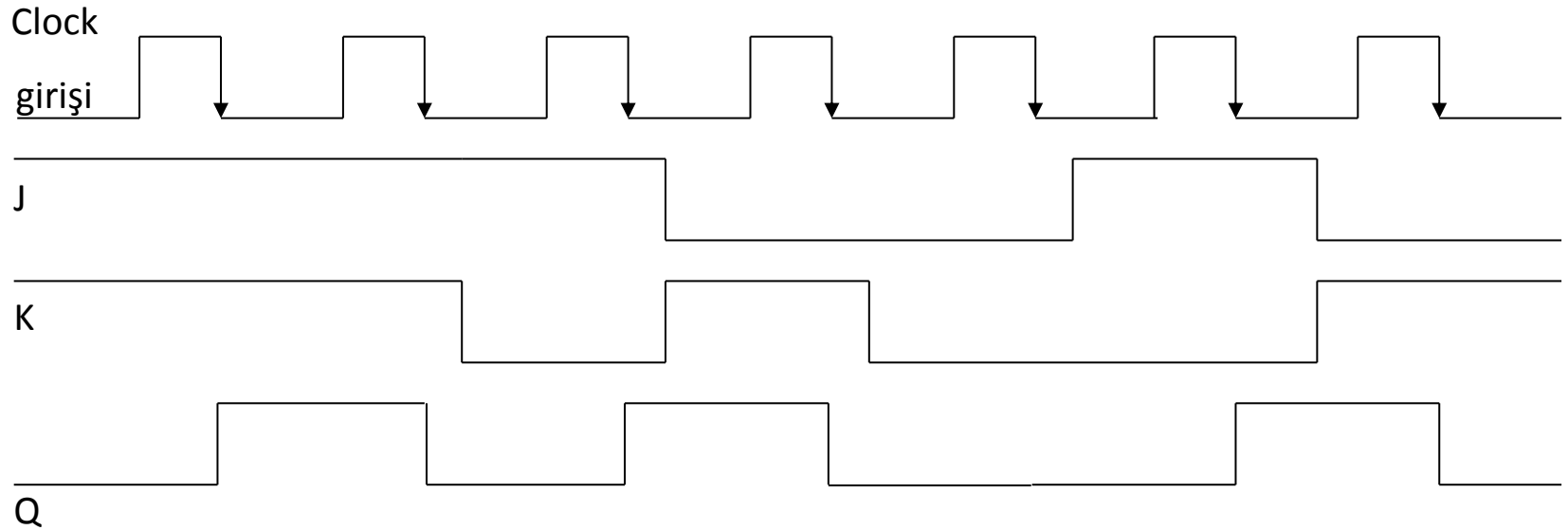
J	K	q	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

J	K	Q
0	0	q
0	1	0
1	0	1
1	1	q'

$$\begin{aligned} Q &= J'K'q + JK' + JKq' \\ &= J'K'q + JK'(q+q') + JKq' \\ &= J'K'q + JK'q + \underline{JK'q'} + JKq' \\ &= \underline{J.q'} + K'.q \text{ (Karakteristik denklemi)} \end{aligned}$$

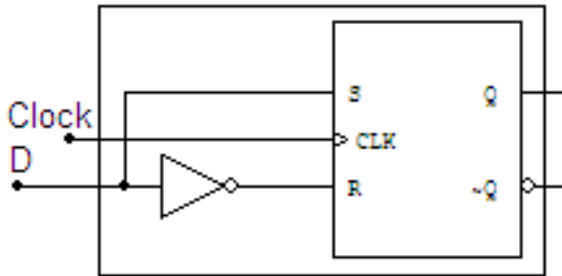
JK tipi flip flop

Negatif kenar tetiklemeli JK flip flozunun davranışını zaman ekseninde inceleyelim; başlangıç durumunda Q'yu 0 alalım.



Flip Flopların Birbirinden Türetilmesi

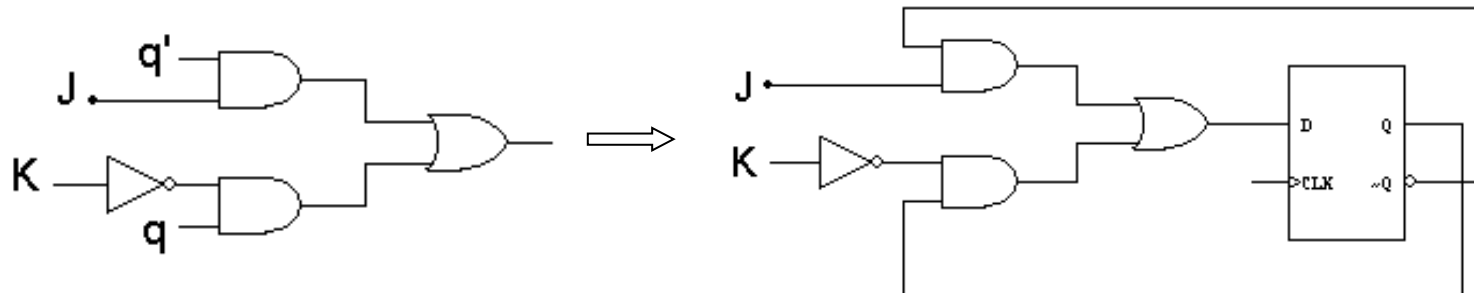
⇒ SR ya da JK flip flobundan D tipi flip flop elde edebiliriz;



$$Q = S + R'.q$$

$$Q = D(q+1) = D + \bar{D}q$$

⇒ JK flip floğunun karakteristik denklemi, $Q = J.q' + K'.q$ olduğundan D flip floğunun girişine bu lojik ifadeyi verirsek, JK flip floğunu elde edebiliriz;

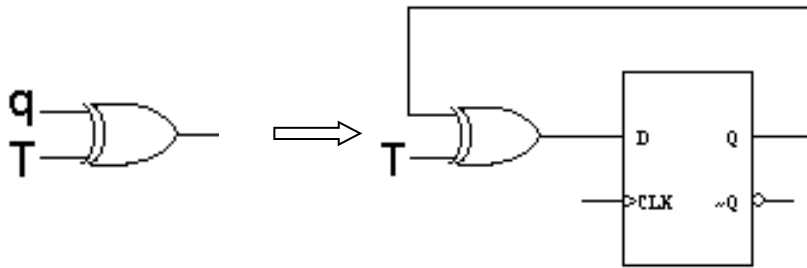


$$Q = D$$

$$Q = J.q' + K'.q$$

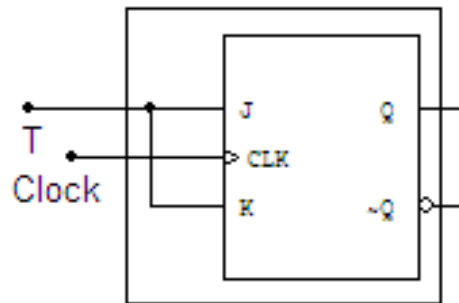
Flip Flopların Birbirinden Türetilmesi

⇒ T flip flobunun karakteristik denklemi $Q = T \oplus q$ olduğundan D flip flobunun girişine bu lojik ifadeyi verirsek, T flip flobunu elde edebiliriz;



$$Q = D$$
$$Q = T \oplus q$$

⇒ JK flip flobunun her iki ucunu birleştirerek T flip flobu elde edebiliriz;



$$Q = T \oplus q = T.q' + T'.q$$

$$Q = J.q' + K'.q$$

Bölüm 1. Ardışıl Devreler

Flip Flopların Uyarma Tablolarının Oluşturulması

Ardışıl Devrelerin Analizi

Ardışıl bir devrenin durum tablosuna bakılarak gerçekleştirilmesi

Flip Flopların Uyarma Tablolarının Oluşturulması

Uyarma tablosu, clock geçişiyle birlikte flip flobun çıkışının değişimini ve bu değişimin olabilmesi için girişlerine ne uygulanması gerektiğini gösteren bir tablodur.

D tipi flip flop için uyarma tablosu: D flip flobunun çıkışının, clock geçişinden önceki D girişine uygulanan değere eşit olduğu ve karakteristik denkleminin de $Q = D$ olduğu söylenmişti. Buna göre D flip flobunun uyarma tablosu aşağıda verilmiştir.

q	Q	D
0	0	0
0	1	1
1	0	0
1	1	1

Flip Flopların Uyarma Tablolarının Oluşturulması

SR tipi flip flop için uyarma tablosu: SR flip flobunun daha önceden elde ettiğimiz doğruluk tablosundan yola çıkarak uyarma tablosunu elde edebiliriz.

S	R	q	Q
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

q	Q	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0



Bu geçişin sağlanabilmesi için için flip flobun ya durumunu koruması ya da reset edilmesi gereklidir.

SR = 00

SR = 01

Öyleyse SR = 0x olmalıdır.

Flip Flopların Uyarma Tablolarının Oluşturulması

T tipi flip flop için uyarma tablosu: T flip flobu, şayet $T=0$ ise mevcut durumunu koruyor, $T=1$ ise mevcut durumunun tersini alıyordu. Bu bilgiden yola çıkarak uyarma tablosunu kolaylıkla oluşturabiliriz.

q Q	T
0 0	0
0 1	1
1 0	1
1 1	0

JK tipi flip flop için uyarma tablosu:

q Q	J K	
0 0	0 x	$\Rightarrow$ Ya JK= 00 ya da JK=01 olmalıdır.
0 1	1 x	$\Rightarrow$ Ya JK= 10 ya da JK=11 olmalıdır.
1 0	x 1	$\Rightarrow$ Ya JK= 01 ya da JK=11 olmalıdır.
1 1	x 0	$\Rightarrow$ Ya JK= 00 ya da JK=10 olmalıdır.

Ardışıl Devrelerin Analizi

Analiz işlemini 3 aşamada yapabiliriz:

1. Çıkışlar ve bir sonraki durumlarla ilgili denklemlerin çıkarılması.
2. Giriş, çıkış ve bir sonraki durumları gösteren durum tablosunun çıkarılması. Bu tablo bir sonraki clock saykılında bellek elemanlarına ne yükleneceğini gösterir.
3. Durum tablosundan da tüm durumları içeren durum diyagramının oluşturulması.

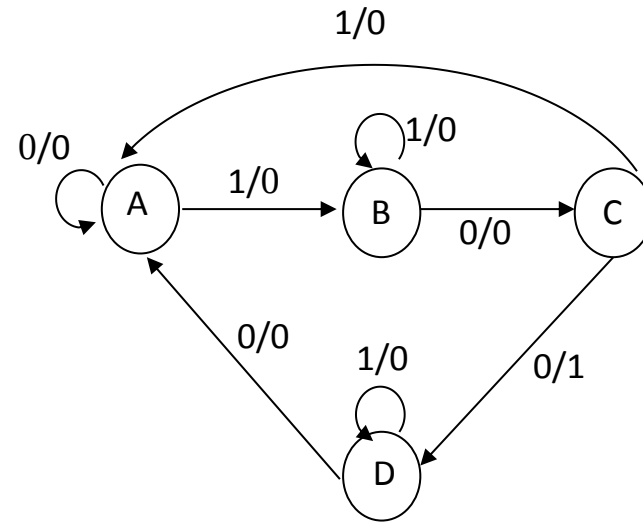
Ardışıl Devrelerin Analizi

1 giriş ve 1 çıkış içeren bir sistem için bu tabirlerin ne manaya geldiği inceleyelim;

Şimdiki Durum	Sonraki Durum		Çıkış (z)	
	x=0	x=1	x=0	x=1
A	A	B	0	0
B	C	B	0	0
C	D	A	1	0
D	A	D	0	0

Durum tablosu

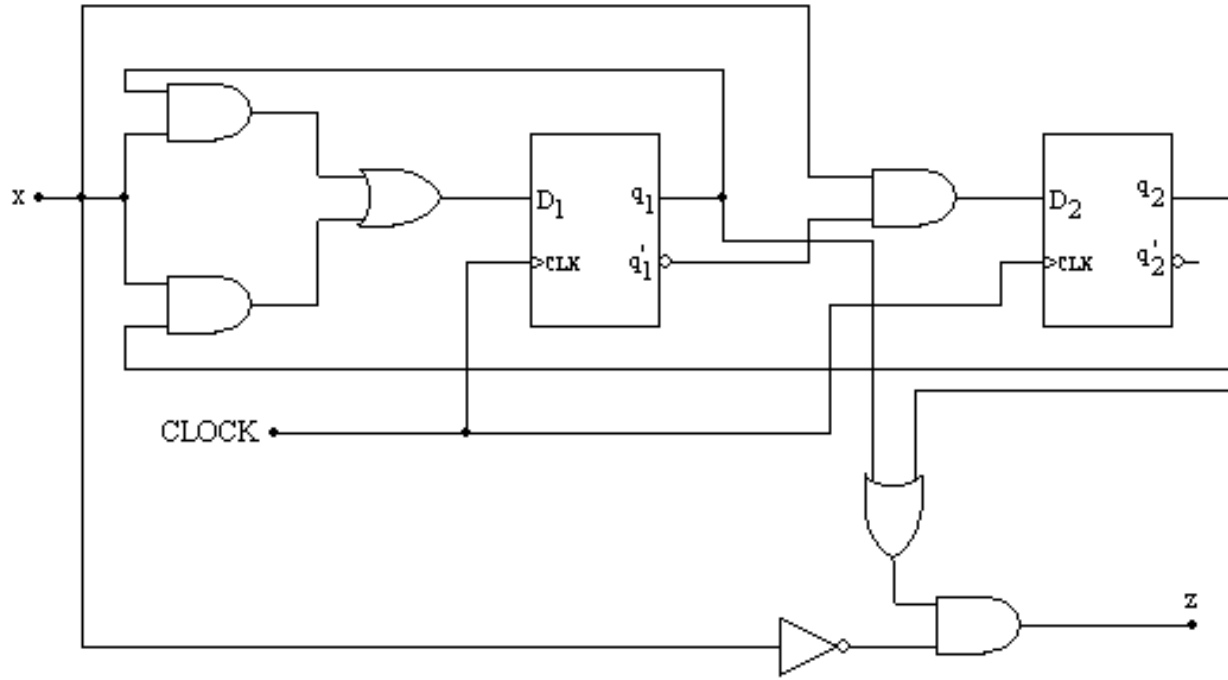
Durumlar



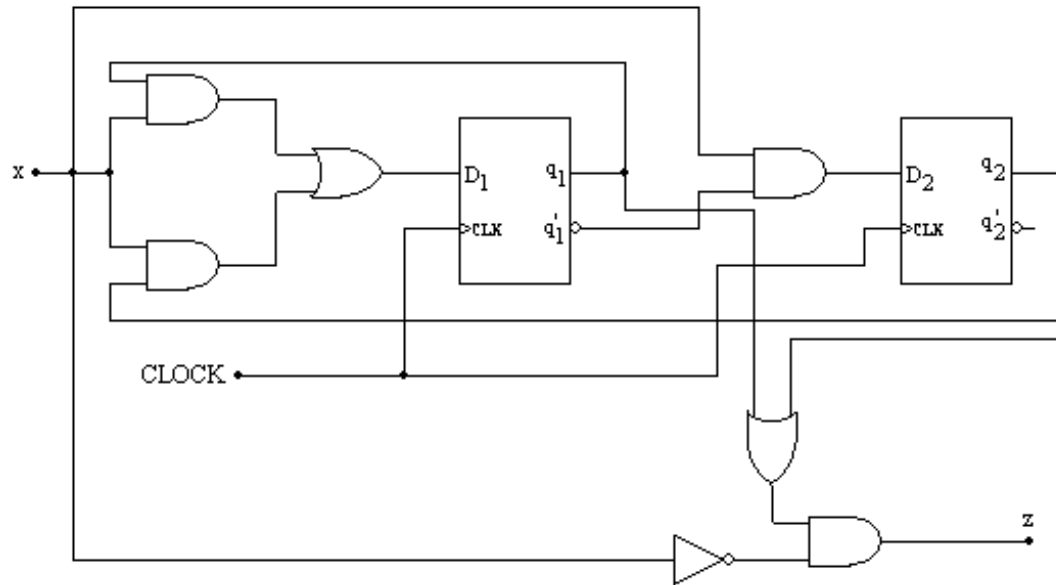
Durum diyagramı

Ardışıl Devrelerin Analizi

Örnek:



Devreye bakarak girişleri, çıkışları ve bellek elemanlarını söyleyebiliriz. Gerek kombinasyonel gerekse ardışıl devrelerde x ile girişler z ile de çıkışlar belirtilir. Bu devre 1 girişe 1 çıkışa ve 2 tane de bellek elemanına sahiptir.



$$D_1 = x.q_1 + x.q_2 = x.(q_1 + q_2)$$

$$D_2 = x.q_1,$$

O halde $Q_1 = x.(q_1+q_2)$ $Q_2 = x.q_1$,

$$z = x' \cdot (q_1 + q_2)$$

Örnek (devamı-2):

İkinci adım, durum denklemlerine bakarak durum tablosunun oluşturulmasıdır;

$$Q_1 = x.(q_1 + q_2)$$

$$Q_2 = x.q_1'$$

$$z = x'.(q_1 + q_2)$$

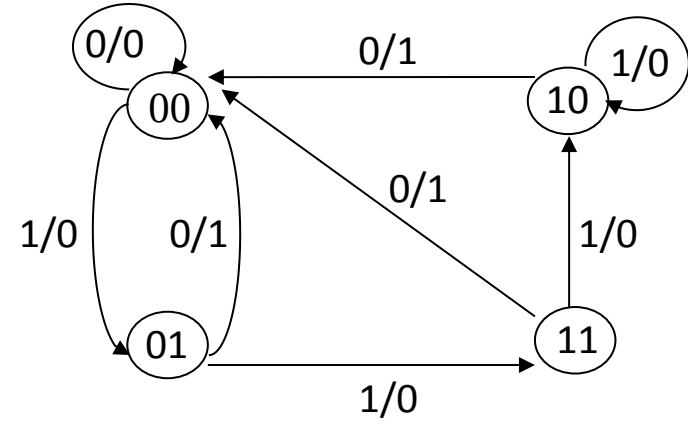
Şimdiki Durum	Sonraki Durum	Çıkış
q_1 q_2 x	Q_1 Q_2	z
0 0 0	0 0	0
0 0 1	0 1	0
0 1 0	0 0	1
0 1 1	1 1	0
1 0 0	0 0	1
1 0 1	1 0	0
1 1 0	0 0	1
1 1 1	1 0	0

Şimdiki Durum	Sonraki Durum (Q_1Q_2)		Çıkış (z)	
q_1 q_2	$x = 0$	$x = 1$	$x = 0$	$x = 1$
(A) 0 0	0 0	0 1	0	0
(B) 0 1	0 0	1 1	1	0
(C) 1 0	0 0	1 0	1	0
(D) 1 1	0 0	1 0	1	0

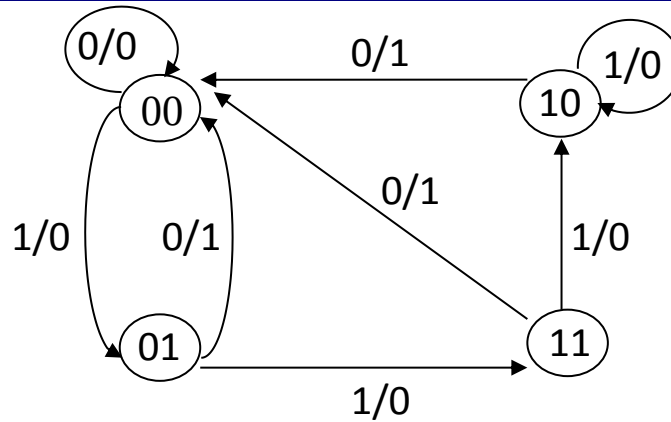
Örnek (devamı-3):

Son adım durum diyagramının çizilmesidir;

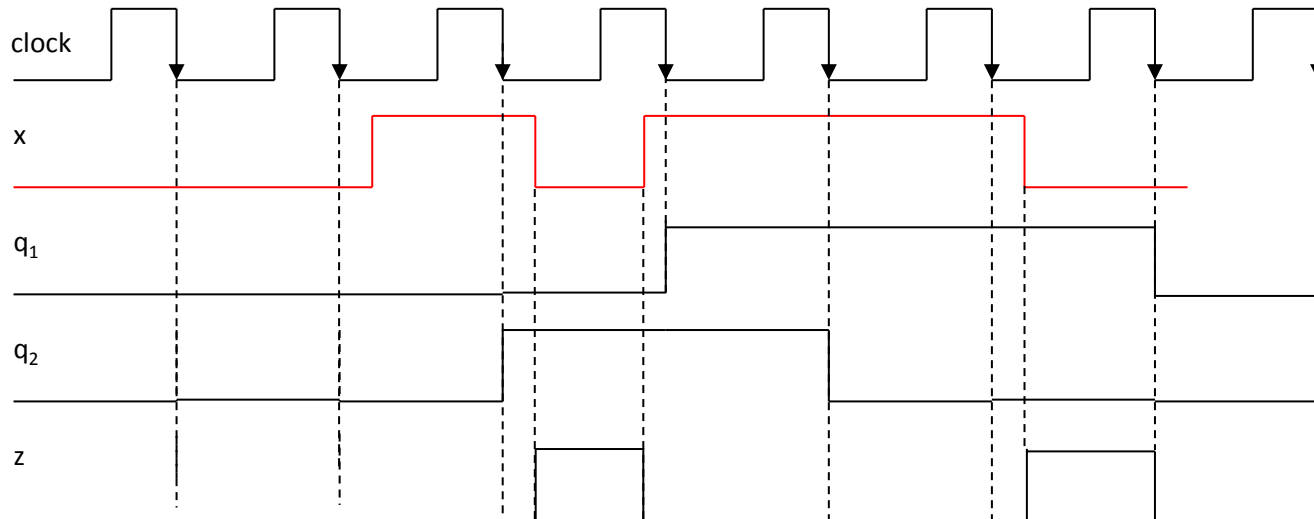
Şimdiki Durum $q_1 q_2$	Sonraki Durum ($Q_1 Q_2$)		Çıkış (z)	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
0 0	0 0	0 1	0	0
0 1	0 0	1 1	1	0
1 0	0 0	1 0	1	0
1 1	0 0	1 0	1	0



Örnek (devamı-4):



Şayet giriş aşağıdaki gibi uygulanırsa devrenin davranışı, zaman ekseninde şu şekilde olur;



Ardışıl Bir Devrenin Durum Tablosuna Bakılarak Gerçeklenmesi

Örnek: Aşağıdaki durum tablosunu, flip floplar kullanarak gerçekleyelim.

Şimdiki Durum	Sonraki Durum		Çıkış (z)	
	x=0	x=1	x=0	x=1
A	A	B	0	0
B	C	B	0	0
C	D	A	1	0
D	A	D	0	0

Durum tablosunda 4 durum olduğundan 2 bellek elemanı (flip flop) gereklidir. Durumları şu şekilde kodlayabiliriz ;

Durumlar	y ₁ y ₂	y ₁ y ₂
A	0 0	0 0
B	0 1	0 1
C	1 0	1 1
D	1 1	1 0

Örnek (devamı-1):

Durumlar	$y_1 y_2$
A	0 0
B	0 1
C	1 0
D	1 1



Şimdiki Durum	Sonraki Durum		Çıkış (z)	
	x=0	x=1	x=0	x=1
A	A	B	0	0
B	C	B	0	0
C	D	A	1	0
D	A	D	0	0



Şimdiki Durum $y_1 y_2$	Sonraki Durum ($Y_1 Y_2$)		Çıkış (z)	
	x=0	x=1	x=0	x=1
0 0	0 0	0 1	0	0
0 1	1 0	0 1	0	0
1 0	1 1	0 0	1	0
1 1	0 0	1 1	0	0

Örnek (devamı-2):

durum tablosu

Şimdiki Durum $y_1 y_2$	Sonraki Durum ($Y_1 Y_2$)		Çıkış (z)	
	$x=0$	$x=1$	$x=0$	$x=1$
0 0	0 0	0 1	0	0
0 1	1 0	0 1	0	0
1 0	1 1	0 0	1	0
1 1	0 0	1 1	0	0

q Q	J K
0 0	0 x
0 1	1 x
1 0	x 1
1 1	x 0

doğruluk tablosu

Şimdiki Durum $x y_1 y_2$	Sonraki Durum $Y_1 Y_2$	Uyarma İşlevleri				Çıkış z
		J_1	K_1	J_2	K_2	
0 0 0	0 0	0	x	0	x	0
0 0 1	1 0	1	x	x	1	0
0 1 0	1 1	x	0	1	x	1
0 1 1	0 0	x	1	x	1	0
1 0 0	0 1	0	x	1	x	0
1 0 1	0 1	0	x	x	0	0
1 1 0	0 0	x	1	0	x	0
1 1 1	1 1	x	0	x	0	0



Örnek (devamı-3):

Şimdiki Durum $x y_1 y_2$	Sonraki Durum $Y_1 Y_2$	Uyarma İşlevleri $J_1 K_1 \quad J_2 K_2$		Çıkış z
0 0 0	0 0	0 x	0 x	0
0 0 1	1 0	1 x	x 1	0
0 1 0	1 1	x 0	1 x	1
0 1 1	0 0	x 1	x 1	0
1 0 0	0 1	0 x	1 x	0
1 0 1	0 1	0 x	x 0	0
1 1 0	0 0	x 1	0 x	0
1 1 1	1 1	x 0	x 0	0

J_1				
$y_1 y_2$	00	01	11	10
x	0	1	x	x
0	0	1	x	x
1	0	0	x	x

$$J_1 = x'y_2$$

K_1				
$y_1 y_2$	00	01	11	10
x	x	x	1	0
0	x	x	1	0
1	x	x	0	1

$$K_1 = xy_2' + x'y_2 = x \oplus y_2$$

J_2				
$y_1 y_2$	00	01	11	10
x	0	x	x	1
0	0	x	x	1
1	1	x	x	0

$$J_2 = xy_1' + x'y_1 = x \oplus y_1$$

K_2				
$y_1 y_2$	00	01	11	10
x	x	1	1	x
0	x	1	1	x
1	x	0	0	x

$$K_2 = x'$$

Doğruluk tablosundan;

$$z = x'y_1y_2'$$

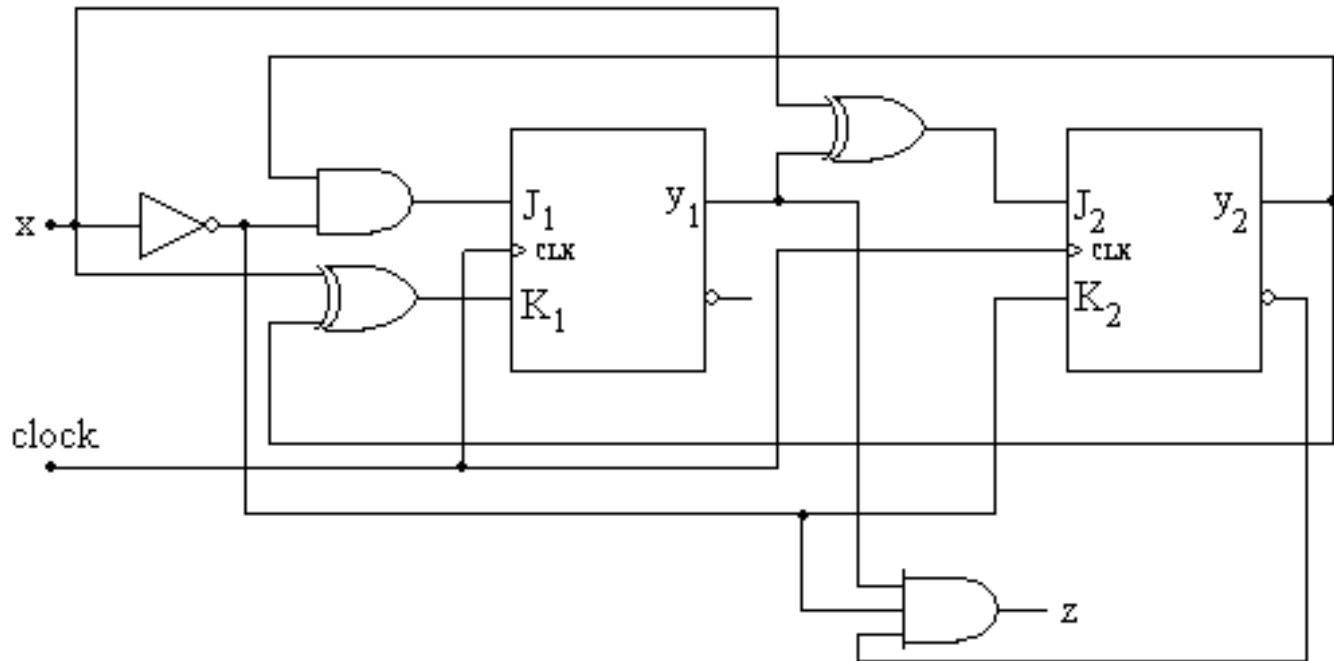
Örnek (devamı-4):

$$J_1 = x'y_2$$

$$K_1 = xy_2' + x'y_2 = x \oplus y_2$$

$$J_2 = xy_1' + x'y_1 = x \oplus y_1$$

$$K_2 = x'$$



Bölüm 1. Ardışıl Devreler

Geçen Hafta

Flip Flopların Uyarma Tablolarının Oluşturulması

Ardışıl Devrelerin Analizi

Ardışıl bir devrenin durum tablosuna bakılarak gerçekleştirilmesi

Bu Hafta

Senkron Ardışıl Devre Tasarımının Adımları

Senkron Ardışıl Devre Tasarımının Adımları

1. Problemin sözlü tanımından hangi durumların olacağına ya da bellek elemanlarına hangi değerlerin atanacağına karar verilir.
2. Sistemin davranışını modelleyebilmek için durum tablosu ve durum diyagramı oluşturulur.
3. Flip flop seçiminin yapılması (belirtilmemişse) ve uyarma tablolarının oluşturulması.
4. Çıkış ve uyarma tablolarından, lojik ifadelerin elde edilmesi ve devrenin çizimi.

Senkron Ardışıl Devre Tasarımının Adımları

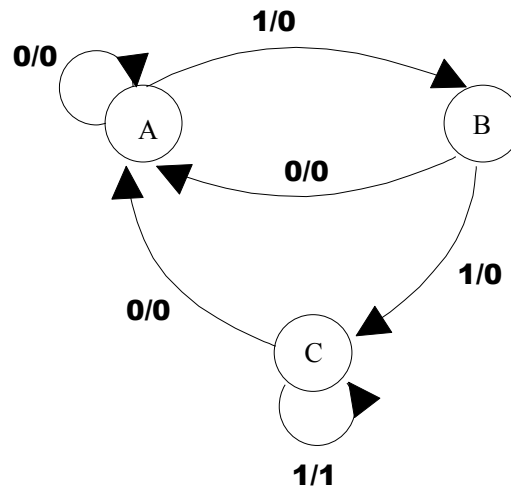
Örnek: 1 giriş ve 1 çıkışa sahip Mealy tipi bir sistemde, son 3 giriş verisi 1 ise çıkışın 1 olması isteniyor.

Bu problemin çözümünde 3 durum vardır; bu durumlar bellek elemanlarında saklanır.

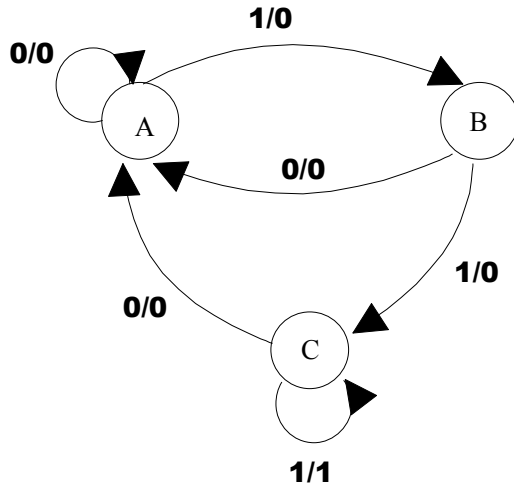
A durumu, girişin 1 olmadığı durumu,

B durumu, girişin bir defa 1 olduğu durumu,

C durumu, girişin iki ya da daha fazla 1 olduğu durumu gösterir.



Örnek (devamı-1):



Durum diyagramı

Şimdiki Durum q	Sonraki Durum(Q)		Çıkış (z)	
	x=0	x=1	x=0	x=1
A	A	B	0	0
B	A	C	0	0
C	A	C	0	1

Devrenin durum tablosu

Durum tablosu 3 durum içerdiğinden 2 tane flip flop kullanmak gerekir.

A durumuna 00, B durumuna 01 ve C durumuna 10 atayalım.

Şimdiki Durum q ₁ q ₂	Sonraki Durum(Q ₁ Q ₂)		Çıkış (z)	
	x=0	x=1	x=0	x=1
0 0	0 0	0 1	0	0
0 1	0 0	1 0	0	0
1 0	0 0	1 0	0	1

Örnek (devamı-2):

Devremizde bellek elemanı olarak JK tipi flip floplar kullanacağımızı farz edelim. Bundan sonraki adım, flip flopların uyarma işlevlerini bulmaktır.

Şimdiki Durum q_1q_2	Sonraki Durum(Q_1Q_2) $x=0$ $x=1$	Uyarma İşlevleri				Çıkış (z) $x=0$ $x=1$	
	$x=0$	$x=1$	$x=0$ için J_1K_1 J_2K_2	$x=1$ için J_1K_1 J_2K_2			
0 0	0 0	0 1	0 x 0 x	0 x 1 x		0	0
0 1	0 0	1 0	0 x x 1	1 x x 1		0	0
1 0	0 0	1 0	x 1 0 x	x 0 0 x		0	1



q Q	J K
0 0	0 x
0 1	1 x
1 0	x 1
1 1	x 0

Örnek (devamı-3):

Uyarma işlevlerindeki her bir sütunu, Karnaugh haritasına taşıyıp indirgeme işlemlerini yapalım;

Şimdiki Durum q_1q_2	Sonraki Durum(Q_1Q_2)		Uyarma İşlevleri				Çıkış (z)	
	x=0	x=1	x=0 için $J_1K_1 \quad J_2K_2$		x=1 için $J_1K_1 \quad J_2K_2$		x=0	x=1
0 0	0 0	0 1	0 x	0 x	0 x	1 x	0	0
0 1	0 0	1 0	0 x	x 1	1 x	x 1	0	0
1 0	0 0	1 0	x 1	0 x	x 0	0 x	0	1

q_1q_2 x \	00	01	11	10
0			x	x
1		1	x	x

$$J_1 = x.q_2$$

q_1q_2 x \	00	01	11	10
0		x	x	
1	1	x	x	

$$J_2 = x.q_1'$$

q_1q_2 x \	00	01	11	10
0	x	x	x	1
1	x	x	x	

$$K_1 = x'$$

q_1q_2 x \	00	01	11	10
0	x	1	x	x
1	x	1	x	x

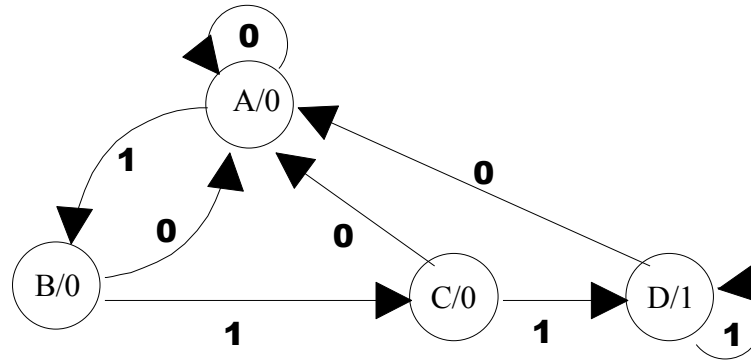
$$K_2 = 1$$

q_1q_2 x \	00	01	11	10
0			x	
1			x	1

$$z = x.q_1$$

Senkron Ardışıl Devre Tasarımının Adımları

Örnek: Ard arda gelen 3 clock saykılında giriş 1 ise çıkışın 1 olmasını sağlayacak Moore türü devre tasarlanması isteniyor.

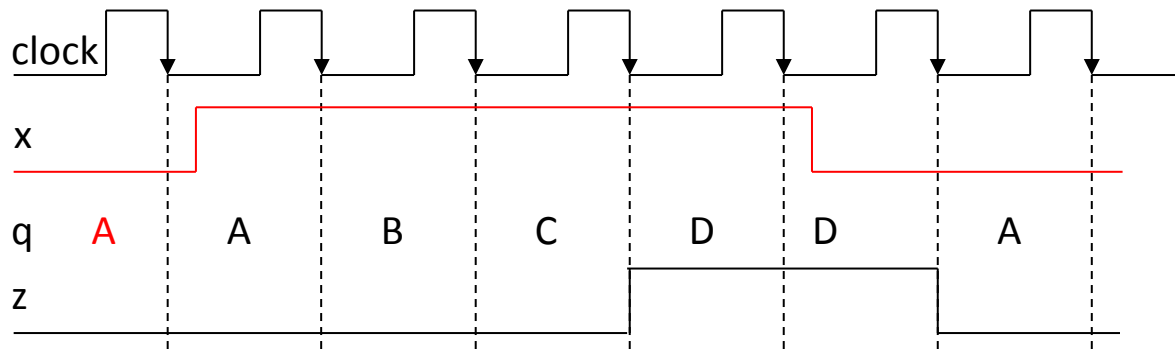


Sistemin başlangıçta A durumunda olduğunu farz ederek verilen x girişi için ardışıl devrenin durum geçişlerini ve çıkışı inceleyelim;

	A	B	C	D	D	D	D	A	B	C	A	B	C	D	A	B
x	0	1	1	1	1	1	1	0	1	1	0	1	1	1	0	1
z	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	1

Örnek (devamı-1):

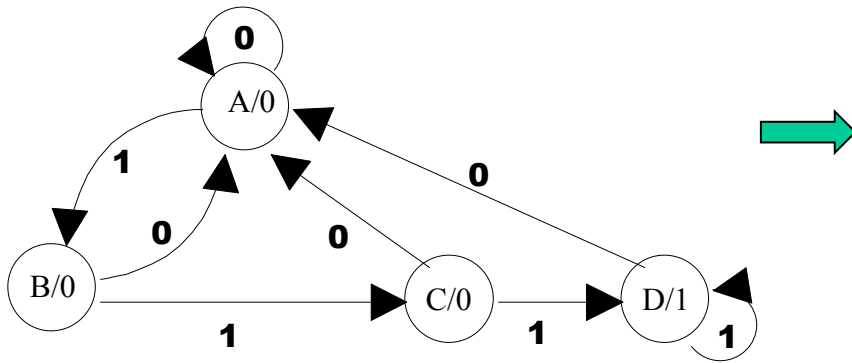
Ardışıl devrenin davranışını verilen x girişi için zaman ekseninde inceleyelim;



Örnek (devamı-2):

Şimdi de A,B,C ve D durumlarına durum ataması yapıp, uyarma işlevlerini de içeren durum tablosunu oluşturalım;

A=00, B=01, C=10, D=11 durumlarını atayalım.



Şimdiki Durum x q ₁ q ₂	Sonraki Durum Q ₁ Q ₂	Uyarma İşlevleri		Çıkış z
		J ₁ K ₁	J ₂ K ₂	
0 0 0	0 0	0 x	0 x	0
0 0 1	0 0	0 x	x 1	0
0 1 0	0 0	x 1	0 x	0
0 1 1	0 0	x 1	x 1	1
1 0 0	0 1	0 x	1 x	0
1 0 1	1 0	1 x	x 1	0
1 1 0	1 1	x 0	1 x	0
1 1 1	1 1	x 0	x 0	1

Örnek (devamı-3):

Uyarma işlevlerindeki her bir sütun, Karnaugh haritası yardımıyla indirgendiğinde, flip flopların girişlerine uygulanması gereken lojik ifadeler ve çıkış aşağıdaki gibi olacaktır;

Şimdiki Durum $x \ q_1 \ q_2$	Sonraki Durum $Q_1 \ Q_2$	Uyarma İşlevleri		Çıkış z
		$J_1 \ K_1$	$J_2 \ K_2$	
0 0 0	0 0	0 x	0 x	0
0 0 1	0 0	0 x	x 1	0
0 1 0	0 0	x 1	0 x	0
0 1 1	0 0	x 1	x 1	1
1 0 0	0 1	0 x	1 x	0
1 0 1	1 0	1 x	x 1	0
1 1 0	1 1	x 0	1 x	0
1 1 1	1 1	x 0	x 0	1

Karnaugh



$$J_1 = q_2 \cdot x$$

$$K_1 = x'$$

$$J_2 = x$$

$$K_2 = x' + q_1'$$

$$z = q_1 \cdot q_2$$

Devremiz Moore türü olduğundan dolayı, çıkış sadece durumlara bağlıdır. D durumunda çıkış 1 olduğundan ve D durumuna da 11 atadığımızdan dolayı $z = q_1 \cdot q_2$ olmuştur

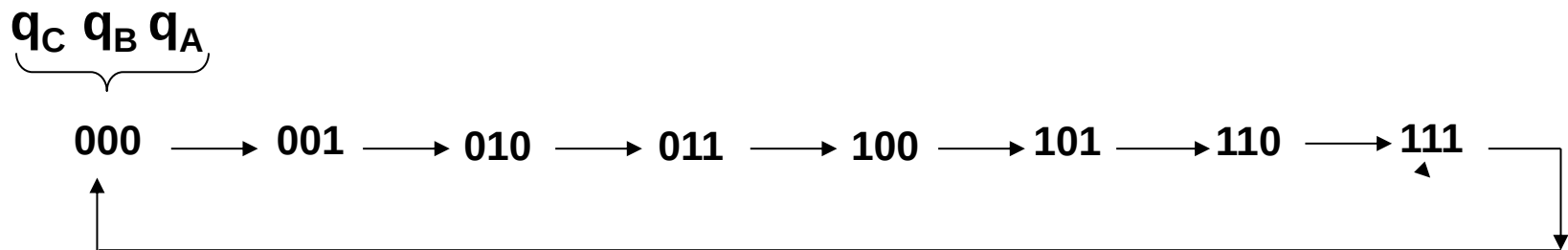
Senkron Ardışıl Devre Tasarımının Adımları

Örnek: Senkron Sayıcı Tasarımı

Sayıcılı devrelerinin genellikle girişleri yoktur; saat geçişiyle durumlar arasında geçiş sağlanır. Çıkışlar, sistemin durumlarıdır, başka bir ifadeyle flip flopların tuttuğu değerlerdir. Bir sayıcının hem ileri hem de geri sayması istenirse, bu işlev için bir giriş kullanılması gerekir.

3 bit ikili bir sayıcının tasarım aşamaları:

Bu sayıcı 0 ile 7 arasında ileri yönde sayma yapacağından 3 adet flip flop kullanılacaktır. Bu flip flopların isimleri q_C (MSB), q_B ve q_A (LSB) olsun.



Örnek (devamı-1):

Şimdiki Durum $q_C q_B q_A$	Sonraki Durum $Q_C Q_B Q_A$
0 0 0	0 0 1
0 0 1	0 1 0
0 1 0	0 1 1
0 1 1	1 0 0
1 0 0	1 0 1
1 0 1	1 1 0
1 1 0	1 1 1
1 1 1	0 0 0

Şimdiki Durum $q_C q_B q_A$	Sonraki Durum $Q_C Q_B Q_A$	Uyarma İşlevleri		
		$J_C K_C$	$J_B K_B$	$J_A K_A$
0 0 0	0 0 1	0 x	0 x	1 x
0 0 1	0 1 0	0 x	1 x	x 1
0 1 0	0 1 1	0 x	x 0	1 x
0 1 1	1 0 0	1 x	x 1	x 1
1 0 0	1 0 1	x 0	0 x	1 x
1 0 1	1 1 0	x 0	1 x	x 1
1 1 0	1 1 1	x 0	x 0	1 x
1 1 1	0 0 0	x 1	x 1	x 1

Uyarma işlevleri

Karnaugh ile indirgendiginde;

$$J_C = K_C = q_B \cdot q_A$$

$$J_B = K_B = q_A$$

$$J_A = K_A = 1$$



4 bitlik bir sayıcı yapmak isteseydik;

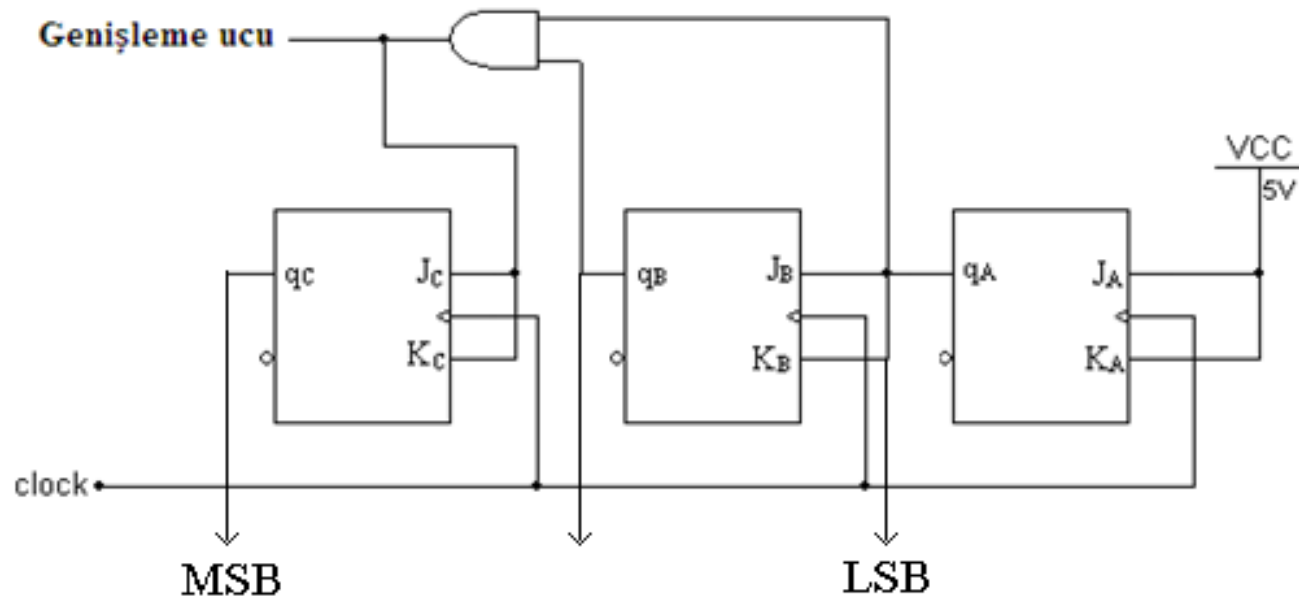
$J_D = K_D = q_C \cdot q_B \cdot q_A$ olacağı tahmin edilebilir.

Örnek (devamı-2):

$$J_C = K_C = q_B \cdot q_A$$

$$J_B = K_B = q_A$$

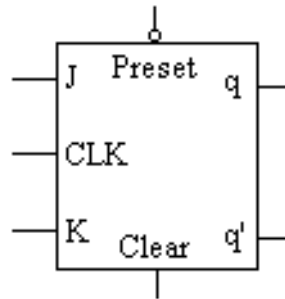
$$J_A = K_A = 1$$



Sayıc11 tasarımı1n1n devre řemas1

NOTLAR:

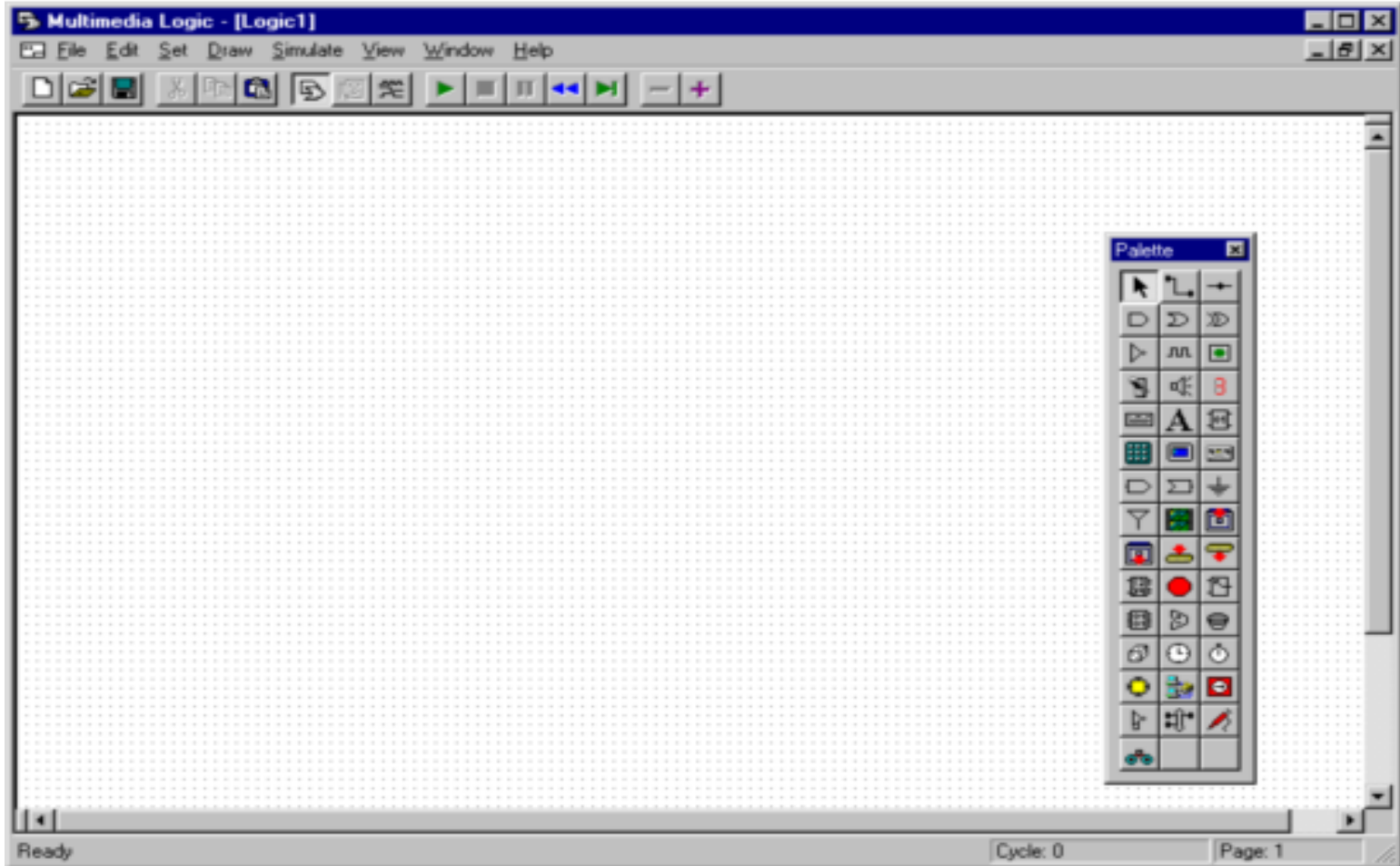
1. İleri yönde sayan sayıcı devrelerin q çıkışları yerine q' çıkışlarından uç alınması durumunda geri yönde sayan sayıcı elde edilir (tersi de geçerlidir).
2. Şayet sayıcı uygulaması sıralı değilse, tasarım aşamaları yine aynıdır. Ancak kullanılmayan durumlar varsa, sonraki durum önemsiz durum olarak ele alınacağından, devrenin böyle bir durumdan başlaması halinde, bir sonraki durumu tahmin etmek mümkün değildir (bu durumu önlemek için flip flopların clear ve preset girişleri vardır).



Bölüm 2. MULTIMEDIA LOGIC SİMÜLATÖR PROGRAMI

MULTIMEDIA LOGIC SİMÜLATÖR PROGRAMI

Bu program tamamen açık kaynak kodlu olup www.softronix.com/logic.html adresinden temin edilebilir.



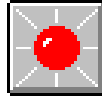
Bağlantı Noktalarının Durumları

Multimedia Logic programında bir node (bağlantı noktası)'un 3 değişik durumu vardır. Bağlantı noktalarındaki değeri öğrenmek için ilk alternatif, bağlantı noktasına bir led (gösterge) bağlamaktır. Led'de beliren durum, aşağıdakilerden biri olabilir:

LO (off veya False)



HI (on veya True)



UNKNOWN (Ne True ne de False)



Diğer bir alternatif ise; programın araç çubuğunda yer alan  simgesine tıklanarak bağlantı noktasına gelindiğinde, farenin sol tuşu basılı tutulduğunda *toggle probe* adlı bu simgenin göstergesinden okumaktır.

Çalışma Modları

Multimedia Logic Simülatör programı farklı şekillerde çalıştırılabilir.



Bu buton yardımıyla program tasarım moduna geçer. Bu moddayken program, üzerinde devre tasarımları yapılmasına imkan verir. Yani palet görüntülenir.



Tasarlanan devreyi çalıştırmaya yarar.



Çalıştırılan devreyi durdurarak başlangıç konumuna getirmeye yarar.

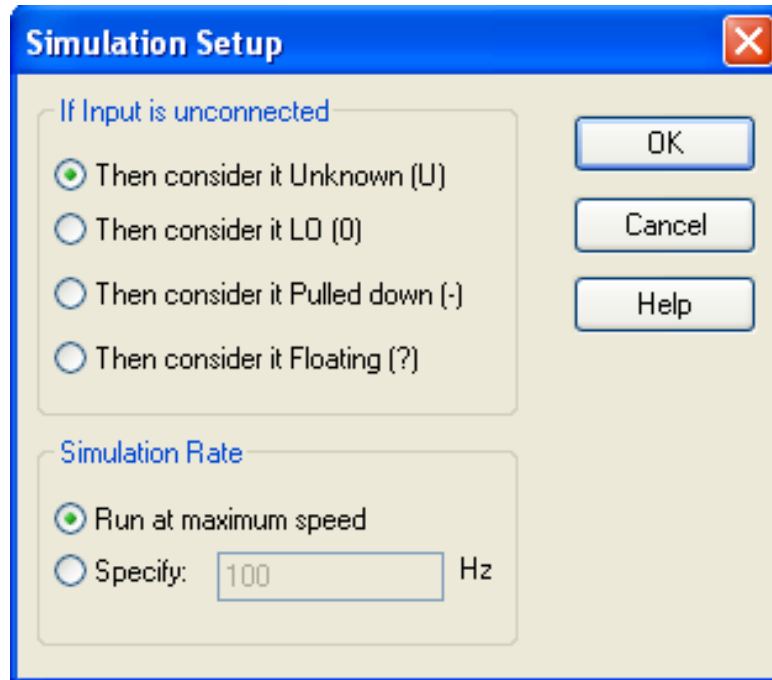


Çalışan devreyi başlangıç durumuna döndürerek devrenin yeniden çalıştırılmasına neden olur.

Multimedia Logic Program Menüleri

Simulate menüsü

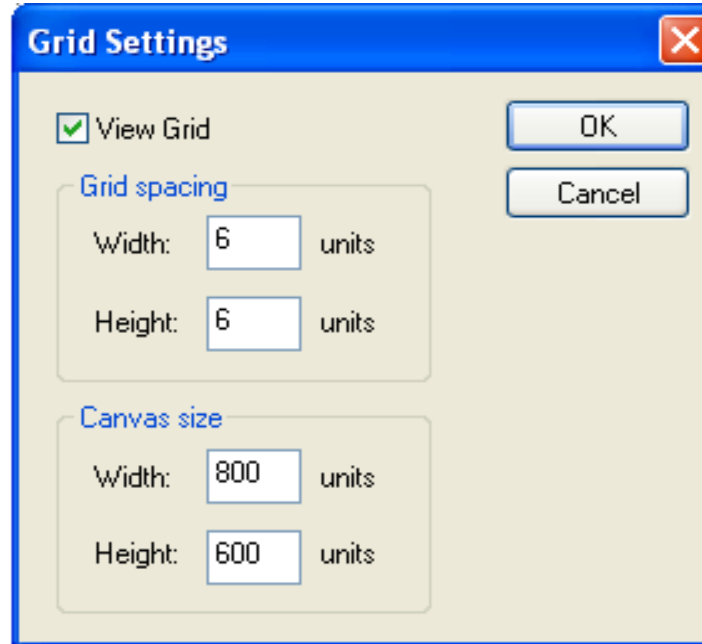
Setup: Simülasyon hızının ayarlanabildiği bir yerdir. Aşağıda şekilde görüldüğü gibi iki seçenek mevcuttur. Simülasyon ya bilgisayarın müsaade ettiği maksimum hızda ya da belirlenen bir hızda (çalışma frekansı) çalışacaktır.



Multimedia Logic Program Menüleri

View menüsü

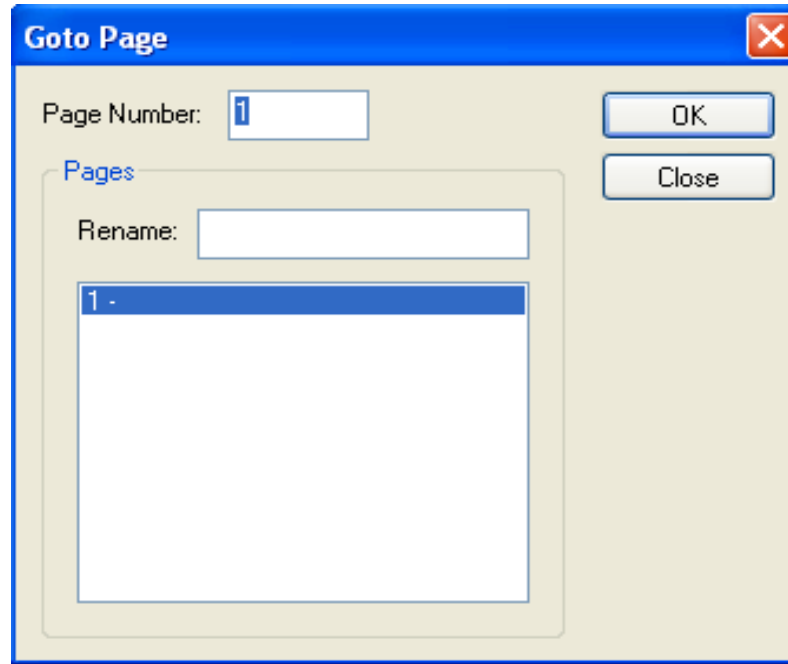
Grid Settings: Aşağıdaki ekran çıktısında da görüldüğü gibi çalışma alanındaki grid noktalarının genişliği, yüksekliği ve en önemlisi tasarım yapılan alanın boyutu değiştirilebilir. Tasarım alanının boyutu en fazla 4096 olarak girilebilir.



Multimedia Logic Program Menüleri

View menüsü

Goto Page: Eğer devre tasarımı birden fazla sayfadan oluşuyorsa, istenilen sayfaya gidilmesine olanak tanır. Ayrıca sayfalar isimlendirildiği takdirde, çok sayıda sayfadan oluşan devrenin istenilen sayfasına gitmek daha kolay olacaktır.



Palet Bileşenleri

Seçim aracı (Selector)

Çalışma alanı içerisindeki elemanların konumunu ayarlamaya yarar. Ayrıca bu simge seçili durumdayken, devre elemanı üzerinde çift tıklama yapılırsa, bu devre elemanı ile ilgili bir iletişim kutusu açılır.

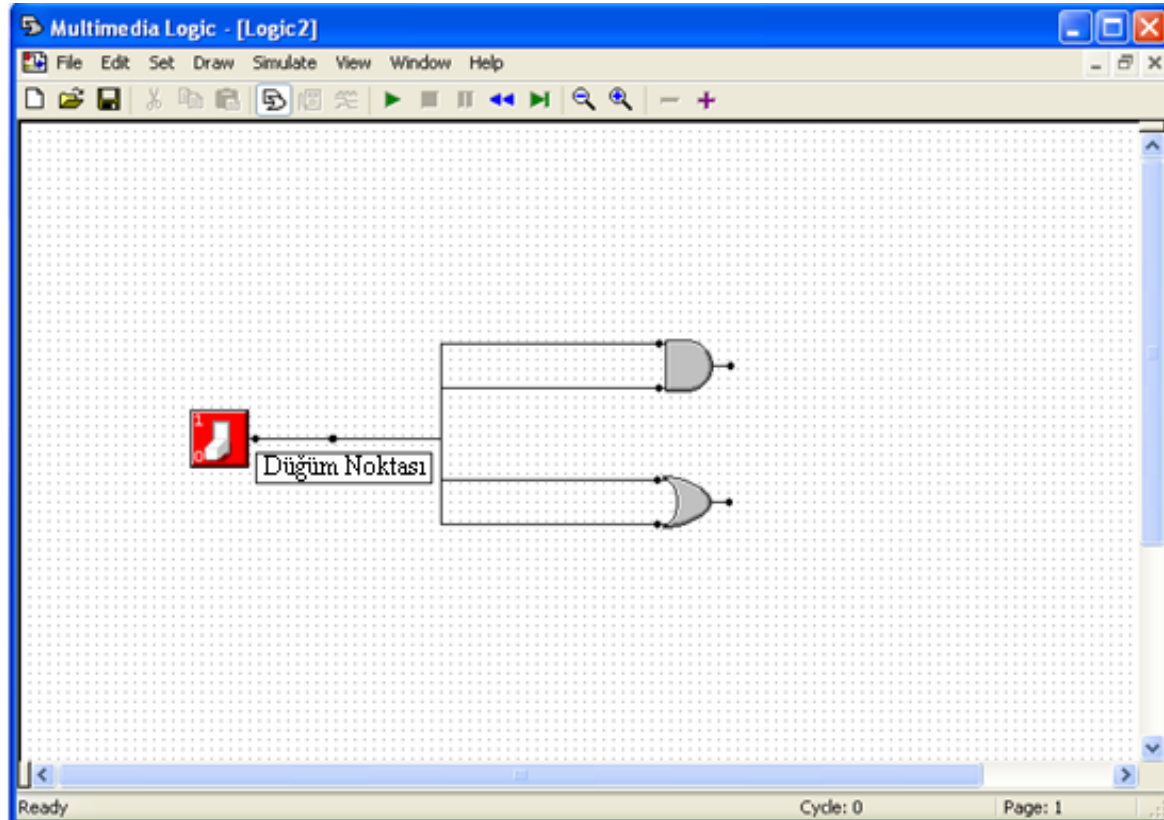
Bağlantı aracı (Wire)

İki elemanı birbirine bağlamaya yarar. İki eleman arasında bağlantı yapılmaya çalışılırken dikkat edilmesi gereken nokta; bir elemanın çıkış noktasından diğer elemanın giriş noktasına bağlantı yapılması gerektiğidir. Bir elemanın çıkışından, birden fazla elemanın girişine bağlantı yapılabileceği, tam tersi olan bir elemanın girişine birden fazla elemanın çıkışının bağlanamayacağı göz önünde bulundurulmalıdır. İki eleman arasındaki bağlantıyı gerçekleştirmek için ilk olarak bağlantı aracı seçilir. Daha sonra, bağlantısı yapılacak olan elemanın bağlantı noktasına farenin imleci getirilir ve sol tuş basılı tutularak diğer elemanın bağlantı noktasının üzerine gelince tuş bırakılır.

Palet Bileşenleri

Düğüm noktası (Node)

Daha estetik kablolama yapmaya imkan tanır. Bu elemana sadece bir elemanın çıkışı bağlanabilirken, çok sayıda elemanın girişine bağlanabilme özelliğine sahiptir.



Palet Bileşenleri

VE kapısı (AND)



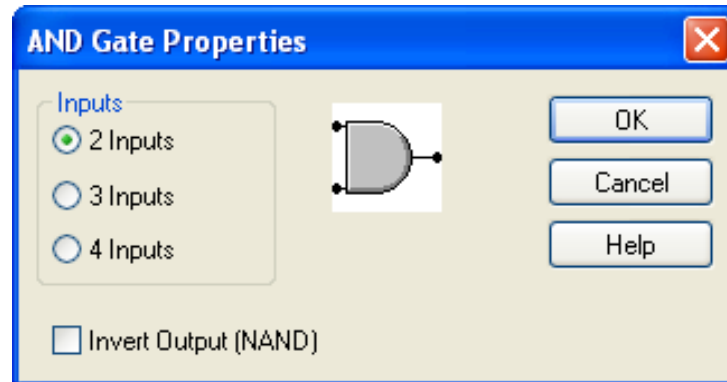
VEYA kapısı (OR)



XOR kapısı



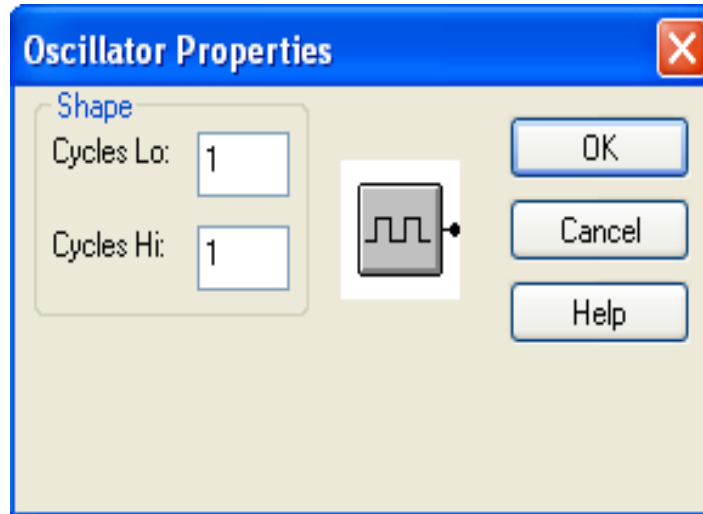
Değil (NOT) kapısı



Palet Bileşenleri

Osilatör aracı

Özellikle bellek elemanları için saat sinyali (clock) üretmeye yarar. İletişim kutusundan ne kadar süre ile lojik 0 ve ne kadar süre ile de lojik 1 seviyesinde kalacağı ayarlanır. Simülasyon hızı ile bağlantısı vardır. Örneğin simülasyon hızı 10 Hz. ise ve osilatör aracının iletişim kutularına da 5 girilmişse, saniyede bir clock sinyali üretilir.

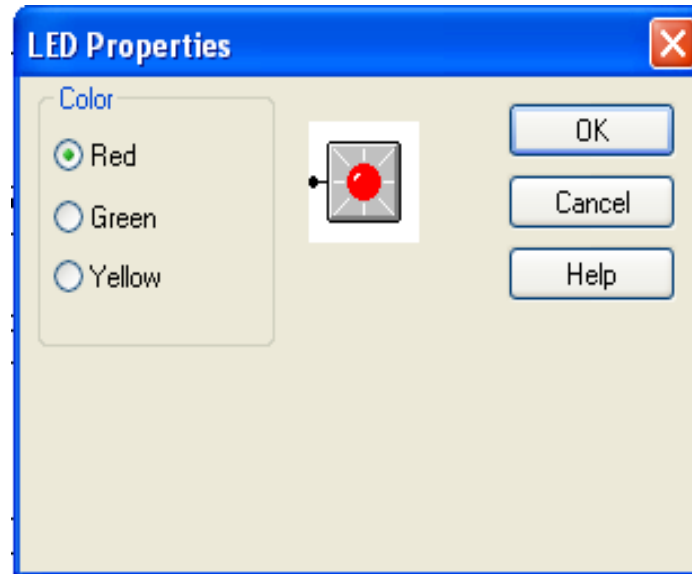


Palet Bileşenleri

Led aracı



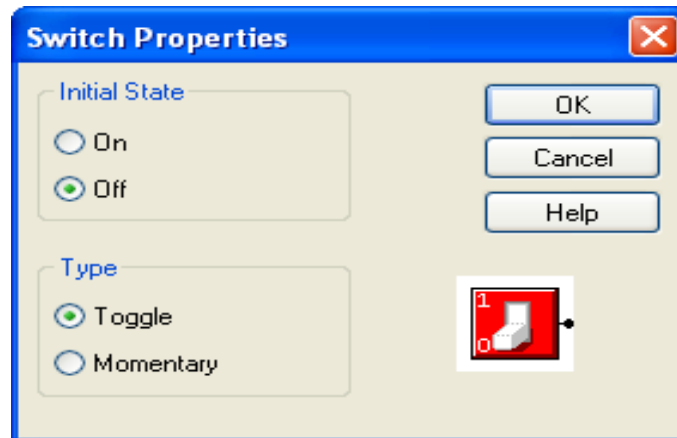
Devrede yer alan herhangi bir bağlantı noktasına bağlanabilir. Çıkış olarak 0 (off),1 (on) ve U (unknown) durumlarına sahiptir. Ayrıca iletişim kutusundan kırmızı, yeşil veya sarı olarak ayarlanabilir.



Palet Bileşenleri

Anahtar (Switch)

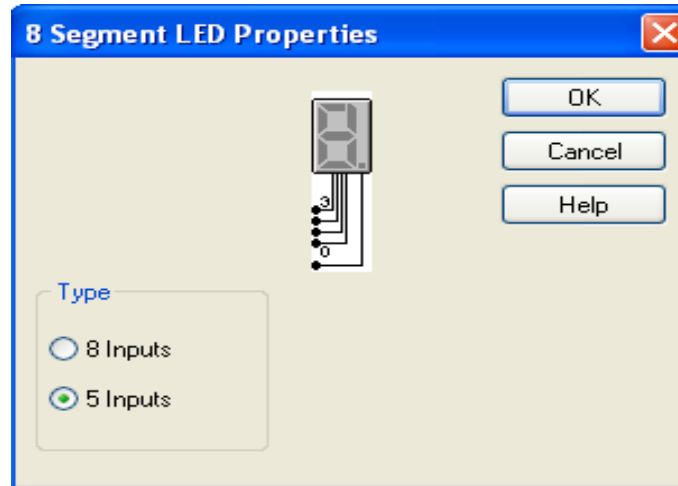
Anahtar elemanı 2 durumlu bir giriş elemanıdır. Simülasyon çalışmaya başladığında bağlı olduğu elemana lojik 0 veya lojik 1 sinyalini gönderir. İletişim kutusundan başlangıç durumuna lojik 0 ya da lojik 1 atanabilir. *Toggle* ve *Momentary* olmak üzere iki tip anahtar vardır. Toggle modda, bağlanmış olduğu elemana ya 1 ya da 0 sinyalini gönderir, Momentary modda ise anlık olarak devreye ilk durumuna bağlı olarak 0 veya 1 sinyalini gönderir ve ilk durumuna geri döner (push buton olarak çalışır).



Palet Bileşenleri

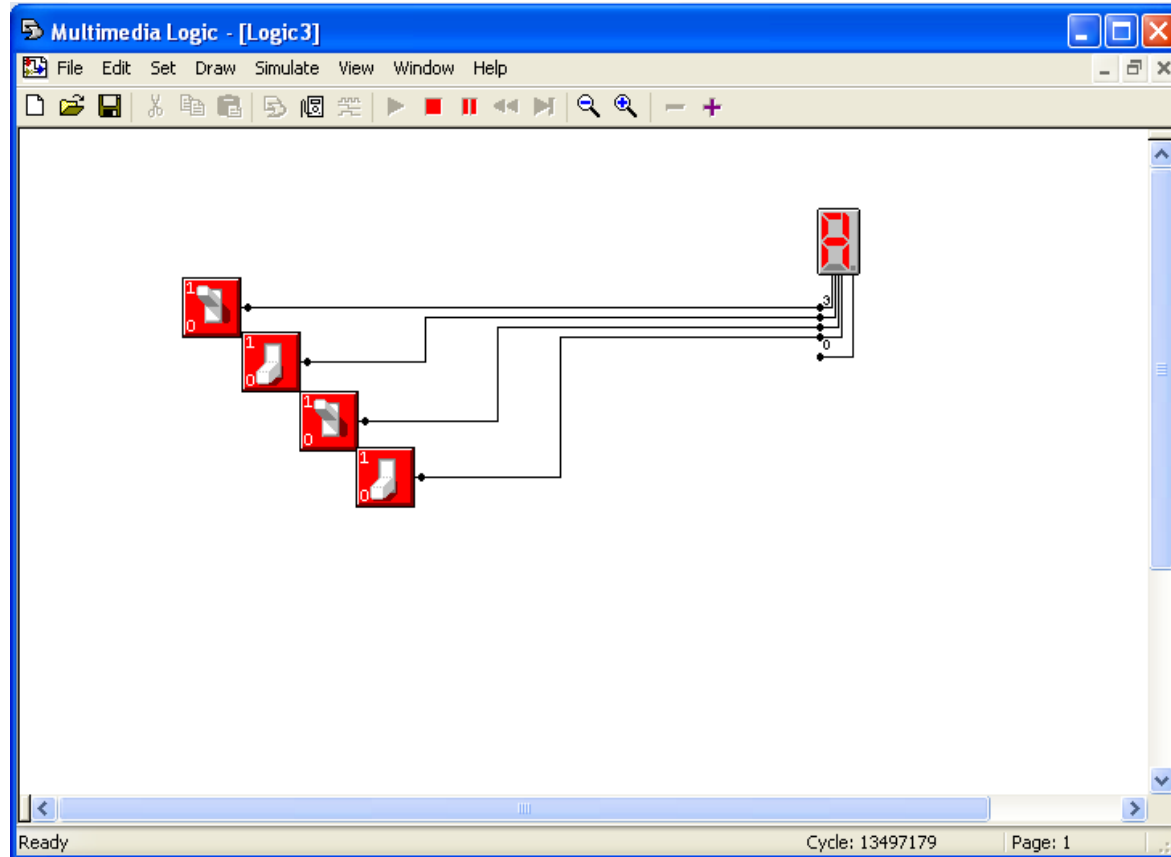
Yedi segment LED

Girişindeki sinyallere bağlı olarak, 0 ile F arasındaki hexadecimal sayıları görüntüler. 5 ve 8 girişli olmak üzere iki çeşidi vardır. 5 girişli modunda 4 bit, görüntülenecek olan sayının ikili değeridir, 1 bit ise noktayı temsil eder. 8 girişli modda ise 7 segmentin ve bir de noktanın on ya da off durumunda olacağı, 8 adet giriş ile belirtilir.



Palet Bileşenleri

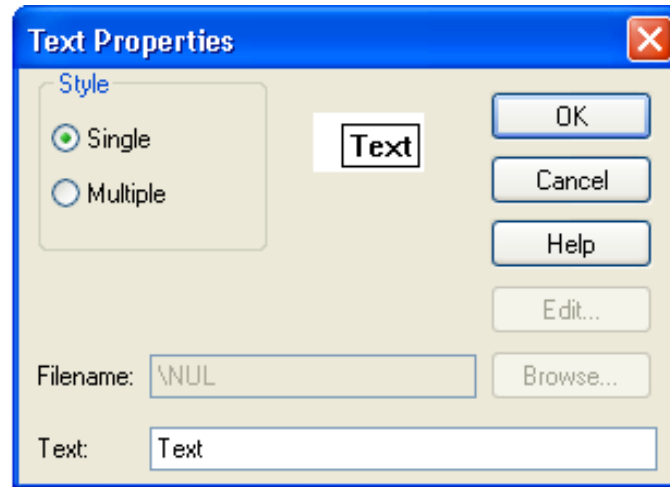
Yedi segment LED



Palet Bileşenleri

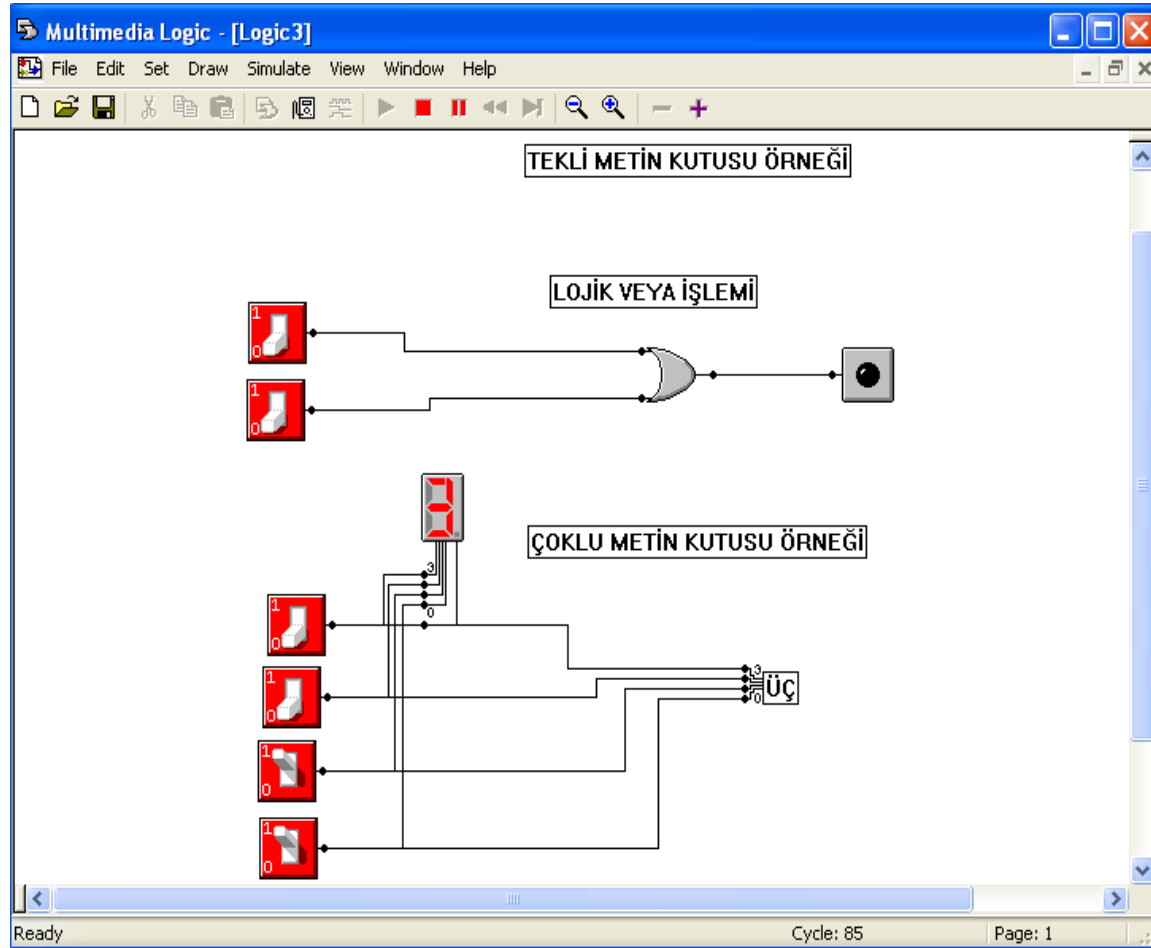
Metin kutusu

Tekli ve çoklu olmak üzere iki çeşidi vardır. Tekli metin kutusu, devrenin gerekli yerlerine açıklama ihtiyacı olduğunda faydalı bir elemandır. Çoklu metin kutusu ise devrenin durumuna göre 16 farklı şekilde çıktı üretebilen bir elemandır. Sahip olduğu 4 girişe uygulanan değere göre, *txt* uzantılı bir dosyanın 16 satırından birini görüntüleyebilmektedir.



Palet Bileşenleri

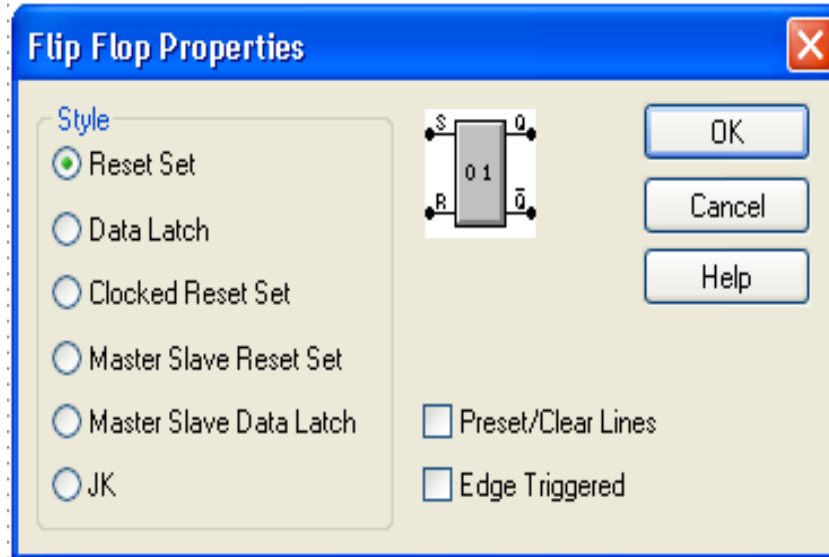
Metin kutusu



Palet Bileşenleri

Flip-Floplar

Multimedia Logic programında kullanılan her bir flip flobun, kenar tetiklemeli (edge triggered) ve seviye tetiklemeli (level triggered) olmak üzere iki çeşidi mevcuttur.



Palet Bileşenleri

Tuş takımı (Keypad)

Üzerinde 0'dan F'ye kadar olan sayıları barındırır ve basıldığında ilgili tuşun hexadecimal (onaltılık) karşılığını çıkışa verir. E çıkışı ise tuş takımından bir tuşa basılıp basılmadığını bildirir. Tuşa basıldığında E çıkışı 1, aksi halde 0'dır.



Palet Bileşenleri

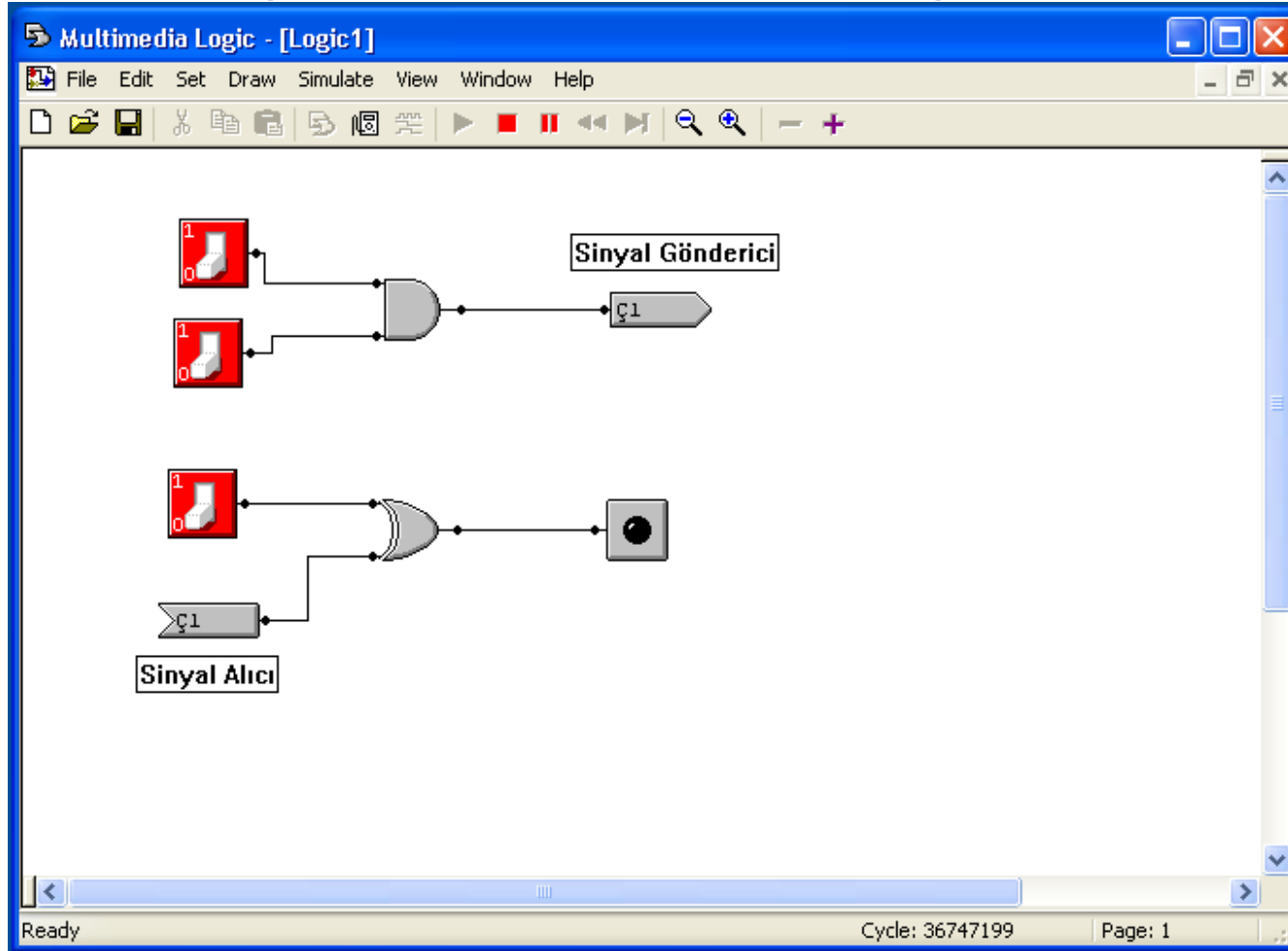
Sinyal gönderici (Signal Sender) 

Sinyal alıcı (Signal Receiver) 

Sinyal gönderici ve sinyal alıcı elemanlar, özellikle devre birden fazla sayfada gerçekleştirildiğinde çok gereklidir. Örneğin tasarlanan bir devrenin bir parçasının çıkışları, diğer sayfada yer alan başka bir parçasının girişleri olabilir. Böylece sayfalar arasında herhangi bir gecikme olmadan sinyaller taşınır. Devrenin daha okunabilir olması için aynı sayfa içinde de kullanılabilir. Dikkat edilmesi gereken nokta ise, sinyali gönderen eleman ile sinyali alan elemanın aynı isimli olması gerektiğidir. Bir sinyal göndericinin isminden iki tane olamazken, aynı isimli birden fazla sinyal alıcı olabilir.

Palet Bileşenleri

Sinyal gönderici (Signal Sender) ve Sinyal alıcı (Signal Receiver)



Palet Bileşenleri

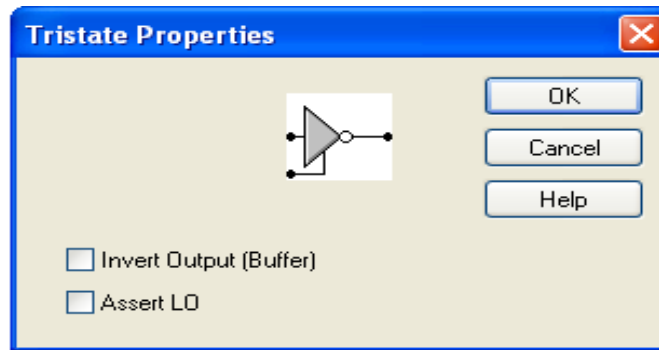
Ground 

Plus 

Bu elemanlar, bağlandıkları girişlere her zaman lojik 0 veya lojik 1 değerini verirler.

Üç durumlu tampon (Tristate) 

Bu eleman özellikle ortak veri yolu tasarımında kullanılır. Bu elemanın sahip olduğu yetkilendirme girişi sayesinde girişine gelen veriyi ya geçirir ya da geçirmez.



Uygulama

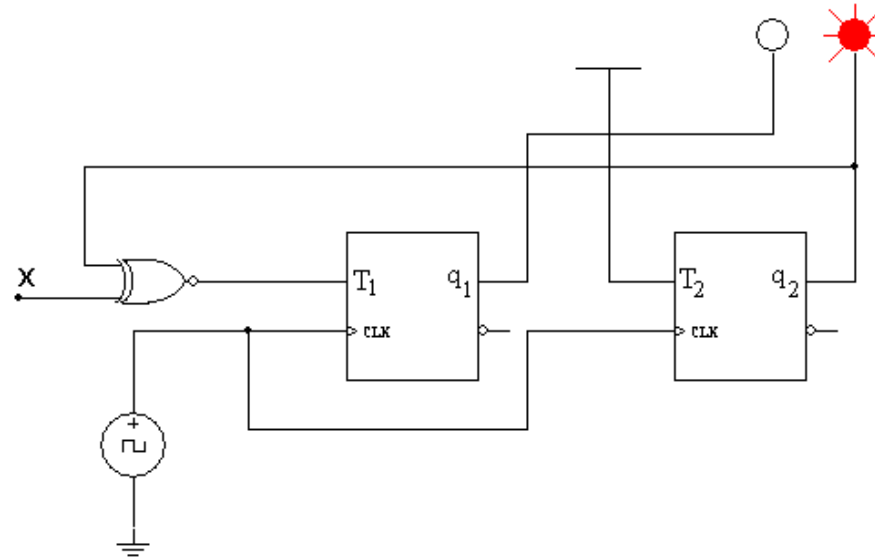
Girişin değerine bağlı olarak, hem ileri hem de geri yönde sayabilen 2 bitlik sayıcı tasarımı.

x girişimiz 1 ise sayıcının ileri yönde saymasını (00-01-10-11-00-...)

x girişimiz 0 ise sayıcının geri yönde saymasını (11-10-01-00-11-...) isteyelim.

$$T_1 = q_2 \otimes x$$

$$T_2 = 1$$



Bölüm 3. KAYDEDİCİLER (REGISTERS)

Bu Hafta

Kaydediciler Üzerindeki Önemli İşlemler

Kaydediciye paralel olarak değer yükleme

Kaydedicinin sıfırlanması (Clear)

Kaydedicinin değerinin 1 arttırılması (Increment)

Gelecek Hafta

Kaydedicinin Load ve Clear kontrol uçlarının bir araya getirilmesi

Kaydedicinin içeriğinin sağa ya da sola kaydırılması (ve seri bilgi girilmesi)

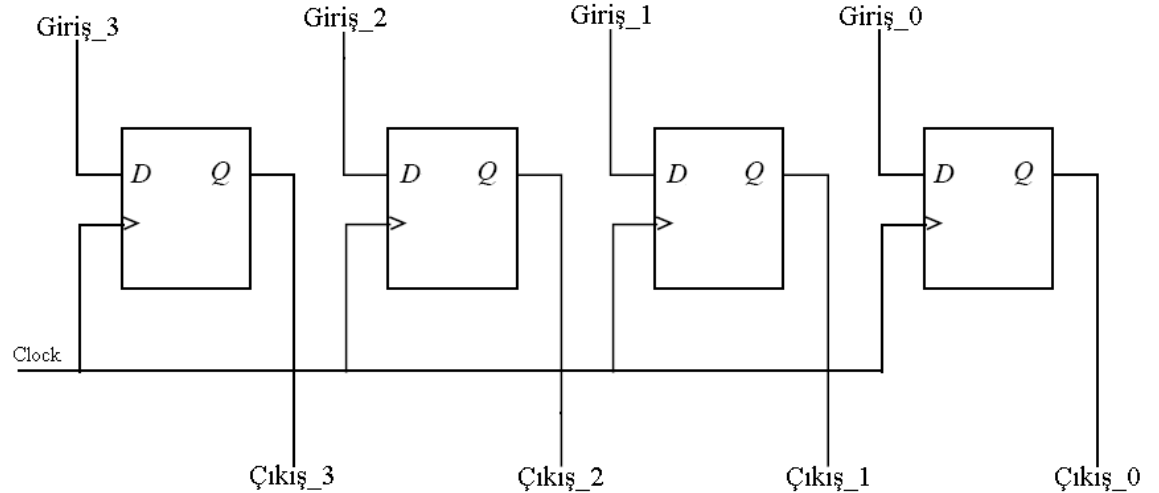
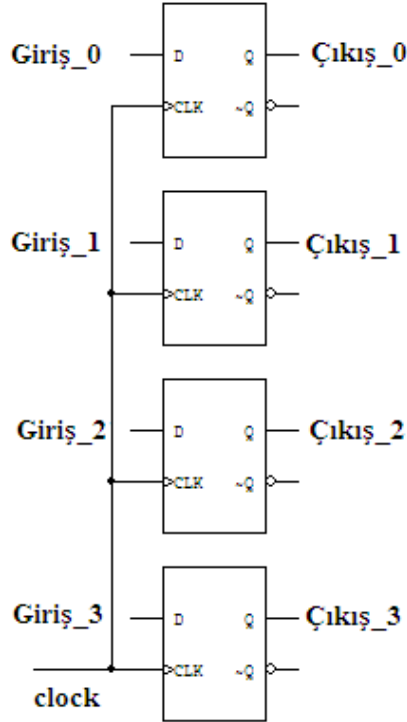
Paralel yükleme ve sağa kaydırma kontrollerinin bir araya getirilmesi

Universal kaydedici tasarımı

KAYDEDİCİLER (REGISTERS)

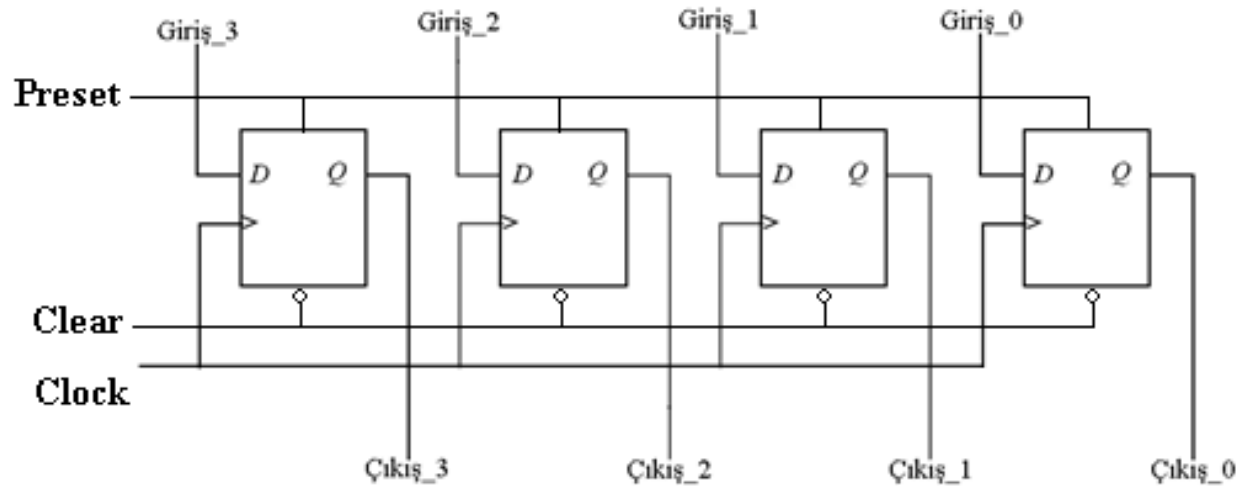
- n adet flip-flobun bir araya getirilmesiyle oluşturulan depolama birimleridir.
- n adet flip-flop, n bitlik bilgiyi saklama kapasitesine sahiptir.

Örnek: Aşağıda 4 bitlik bir kaydedici, D tipi flip floplar kullanılarak oluşturulmuştur.



Preset ve Clear Uçları

Kaydediciler giriş ve çıkış uçlarından başka, clock'tan bağımsız olarak çalışan *Preset* ve *Clear* uçlarına da sahiptirler. Bu uçlar sayesinde, özellikle başlangıçta kaydedicinin hangi değerle yüklü olacağı ayarlanabilir.



Kaydediciler Üzerindeki Önemli İşlemler

- Kaydediciye paralel olarak değer yükleme
- Kaydedicinin *clear* edilmesi ya da sıfırlanması
- Kaydedicinin *set* edilmesi
- Kaydedicinin içeriğinin sağa ya da sola kaydırılması (ve seri bilgi girilmesi)
- Kaydedicinin değerinin 1 arttırılması ya da 1 azaltılması
- Kaydedici içeriğinin 2'ye tümleyeninin alınması
- Kaydedicideki bilginin başka bir kaydediciye aktarılması gibi...

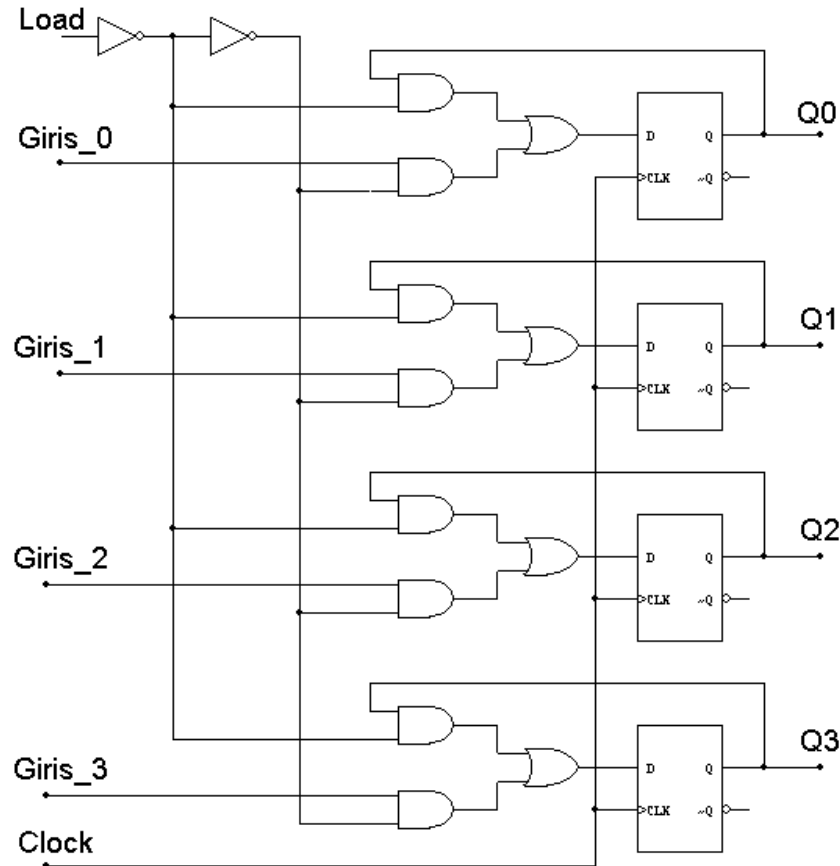
Bahsedilen temel işlemler bir arada kullanılarak, kaydediciler üzerinde daha fazla sayıda işlem tanımlanabilir.

Kaydediciye Paralel Olarak Değer Yükleme

D tipi flip floplardan oluşturulan bir kaydedici için;

Load = 0 ise çıkışların durumlarını muhafaza etmesini,

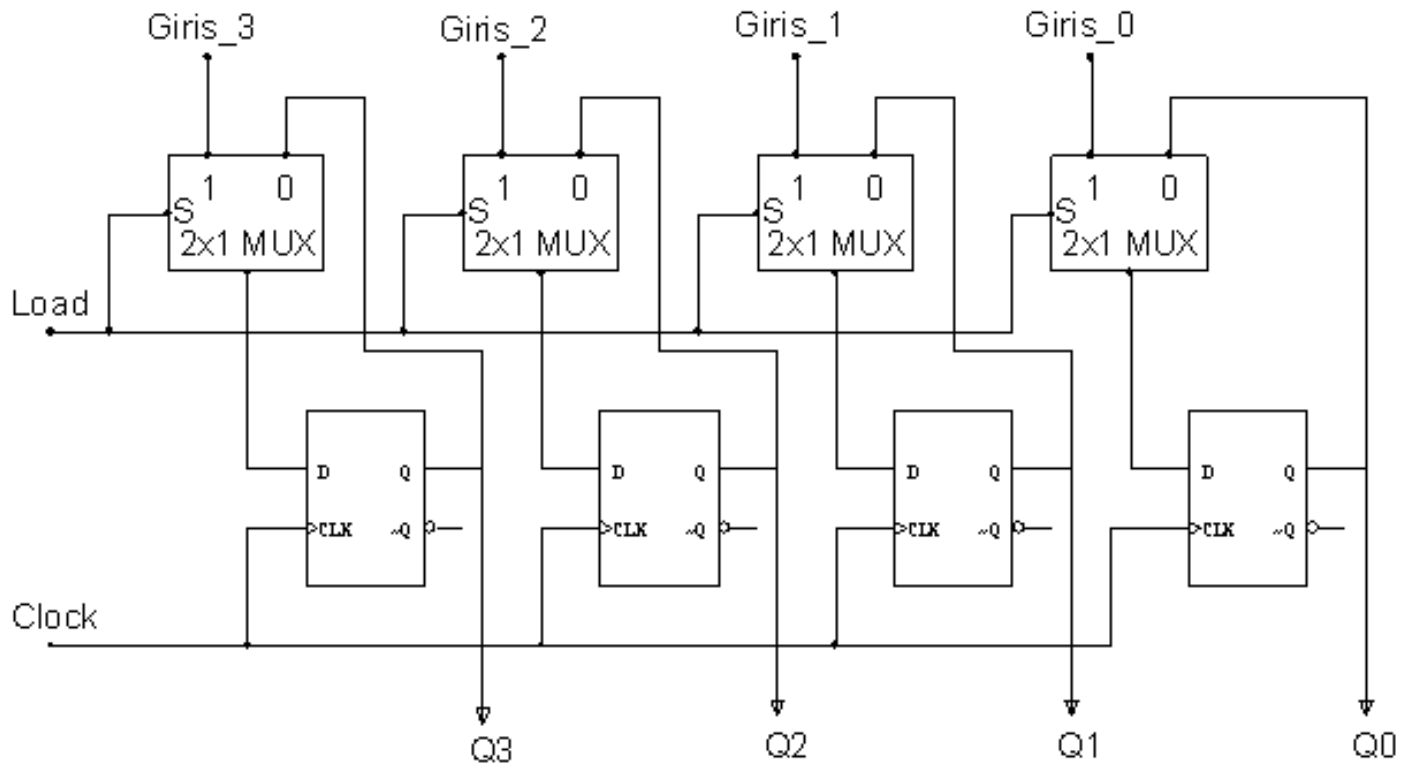
Load = 1 olduğunda ise, girişlerin kaydediciye yüklenmesini istiyorsak,



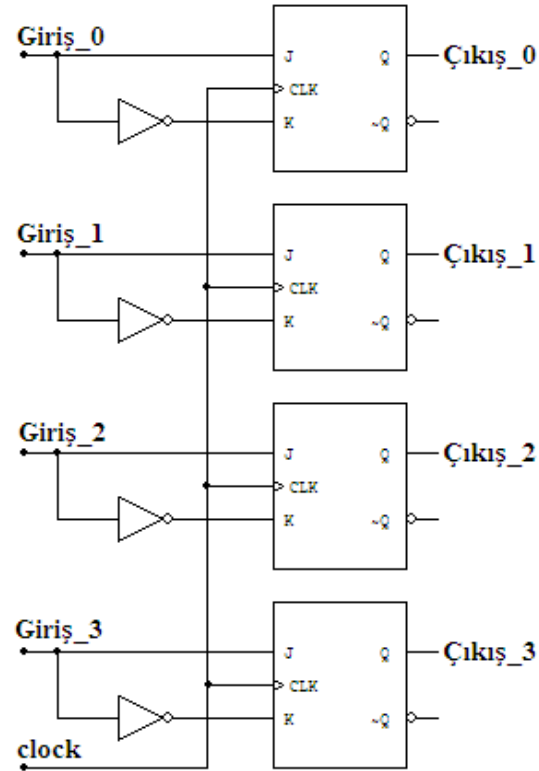
Kaydediciye Paralel Olarak Değer Yükleme

D tipi flip floplardan oluşturulmuş bir kaydediciye Load işlevini kazandırmak için 2×1 MUX kullanılması da düşünülebilir.

MUX'un çıkış ifadesi $y = S'.I_0 + S.I_1$



Kaydediciye Paralel Olarak Değer Yükleme – JK tipi



Giriş uçlarından 0 bilgisi geldiğinde *Reset* durumu, 1 bilgisi geldiğinde ise *Set* durumu oluşur.

Kaydediciye Paralel Olarak Değer Yükleme – JK tipi

JK tipi flip floplardan oluşturulmuş bir kaydediciye Load girişi eklemek istersek, aşağıda verilen 1 bitlik kaydedicinin durum tablosundan faydalanabiliriz.

Kontrol Girişi	Giriş Biti	Sonraki Durum	Flip-Flop Girişleri	
Load	Giriş_0	Q	J	K
0	0	Durumunu koru	0	0
0	1	Durumunu koru	0	0
1	0	Girişteki Bilgi (0)	0	1
1	1	Girişteki Bilgi (1)	1	0

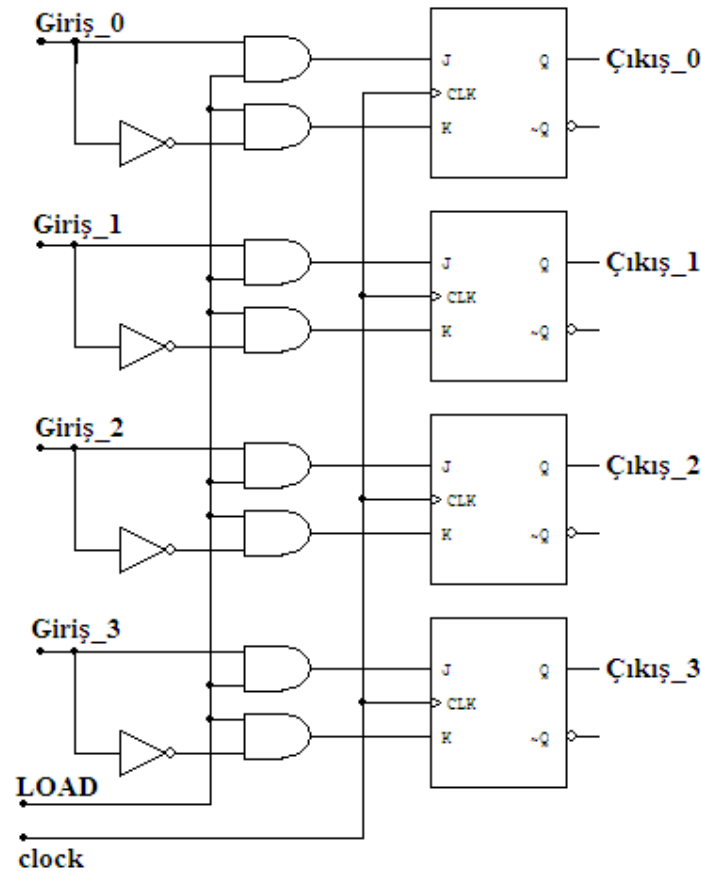
$$J = \text{Load} \cdot \text{Giriş_0}$$

$$K = \text{Load} \cdot \text{Giris_0}'$$

Kaydediciye Paralel Olarak Değer Yükleme – JK tipi

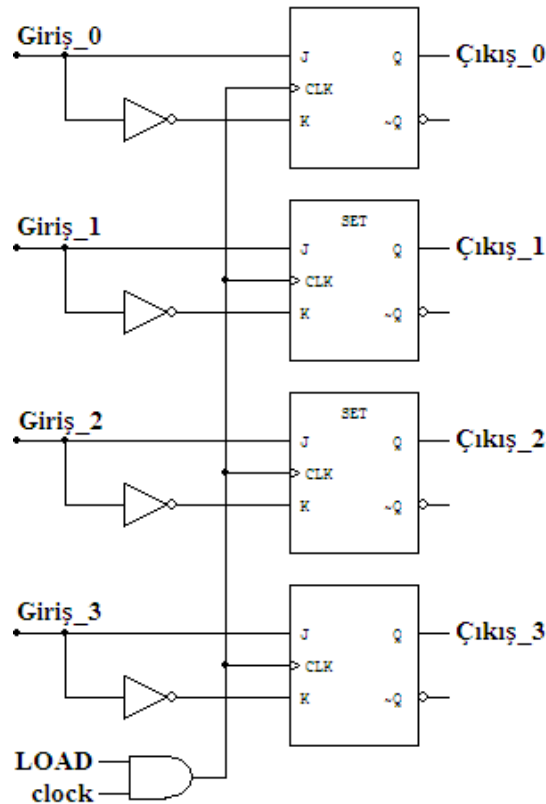
$J = \text{Load.Giriş}_0$

$K = \text{Load.Giris}_0'$



Kaydediciye Paralel Olarak Değer Yükleme – JK tipi

Kaydediciye paralel olarak değer yüklemenin diğer bir yöntemi ise; yükleyeceğimiz girişleri direkt olarak flip flopların girişlerine vermek ve LOAD girişini de clock sinyali ile lojik VE işlemine tabi tutup flip flopların CLK girişlerine uygulamaktır.



Kaydedicinin Sıfırlanması (Clear)

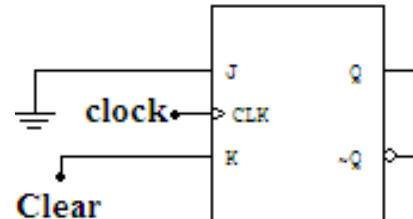
Flip-floplar genellikle clock'tan bağımsız olarak çalışan *Clear* uçlarına sahip olmalarına rağmen, böyle bir ucun olmadığı durumda clock geçişiyle kaydediciyi sıfırlamak mümkündür.

JK tipi flip floplardan oluşturulmuş bir kaydediciye Clear girişi eklemek istersek, aşağıda verilen 1 bitlik kaydedicinin durum tablosundan faydalanabiliriz.

Clear	Sonraki Durum (Q)	J	K
0	Durumunu Kori	0	0
1	0	0	1

$J = 0$

$K = \text{Clear}$



Kaydedicinin İçeriğinin 1 Arttırılması (Increment)

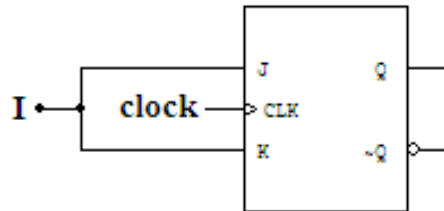
1 bitlik bir kaydedicinin değerinin 1 arttırılması için gerekli olan durum tablosu aşağıdaki gibidir;

Arttır (I)	Sonraki Durum (Q)	J	K
0	Durumunu Korum	0	0
1	Çıkışın tersini al	1	1

Arttır(I)	Şimdiki Durum(q)	Sonraki Durum(Q)	J	K
0	0	0	0	x
0	1	1	x	0
1	0	1	1	x
1	1	0	x	1

$$J = I$$

$$K = I$$



2 Bitlik Kaydedicinin İçeriğinin 1 Arttırılması

Arttır (I)	Şimdiki Durum(q1q0)	Sonraki Durum(Q1Q0)	J1	K1	J0	K0
0	0 0	0 0	0	x	0	x
0	0 1	0 1	0	x	x	0
0	1 0	1 0	x	0	0	x
0	1 1	1 1	x	0	x	0
1	0 0	0 1	0	x	1	x
1	0 1	1 0	1	x	x	1
1	1 0	1 1	x	0	1	x
1	1 1	0 0	x	1	x	1

2 Bitlik Kaydedicinin İçeriğinin 1 Arttırılması

Arttır (I)	Şimdiki Durum(q1q0)	Sonraki Durum(Q1Q0)	J1	K1	J0	K0
0	0 0	0 0	0	x	0	x
0	0 1	0 1	0	x	x	0
0	1 0	1 0	x	0	0	x
0	1 1	1 1	x	0	x	0
1	0 0	0 1	0	x	1	x
1	0 1	1 0	1	x	x	1
1	1 0	1 1	x	0	1	x
1	1 1	0 0	x	1	x	1

q0 Iq1 \	0	1
00		
01	x	x
11	x	x
10		1

$$J1 = I.q0$$

q0 Iq1 \	0	1
00	x	x
01		
11		1
10	x	x

$$K1 = I.q0$$

q0 Iq1 \	0	1
00		x
01		x
11	1	x
10	1	x

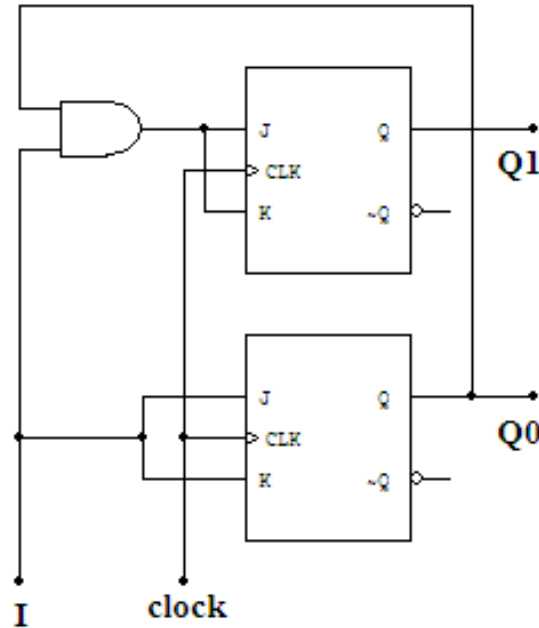
$$J0 = I$$

q0 Iq1 \	0	1
00	x	
01	x	
11	x	1
10	x	1

$$K0 = I$$

2 Bitlik Kaydedicinin İçeriğinin 1 Arttırılması

$$\begin{aligned} J1 &= I.q0 \\ K1 &= I.q0 \\ J0 &= I \\ K0 &= I \end{aligned}$$



Arttır (I) girişine sahip 3 bitlik bir kaydedici tasarlanmak istenirse, tahmin edilebileceği gibi, Q2 flip flobunun girişlerine, $J2 = K2 = I.q1.q0$ uygulanması gerekecektir (q1q0=11 iken I=1 ise Q2 flip flobunun JK girişlerine 11 uygulanacağından Q2 çıkışı, mevcut çıkışının değili olacaktır).

Bölüm 3. KAYDEDİCİLER (REGISTERS)

Geçen Hafta

Kaydediciler Üzerindeki Önemli İşlemler

Kaydediciye paralel olarak değer yükleme

Kaydedicinin sıfırlanması (Clear)

Kaydedicinin değerinin 1 arttırılması (Increment)

Bu Hafta

Kaydedicinin Load ve Clear kontrol uçlarının bir araya getirilmesi

Kaydedicinin içeriğinin sağa ya da sola kaydırılması (ve seri bilgi girilmesi)

Paralel yükleme ve sağa kaydırma kontrollerinin bir araya getirilmesi

Üniversal kaydedici tasarımı

Kaydedicinin Load ve Clear (C) Kontrol Uçlarının Bir Araya Getirilmesi

1 bitlik kaydedici için Load işlevi,

$$J = (\text{Giriş}_0).(\text{Load})$$

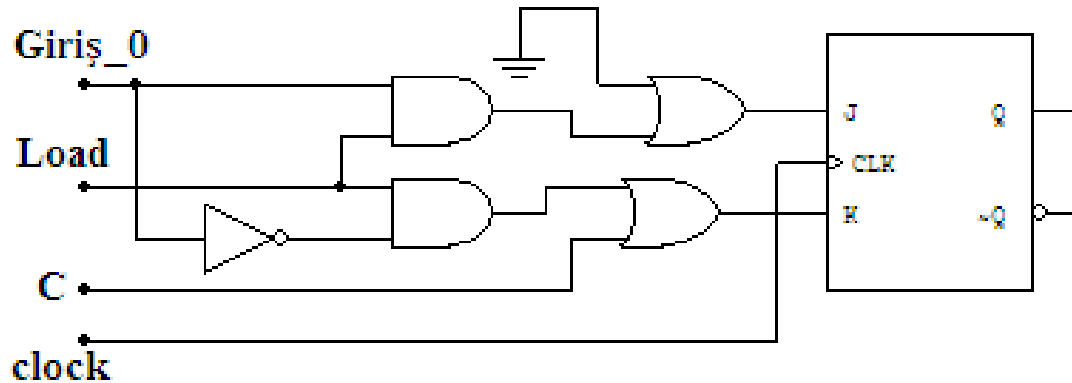
$$K = (\text{Giriş}_0)' . (\text{Load})$$

Clear işlevi,

$$J = 0$$

$$K = C$$

Bu işlevleri bir araya getirebilmek için VEYA kapısı (işlev sayısı kadar girişi olmalıdır) kullanılmalıdır (Bu iki işlevin aynı anda olmayacağı varsayılmıştır).



Kaydedicinin İçeriğinin Sağa ya da Sola Kaydırılması

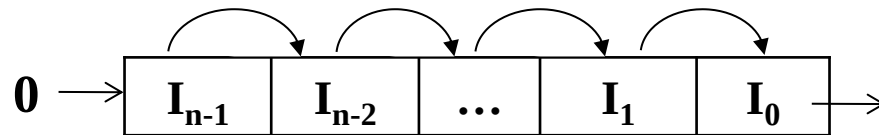
Kaydedici içeriğinin 1 bit sola ya da sağa kaydırılması işlemidir. Kaydırma işlemleri neticesinde kaydedicinin en düşük anlamlı ve en yüksek anlamlı bitlerine yüklenecek değere göre 3 farklı kaydırma işlemi olabilir.

■ Sıfır İle Kaydırma

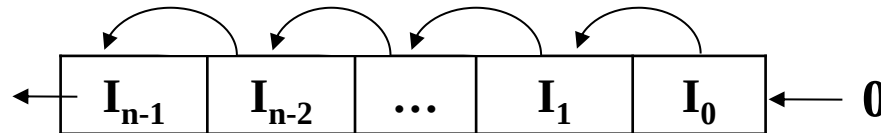
Kaydırma işleminin yönüne göre, LSB ya da MSB bitine 0 yüklenir.

Kaydedici içeriği ($I_{n-1} I_{n-2} \dots I_1 I_0$) şeklindeyse,

Sağa kaydırma işleminde $I_i \leftarrow I_{i+1}$, $i = n-2, n-1, \dots, 1, 0$



Sola kaydırma işleminde $I_i \leftarrow I_{i-1}$, $i = n-1, n-2, \dots, 1$ olacaktır.



Sıfır İle Kaydırma

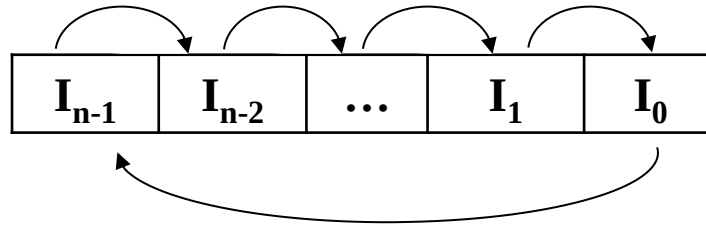
Örnek: 4 bitlik bir kaydediciyi 4 kez sıfır ile sağa ya da sola kaydıralım.
Kaydedici içeriği 1101_2 olsun.

1101	1101
1 bit sağa 0110	1 bit sola 101 0
1 bit sağa 0011	1 bit sola 01 00
1 bit sağa 0001	1 bit sola 1 000
1 bit sağa 0000	1 bit sola 0000

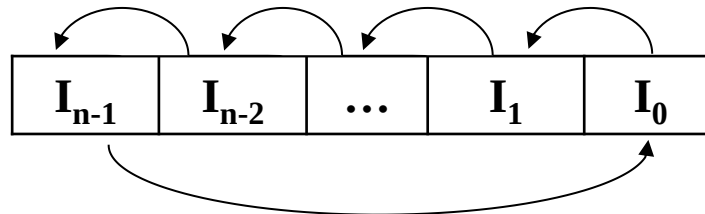
Kaydedicinin İçeriğinin Sağa ya da Sola Kaydırılması

■ Döngüsel Kaydırma

Döngüsel sağa kaydırmada, LSB biti MSB bitine aktarılır ($I_{n-1} \leftarrow I_0$).



Döngüsel sola kaydırmada ise MSB biti LSB bitine aktarılır ($I_0 \leftarrow I_{n-1}$).



n bitlik bir kaydedici, n kez döngüsel kaydırılacak olursa, başlangıç durumuna dönecektir.

Döngüsel Kaydırma

Örnek: 4 bitlik bir kaydediciyi 4 kez döngüsel sağa ya da sola kaydıralım.
Kaydedici içeriği 1101_2 olsun.

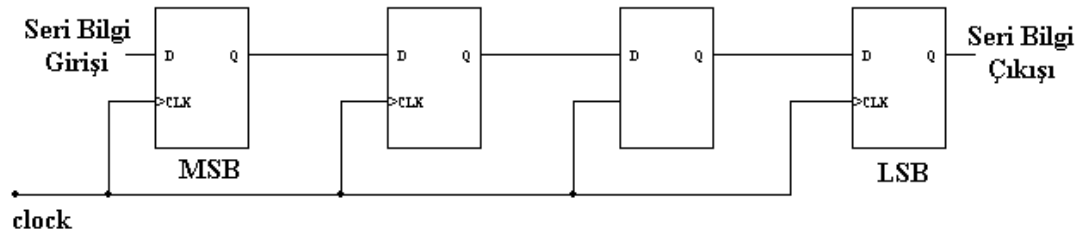
1101	1101
1 bit sağa 1110	1 bit sola 101 1
1 bit sağa 0111	1 bit sola 01 11
1 bit sağa 1011	1 bit sola 1 110
1 bit sağa 1101	1 bit sola 1101

Kaydedicinin İçeriğinin Sağa ya da Sola Kaydırılması

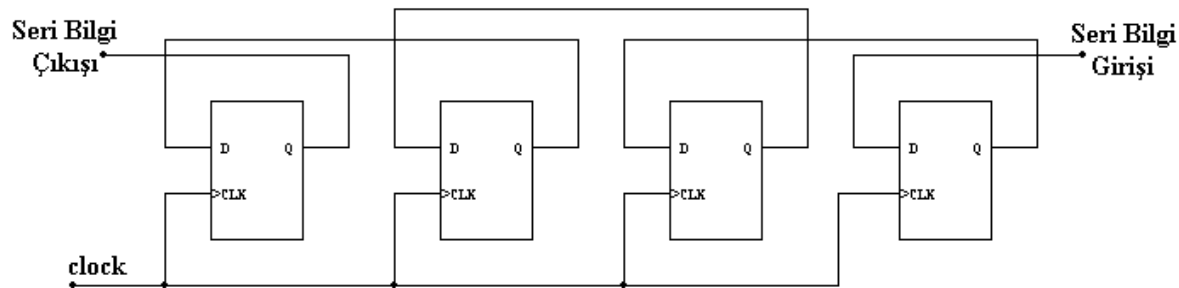
■ Seri Bilgi ile Kaydırma

⇒ Seri bilgi girilerek sağa kaydırma işleminde, MSB bitine yeni bir değer yüklenir, LSB bitinin eski değeri kaybolur.

⇒ Seri bilgi girilerek sola kaydırma işleminde, LSB bitine yeni bir değer yüklenir, MSB bitinin eski değeri kaybolur.



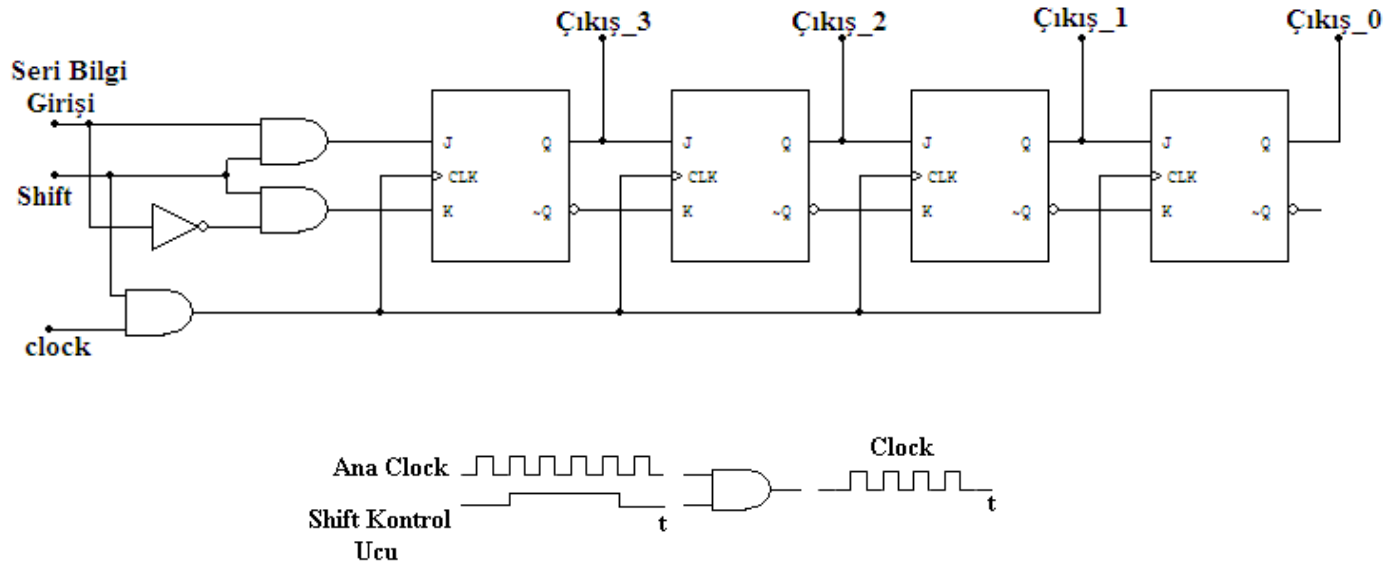
Sağa kaydırma



Sola kaydırma

Seri Bilgi ile Kaydırma

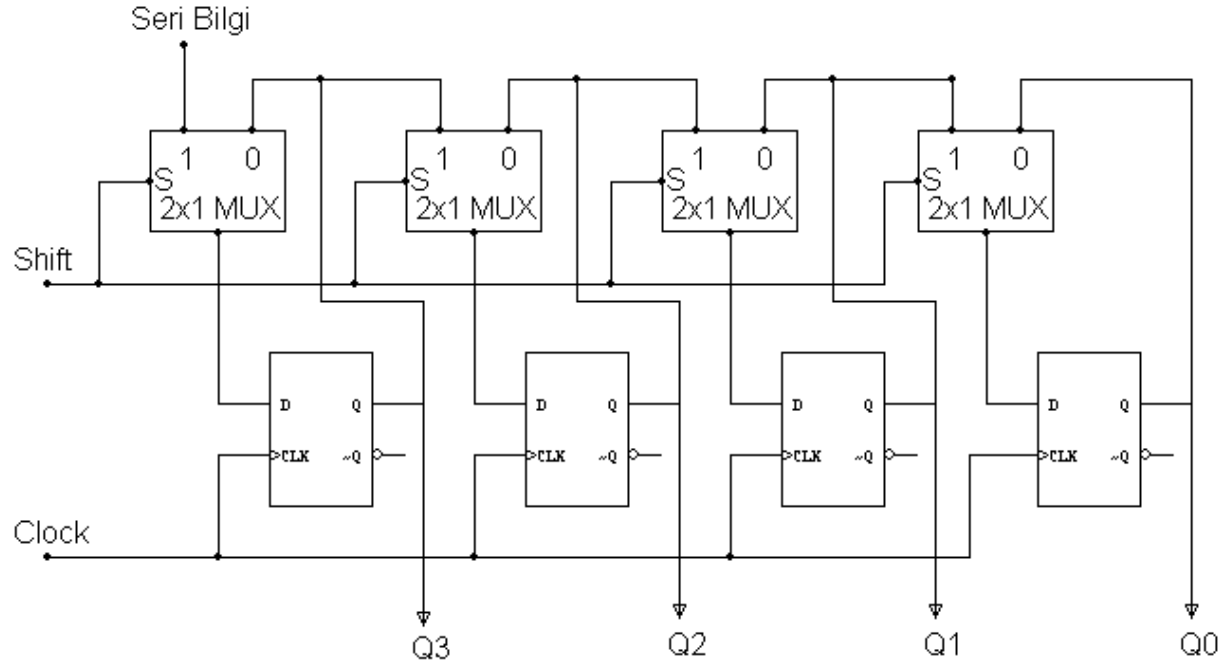
Şayet kaydırma işlemini kontrollü bir şekilde yapmak istersek bir *Shift* kontrol ucunu devremize eklememiz gerekecektir.



Bu devre aynı zamanda seri giriş - seri çıkış olarak kullanılabilir.

Seri Bilgi ile Kaydırma

D tipi flip floplardan oluşturulan 4 bitlik kaydediciye, seri bilgi girişi ile sağa kaydırma işlevini kazandırmak için aşağıdaki düzenek kullanılabilir;

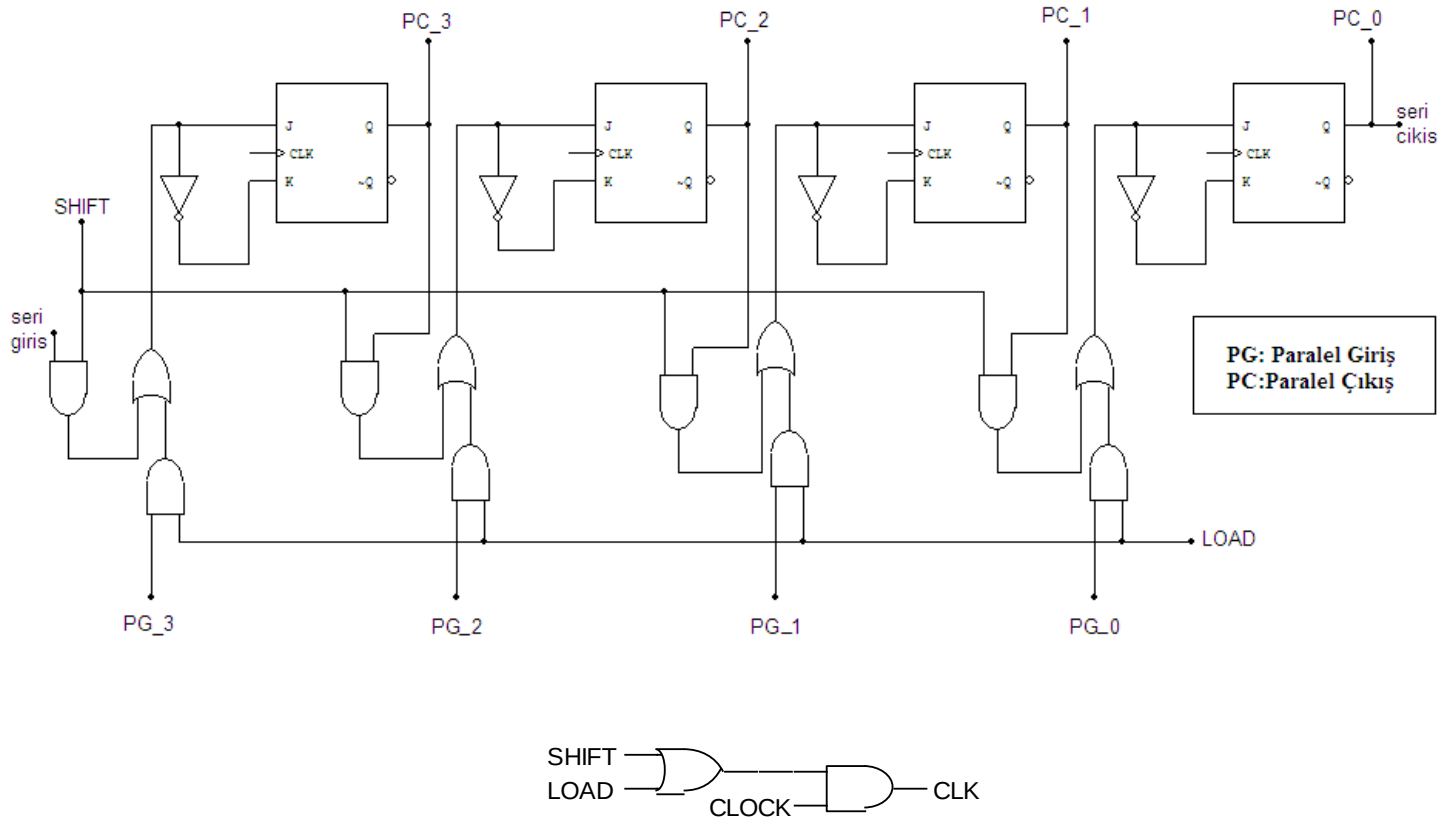


Shift = 0 için kaydedici durumunu korur,

Shift = 1 için $Q3 \leftarrow \text{Seri Bilgi}$, $Q2 \leftarrow Q3$, $Q1 \leftarrow Q2$, $Q0 \leftarrow Q1$

Paralel Yükleme ve Sağa Kaydırma Kontrollerinin Bir Araya Getirilmesi

Aşağıdaki düzenekte kaydedicinin durumunu koruması için clock sinyali (CLK) pasif hale getirilmektedir (her iki kontrol girişinin aktif olmadığı durum).



Paralel Yükleme ve Sağa Kaydırma Kontrollerinin Bir Araya Getirilmesi

Bu işlevlerin bir araya getirmenin diğer bir yöntemi de kaydırma işlevini de yükleme işlevi gibi düşünmektir. Bu durumda kaydediciyi oluşturan flip flopların girişleri şu şekilde olur;

$$J3 = (\text{LOAD}).(\text{PG}_3) + (\text{SHIFT}).(\text{seri_giriş_verisi})$$

$$K3 = (\text{LOAD}).(\text{PG}_3)' + (\text{SHIFT}).(\text{seri_giriş_verisi})'$$

$$J2 = (\text{LOAD}).(\text{PG}_2) + (\text{SHIFT}).(\text{PC}_3)$$

$$K2 = (\text{LOAD}).(\text{PG}_2)' + (\text{SHIFT}).(\text{PC}_3)'$$

$$J1 = (\text{LOAD}).(\text{PG}_1) + (\text{SHIFT}).(\text{PC}_2)$$

$$K1 = (\text{LOAD}).(\text{PG}_1)' + (\text{SHIFT}).(\text{PC}_2)'$$

$$J0 = (\text{LOAD}).(\text{PG}_0) + (\text{SHIFT}).(\text{PC}_1)$$

$$K0 = (\text{LOAD}).(\text{PG}_0)' + (\text{SHIFT}).(\text{PC}_1)'$$

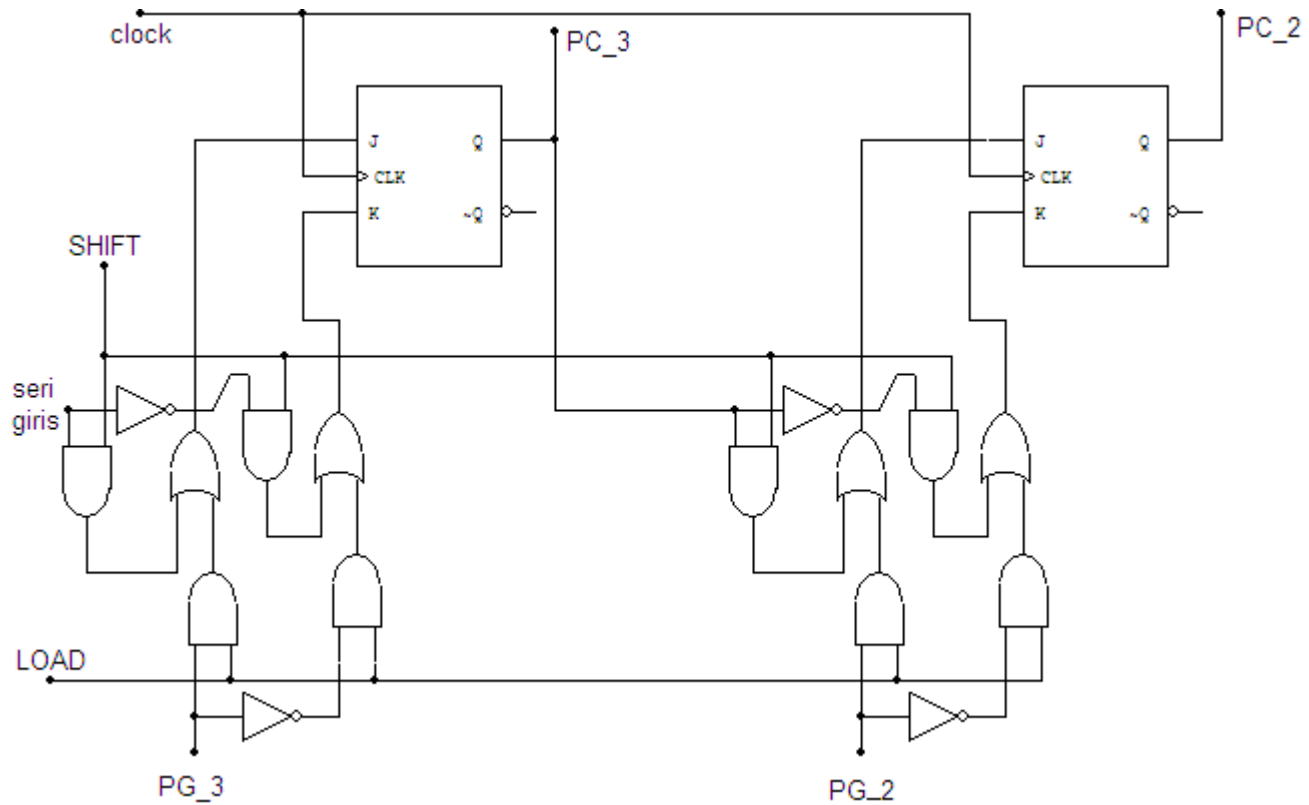
Paralel Yükleme ve Sağa Kaydırma Kontrollerinin Bir Araya Getirilmesi

$$J3 = (\text{LOAD}).(\text{PG}_3) + (\text{SHIFT}).(\text{seri giriş})$$

$$K3 = (\text{LOAD}).(\text{PG}_3)' + (\text{SHIFT}).(\text{seri giriş})'$$

$$J2 = (\text{LOAD}).(\text{PG}_2) + (\text{SHIFT}).(\text{PC}_3)$$

$$K2 = (\text{LOAD}).(\text{PG}_2)' + (\text{SHIFT}).(\text{PC}_3)'$$



Paralel yükleme ve sağa kaydırma kontrollerinin kaydedicinin 2 bit i için bir araya getirilmesi

Bölüm 4. BİLGİSAYAR SİSTEMLERİNİN HİYERARŞİK YAPISI

Hiyerarşik Yapı

Von-Neumann Mimarisi

Yol Kavramı

Bire Bir Bağlantı

Ortak Yol

Bilgisayar Sistemlerinin Hiyerarşik Yapısı

Nasıl ki programlamada, büyük ölçekli problemler modüllere bölünür ve her modül de ayrı ayrı tasarlanırsa, bilgisayar organizasyonunda da benzer yapı vardır. Temel olarak bilgisayar hiyerarşisi 7 seviyeden oluşur.

Seviye 6	Kullanıcı Seviyesi	Çalıştırılabilir programlar
Seviye 5	Yüksek Seviyeli Diller	C, Java gibi
Seviye 4	Assembly Dili	Assembly kodu
Seviye 3	Sistem Yazılımları	İşletim Sistemi, Derleyiciler gibi
Seviye 2	Makine Dili	Komut seti mimarisi
Seviye 1	Kontrol	Donanımsal ya da mikro programlanmış
Seviye 0	Lojik Seviye	Kapılar, devreler, ...

Bilgisayar Sistemlerinin Hiyerarşik Yapısı

Seviye 6, kullanıcı seviyesidir. Herkesin kullandığı kelime işlemci programlarının, grafik, e-posta, oyun gibi programların çalıştırıldığı seviyedir.

Seviye 5, yüksek seviyeli diller seviyesidir. C, Java gibi dilleri içerir. Bu diller derleyiciler ya da yorumlayıcılar kullanılarak makinenin anlayacağı dile çevrilmelidirler. Bu seviyede kullanıcı, veri tiplerini ve bu veriler üzerindeki gerçekleştirebileceği işlemleri bilmek zorunda olmasına rağmen daha düşük seviyelerde bu işlemlerin nasıl gerçekleştirileceğini bilmek zorunda değildir.

Seviye 4, assembly dili seviyesidir. Assembly dilinde, yerine getirilmek istenen işlevin manasını içeren kısaltmalar kullanılır (örneğin toplama işlemi için *add*, çıkartma işlemi için *sub* kullanılması gibi). Bu seviyeden sonra, oluşturulan assembly kodları bire bir makine kodlarına dönüştürülebilirler.

Bilgisayar Sistemlerinin Hiyerarşik Yapısı

Seviye 3, sistem yazılımları seviyesidir. Özellikle işletim sisteminin sorumlulukları ile ilgilidir. Bu seviye, çoklu programlama, bellek organizasyonu, bir arada çalışan programlar arası senkronizasyon gibi fonksiyonları içerir. Bu konular, işletim sistemleri dersinin kapsamındadır. Assembly dilinden makine kodlarına çevrilen komutlar, değişime uğratılmadan bu seviyeye geçirilirler.

Seviye 2, makine seviyesidir. Bu seviyede, belli bir mimariye sahip bilgisayar sistemi tarafından tanınabilen makine komutları vardır. Her işlemcinin kendine has bir komut seti mimarisi vardır. Makine dilinde yazılan programlar derleyici, yorumlayıcı ya da bir dönüştürücüye gereksinim duymazlar ve elektronik devreler üzerinde direkt olarak icra edilirler.

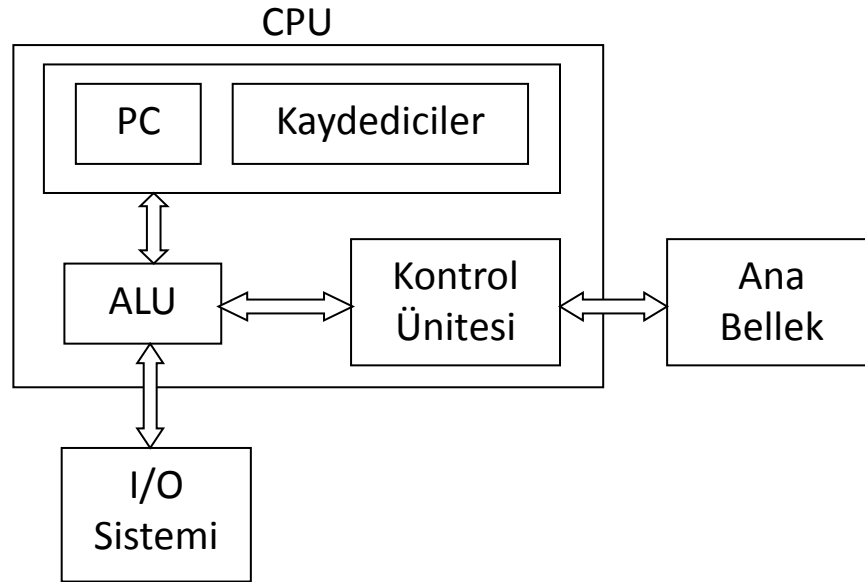
Bilgisayar Sistemlerinin Hiyerarşik Yapısı

Seviye 1, kontrol seviyesidir. Kontrol ünitesi, bir üst seviyeden gelen makine komutlarını yorumlar ve gerekli işlemi yerine getirir. Kontrol ünitesi, donanımsal (hardwired) ya da mikro programlanmış (microprogrammed) kontrol olmak üzere 2 türlü gerçekleştirilebilir. Donanımsal kontrolde, komutların icra edilmesi için gerekli olan kontrol sinyalleri direkt olarak elektronik düzenekler vasıtasıyla oluşturulur ve bu sinyaller bilgisayarı oluşturan temel bileşenlere (Merkezi işlem birimi, bellek gibi) uygulanırlar. Mikro programlanmış kontrolde ise komutlar bir mikro program sayesinde işletilirler. Mikro program donanım tarafından direkt olarak icra edilebilen düşük seviyeli bir dille yazılır. Seviye 2'deki makine komutları mikro programa yönlendirilir ve burada her makine seviyeli komut birden fazla mikro koda dönüştürülür. Bu mikro kodlar sayesinde de gerekli işlemler yapılır.

Seviye 0, lojik seviyedir. Bu seviyede kapılar, elektronik devreler, yollar gibi temel fiziksel komponentler vardır.

von-Neumann Mimarisi

Bu mimari 3 temel bileşenden oluşur; CPU (Central Processing Unit), bellek ve Giriş/Çıkış cihazları. CPU, ALU (Arithmetic and Logic Unit), kontrol ünitesi, program sayacı (Program Counter-PC) ve kaydedicilerden oluşur.

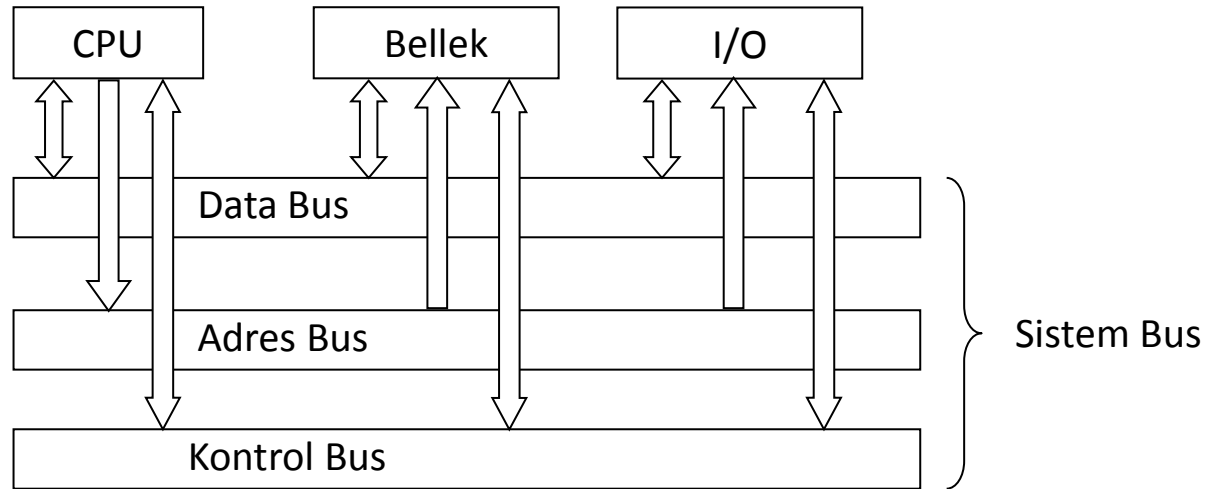


Bu mimari ardışıl olarak komut işleme yeteneğine sahiptir.

- Kontrol ünitesi, PC aracılığıyla bir sonraki komutu alır,
- Komut, ALU'nun anlayabileceği dile çevrilir,
- Operand söz konusu ise bellekten ya da I/O birimlerinden CPU kaydedicilerine alınır,
- ALU komutu işletir ve sonucu kaydedicilere, belleğe ya da I/O birimlerine koyar.

von-Neumann Mimarisi

Günümüz bilgisayarları da bu mimariye dayanmaktadır. Ancak bazı genişletmeler yapılmıştır. Özellikle CPU, ana bellek ve I/O cihazları arasındaki sınırlı bağlantılar yerine sistem yolu ile tüm bileşenlerin birbirine bağlanması sağlanmıştır. Ayrıca kayan noktalı sayılar için ek işlemci desteği ve paralel işlem yapabilme yeteneği de kazandırılmıştır.



BUS (Yol) Yapısı

Bir bilgisayar sisteminde genel olarak 3 tip bus vardır;

- Adres yolu
- Data yolu
- Kontrol yolu

Bus'lar iki şekilde organize edilebilir;

- Tekli (Adres ve data için aynı hatlar kullanılır)
- Çoklu (Adres ve data için ayrı yollar kullanılır)

Genellikle çoklu bus yapısı kullanılır.

Bağlantı Türleri

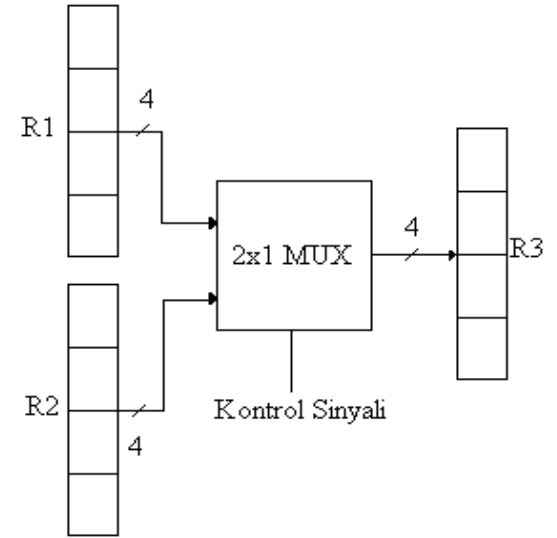
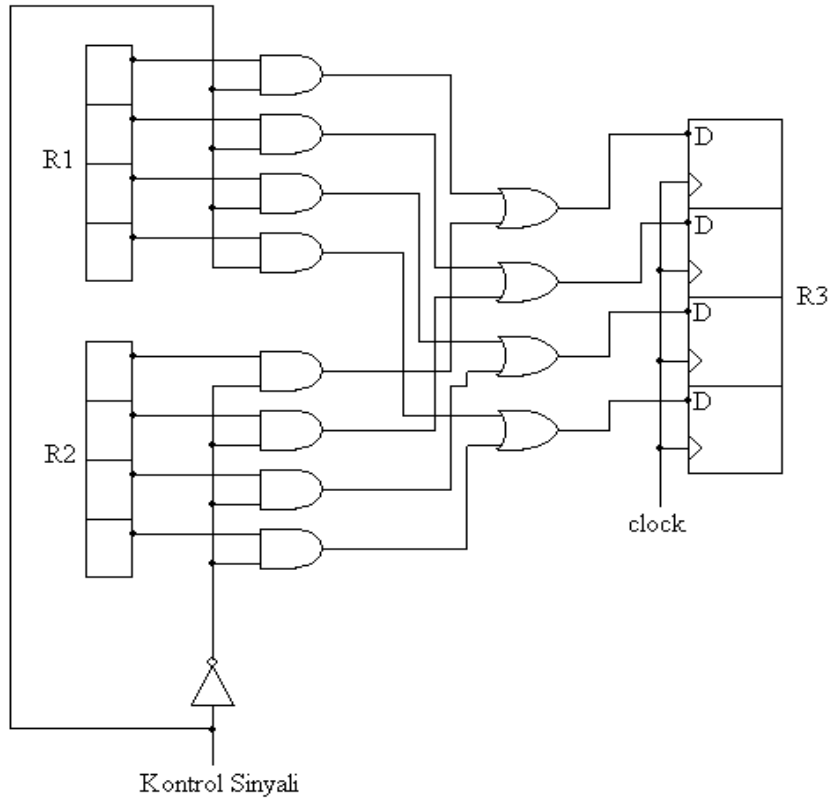
Bilgisayar programlarının önemli bir kısmını kaydediciler arası veri transferi oluşturur.

Bir kaydedicideki veri, birkaç kaydediciye transfer edilebilir veya bir kaydediciye bir ya da daha fazla kaydediciden bilgi gelebilir.

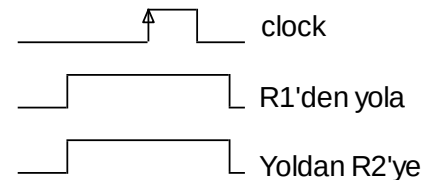
Bu bileşenler arasındaki bağlantılar, bire bir olabilir. Bileşen sayısı arttıkça bağlantı sayısı da artacağından bir kablo karmaşası meydana gelecektir. Fakat veri transferi daha hızlıdır.

Ortak bir bus yapısı kullanıldığında ise daha sade bir yapı elde edilir. Tüm bileşenler arasındaki bağlantılar, bu yollar vasıtasıyla olur. Ortak bus, zaman paylaşımli olarak kullanılır. Aynı anda tek bir kaydedicinin içeriği ortak yola aktarılabiliryorken, ortak yoldan birden fazla kaydediciye veri aktarılabilir.

Bire Bir Bağlantı



Kontrol sinyalinin değerine göre R1'in ya da R2'nin içeriği R3'e aktarılmaktadır.



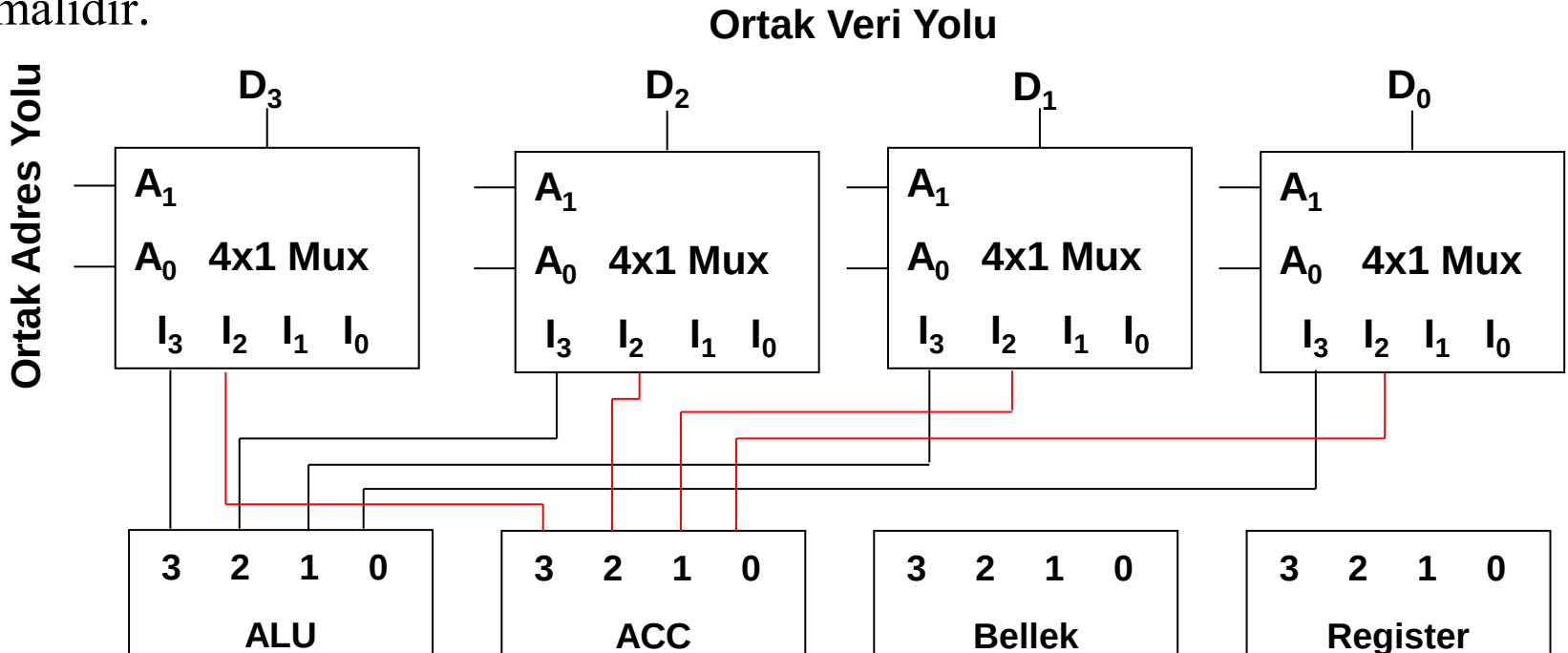
Ali Gülbağ

Ortak Yol Oluşturma Yöntemleri

1. Multiplexer'lar (MUX) kullanılarak,
2. Üç durumlu buffer'lar (tristate) kullanılarak.

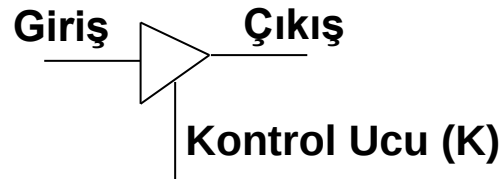
Örnek: MUX kullanarak 4 bitlik bir ortak yol tasarımı.

Ortak yolumuz 4 bitlik olacağından 4 adet MUX kullanılmalıdır. Bu ortak yola 4 tane bileşenin bağlanacağını düşünürsek MUX'lar 4 adet girişe sahip olmalıdır.



Üç Durumlu Buffer (Tristate)

Ortak yol oluşturmak için kod çözücü ve üç durumlu tamponlardan da faydalanılabilir. Bu komponentin çalışma şekli aşağıda özetlenmiştir.

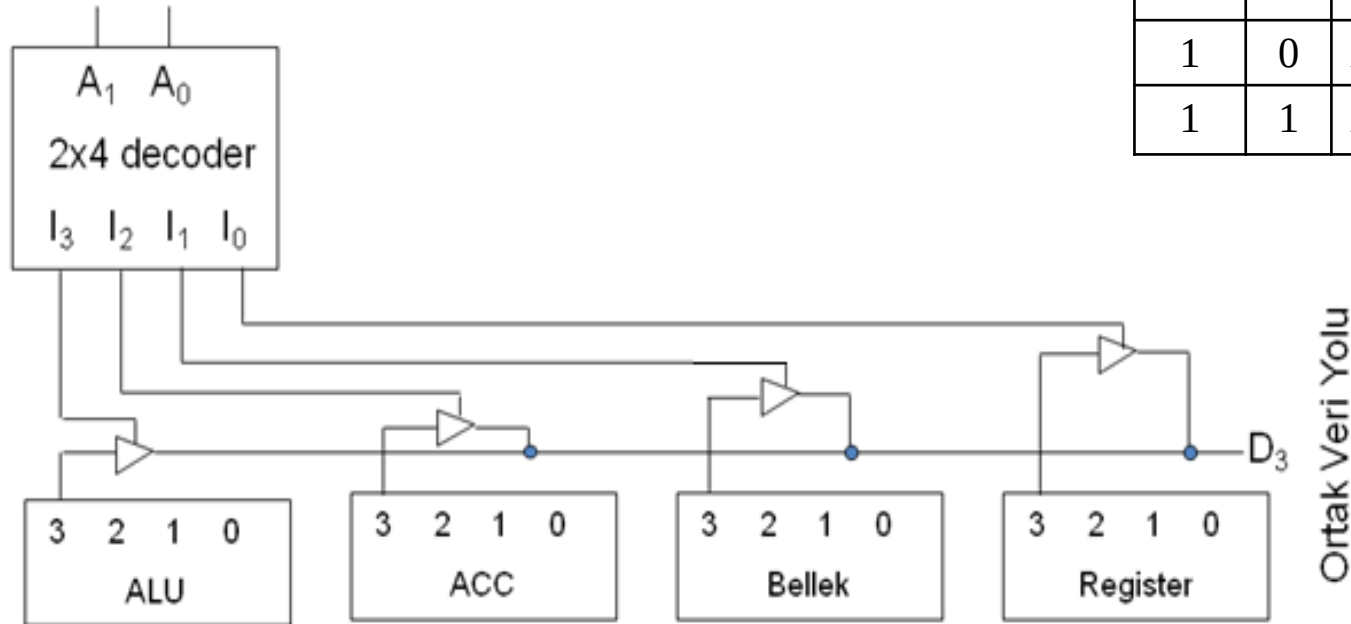


$K = 1$ ise Çıkış $\leftarrow$ Giriş

$K = 0$ ise Çıkışta yüksek empedans görülür (yalıtım sağlanır).

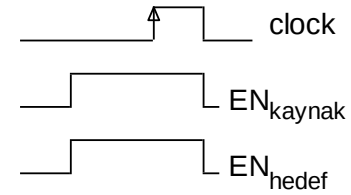
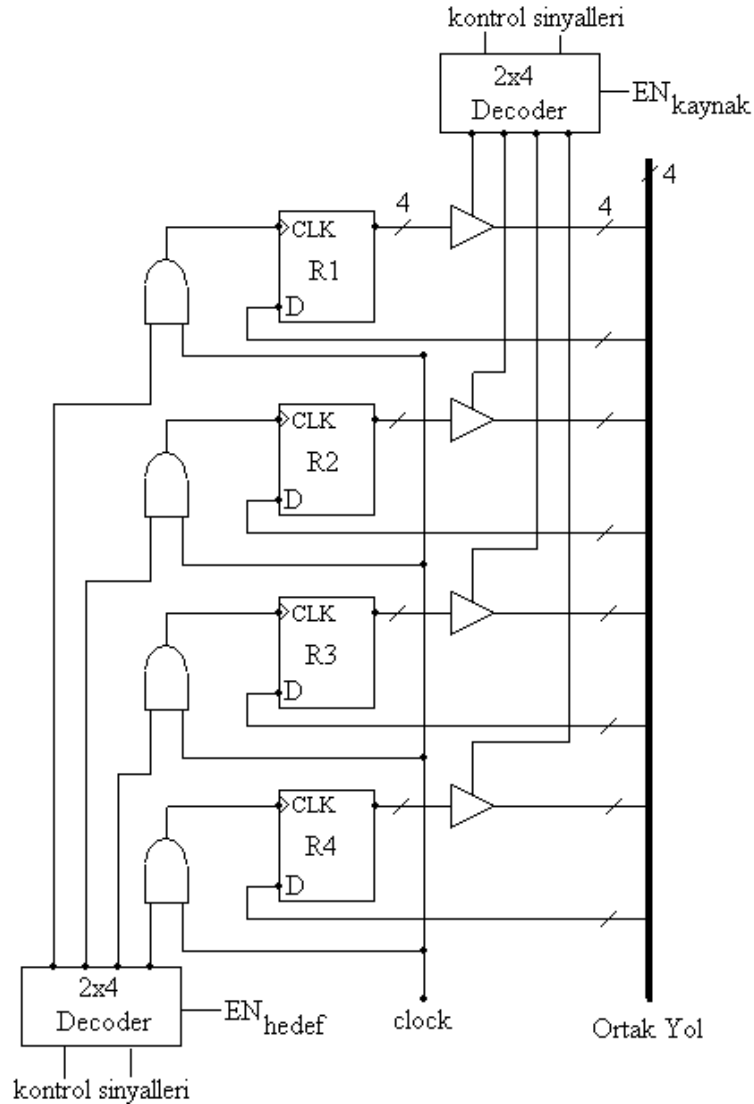
Tristate'ler Kullanılarak Ortak Yol Tasarımı

Ortak Adres Yolu



A_1	A_0	Yolu kullanacak eleman
0	0	Register
0	1	Bellek
1	0	ACC
1	1	ALU

Kaydedicilerin içeriğinin yola aktarılmasını ve kaydediciye yoldan veri alınmasını sağlayan düzenek



Aktarım için gerekli sinyaller

Bölüm 4. BİLGİSAYAR SİSTEMLERİNİN HİYERARŞİK YAPISI

BELLEK

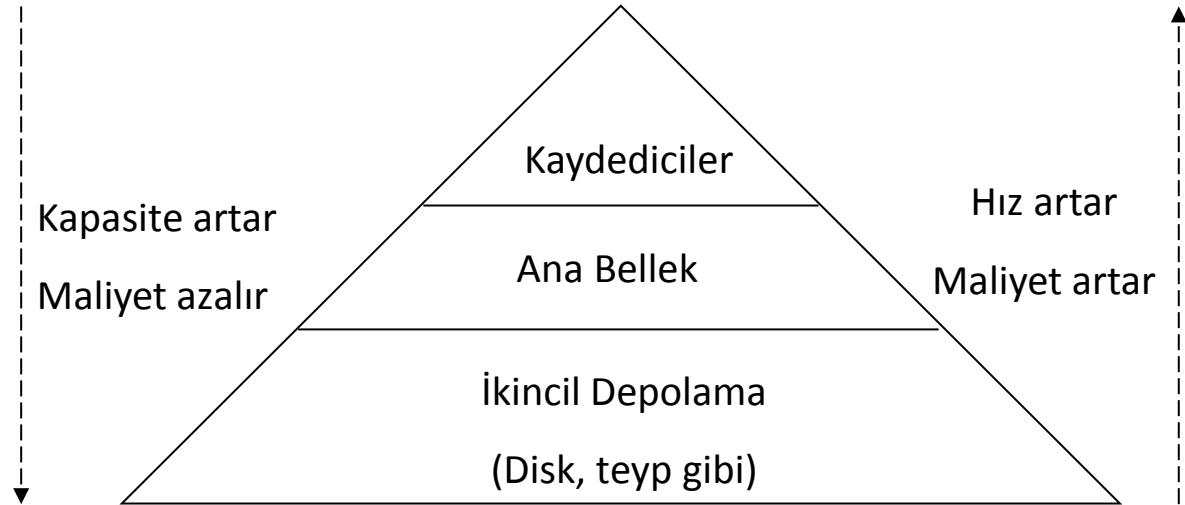
ROM Bellek

RAM Bellek

CPU'nun Gereksinim Duyduğu Registerler

BELLEK (Memory)

İkili bilginin yüklenmesi ve işlenmesi aşamalarında şimdiye kadar flip flopları ve kaydedicileri gördük. Fakat bir bilgisayar sisteminde var olan kaydedicilerin sayısı birkaç yüz kadardır diyebiliriz. Maliyet, enerji tüketimi ya da işlemcide kapladığı yer bakımından bu sayı sınırlı tutulmaktadır. Oysa bir bilgisayar sisteminde çalıştırmak istediğimiz programlar, bu kaydedicilerde tutulamayacak kadar büyüktür. Bilgisayar sistemlerinde bellek, hiyerarşik yapıdadır.



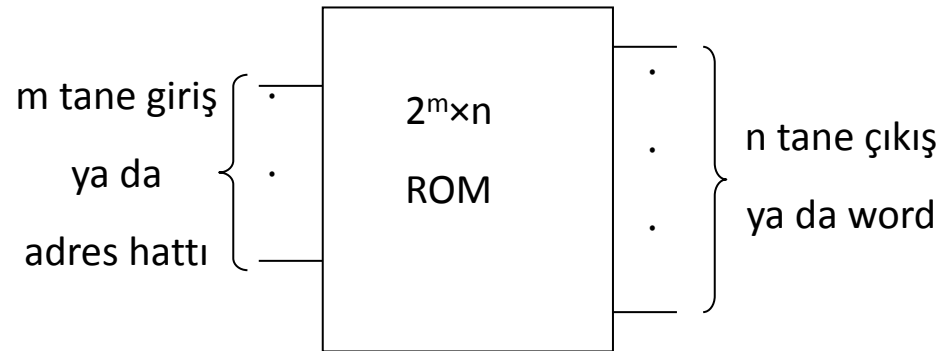
Bellek Türleri

Temel yarı iletken bellek türleri **RAM** (Random Access Memory) ve **ROM** (Read Only Memory)'dur. Rastgele erişilen belleklerden okuma ve yazma işlemleri yapılabilirken ROM türü belleklerden sadece okuma yapılabilir. RAM bellekler uçucu bir yapıya sahiptir yani beslemesi kesildiğinde üzerindeki bilgiler kaybolur.

Bu bellek elemanlarında tutulan ikili bilgi, **byte** ve **word** tabirleriyle anılır. Byte terimi 8 bitlik ikili bilgi için kullanılan teknik bir tabirdir. Word ise işlemci mimarisine göre değişen bilgi miktarını içerir. Bilgisayar mimarisine göre word'ün uzunluğu 1 byte, 2 byte, 4 byte ya da 8 byte olabilir.

ROM Bellek

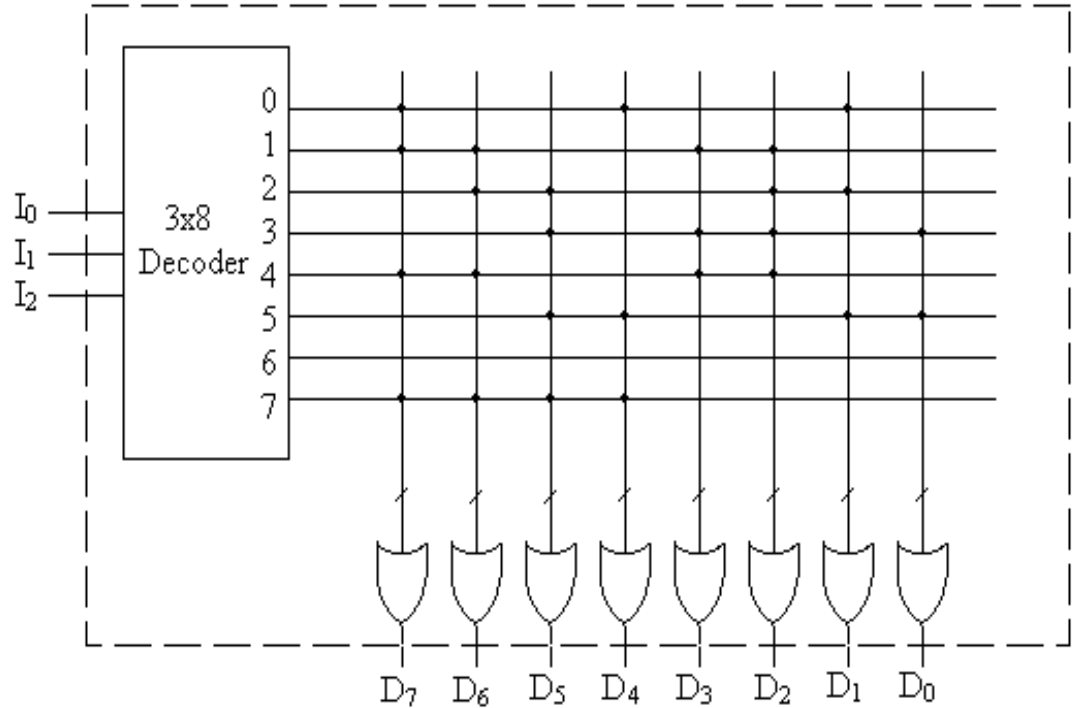
Aslında ROM yapı itibarıyla, kod çözücü ve VEYA kapılarından oluşan bir kombinasyonel devre olarak düşünülebilir.



ROM Bellek

Örnek olarak 8×8 bitlik bir ROM'un yapısı aşağıdaki gibidir;

Girişler			Çıkışlar
I ₂	I ₁	I ₀	D ₇D ₀
0	0	0	10010010
0	0	1	11001100
0	1	0	01100110
0	1	1	00101101
1	0	0	11001100
1	0	1	00110011
1	1	0	00000000
1	1	1	11110000



ROM, 8 bitlik 8 lokasyondan oluşmaktadır. Dolayısıyla 8 lokasyonu adresleyebilmek için 3 adres hattına ihtiyaç vardır.

ROM Türleri

ROM: Kalıcı datayı tutmak için maskeleme tekniğiyle üretilirler.

PROM: Programlanabilir ROM.

EPROM: Ultra Viole ışığıyla silinebilir PROM.

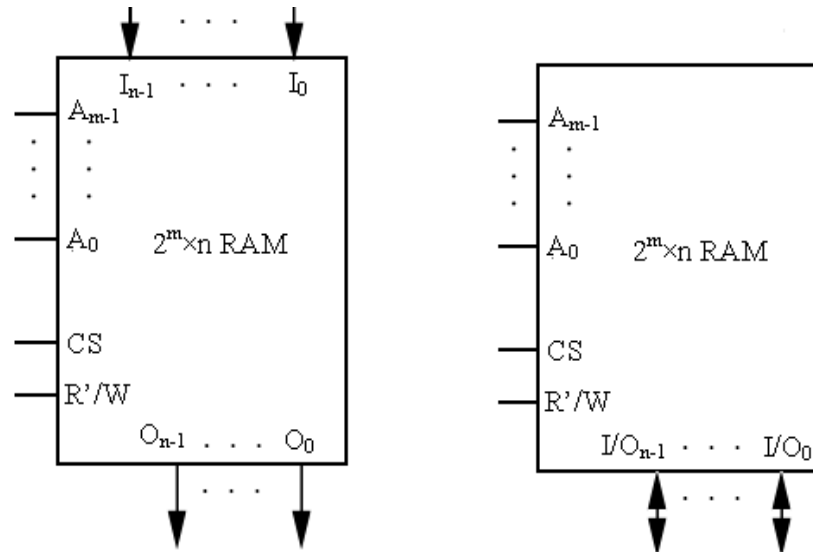
EEPROM: Elektriksel olarak silinebilen PROM.

FLASH: EEPROM'a benzer yapıdadır, ancak veriler blok olarak okunup yazılabilir.

RAM (Random Access Memory)

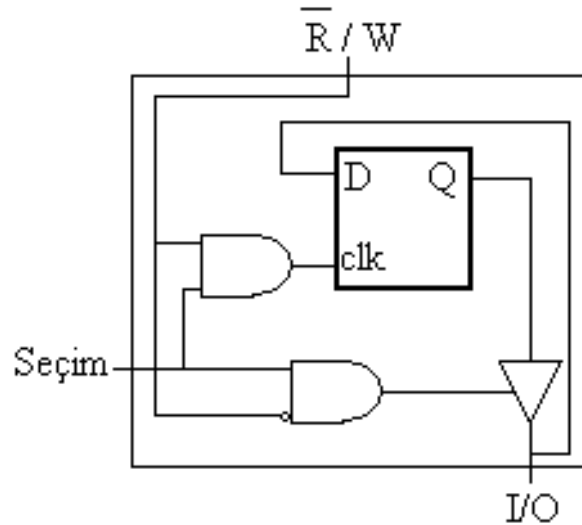
Uçucu (volatile) yapıya sahiptirler. RAM belleklerde herhangi bir lokasyona erişim süresi aynıdır. Bu durum ROM bellekler için de geçerlidir. Oysa kalıcı belleklerde erişim süresi aynı değildir.

m bitlik adres alanına sahip bir RAM bellek, 2^m sayıda lokasyona sahiptir. Lokasyonlar, 0 ile (2^m-1) arasında numaralandırılır. Her bir lokasyonda n bitlik ikili bilgi tutulur. Bu n bitlik veriye bellek sözcüğü (memory word) denir. Bellek isimlendirilirken, $2^m \times n$ -bit tabiri kullanılır.



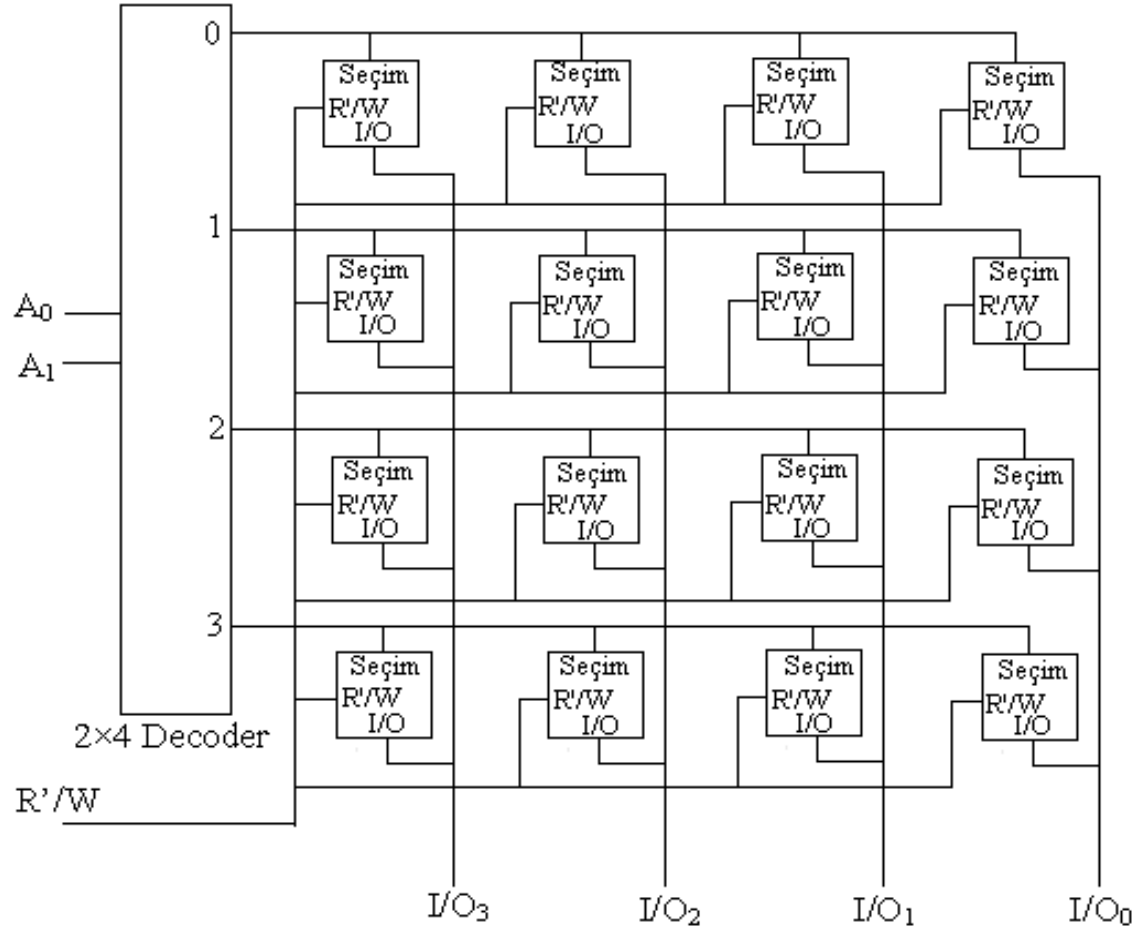
RAM Bellek Hücresi

RAM bellek yapı itibariyle 1 bitlik veriyi tutan bellek hücrelerinden oluşur. Fonksiyonellik açısından örnek bir gerçekleştirim aşağıdaki gibidir:



RAM Chip'i

Bellek hücreleri bir araya getirilerek RAM chip'leri oluşturulur. Aşağıda 4×4-bitlik bir RAM chip'i gösterilmiştir:



RAM Organizasyonu

Kapasitesi az olan bellekler için tek kod çözücü ile satır bazında bellek hücrelerine erişilirken, büyük kapasiteli belleklerde iki kod çözücü kullanılarak satır-sütun bazında bellek hücrelerine erişilir.

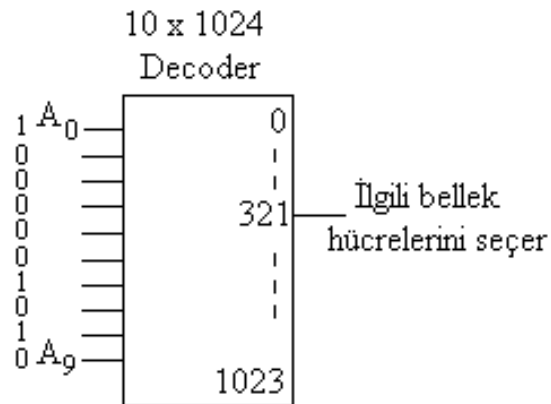
Örneğin kapasitesi 1K Word olan bellek için 321 (0101000001_2) adresindeki veriye erişmek istediğimizi düşünelim. Belleğimiz 2 türlü organize edilebilir:

1.Yöntem: Belleğimizi adresleyebilmek için 10 bite ($2^{10}=1K$) ihtiyacımız vardır. Dolayısıyla 10×1024 kod çözücü kullanmamız gerekecektir.

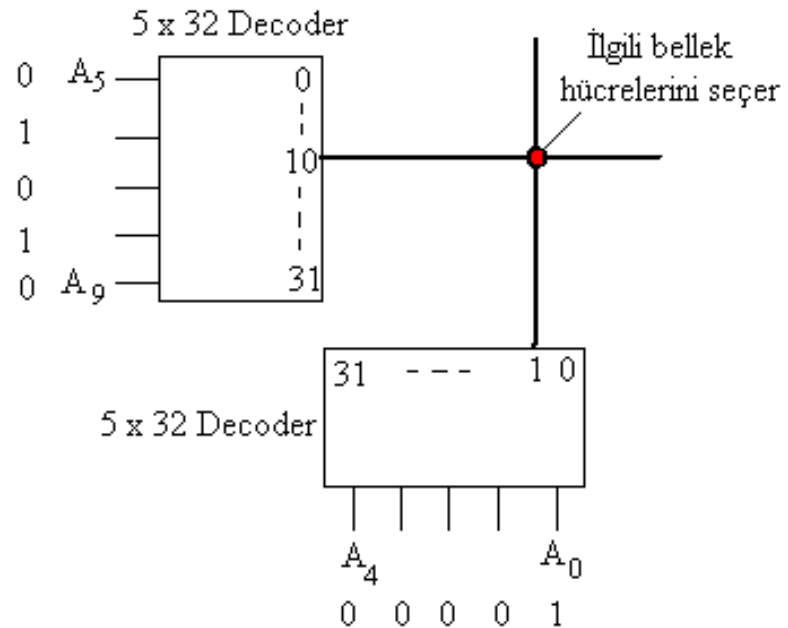
2.Yöntem: 2 tane 5×32 kod çözücü kullanarak ($2^5 \cdot 2^5 = 2^{10}$). Erişmek istediğimiz adres iki kısma ayrılır ve kod çözücülere uygulanır.

Satır	Sütun
<u>01010</u>	<u>00001</u>
$(10)_{10}$	$(1)_{10}$

RAM Organizasyonu



1.Yöntem



2.Yöntem

RAM Türleri

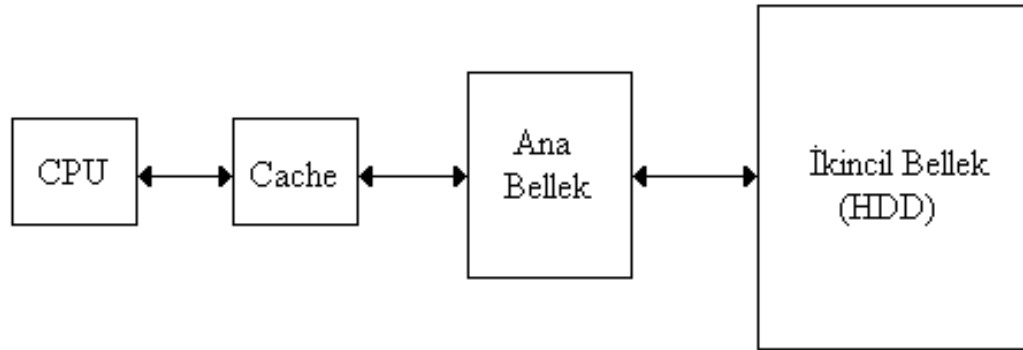
Statik (**SRAM**) ve dinamik (**DRAM**) olmak üzere ikiye ayrılır.

Statik RAM'lere erişim süresi dinamik RAM'lere göre daha hızlıdır. Fakat bit başına kapladığı yer daha fazladır. Statik RAM'ler bit başına 6 transistör, dinamik RAM'ler ise 1 transistör ve 1 kapasitör içerir. Dinamik RAM'ler kapasite elemanı içerdiğinden dolayı belli aralıklarla tazelenmesi (refresh) gerekir.

Maliyet açısından dinamik RAM'ler daha ekonomiktir. Dinamik RAM'ler ana bellek olarak kullanılırken, statik RAM'ler cache bellek olarak kullanılmaktadır. Cache belleğe, ana bellekteki programların belli bir bloğu aktarılarak, CPU'nun bilgiye daha hızlı erişmesi sağlanır. Kombine kullanım sayesinde performans-maliyet oranı dengelenmeye çalışılmaktadır.

Bellek Hiyerarşisi

Bir bilgisayar sistemindeki bellek hiyerarşisi aşağıda gösterilmiştir.



CPU'nun Gereksinim Duyduğu Registerler

CPU, bellek ve giriş/çıkış (I/O) cihazları ile haberleşeceğinden dolayı bazı registerlere ihtiyaç duyacaktır. Bunlar;

PC (Program Counter)

IR (Instruction Register)

DR (Data Register)

AR (Adres Register)

ACC (Accumulator)

INR (INput Register)

OUTR (OUTput Register)

TR (Temporary Register)

CCR (Condition Code Register)

IX (Index Register)

SP (Stack Pointer)

BÖLÜM 5. KOMUT SETİ MİMARİSİ

Adresleme Metotları

- İvedi (Immediate) Adresleme
- Direkt (Direct) Adresleme
- Dolaylı (Indirect) Adresleme
- İndis (Index) Adresleme
- Göreceli (Relative) Adresleme
- Doğal (Inherent) Adresleme

Register Transfer Dili

KOMUT SETİ MİMARİSİ

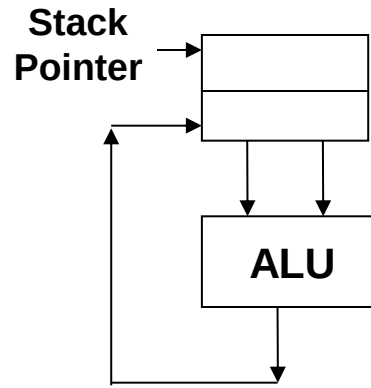
CPU, komutlar üzerinde işlem yapar. Komutlar ise CPU'nun yapması gereken işleri tanımlar. Bir komut, işlem bilgisini (opcode) ve adres bilgisini içerir. Belirtilen adresteki veri operand olarak anılır. Operand ise bellekte ya da kaydedicide olabilir.

Bir işlemcinin komut seti, operandın nereden alınacağına bağlı olarak tasarlanır. Genel olarak 3 tip mimari göze çarpmaktadır;

- 1. Yığın (Stack) mimarisi**
- 2. Akü mimarisi**
- 3. Genel amaçlı register mimarisi**

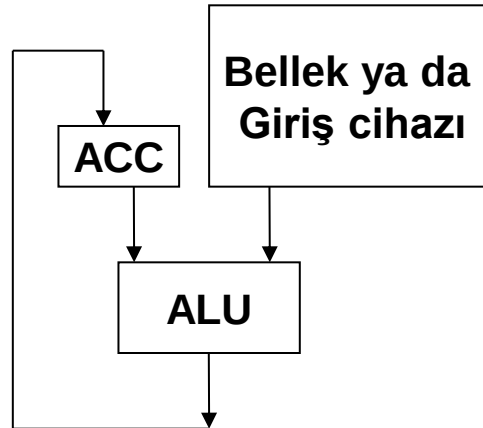
Yığın (Stack) Mimarisi

İşlemler yığının üst elemanları üzerinde yapılır ve sonuç yine yığına atılır. Bu mimaride yığına bilgi aktarma işlemi *push* ve yığından bilgi alma işlemi de *pop* komutlarıyla olur.



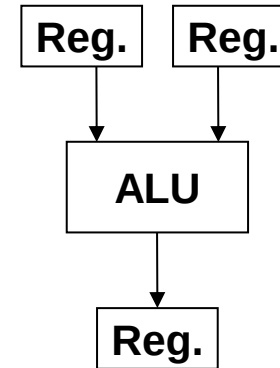
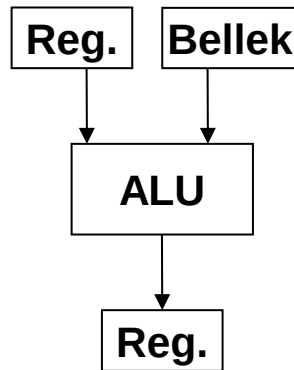
Akü Mimarisi

Operandlardan biri bellekte diğeri ise aküdedir (ACC). İşlem yapılır ve sonuç aküye transfer edilir. Akü, komutlarda ayrıca belirtilmez (imalıdır). İleride tasarlayacağımız temel bilgisayar sistemimizde bu mimariyi kullanacağız.



Genel Amaçlı Register Mimarisi

Bu mimaride operandlar, register ya da bellek olarak belirtilir. Register-bellek ya da register-register tipinde olabilir. Operandlar üzerinde işlemler yapıldıktan sonra sonuçlar kaydedicilere aktarılır.



Örnek: Bellekteki iki sayıyı (sayi_1 ve sayi_2) toplayıp, sonucu (toplam) tekrar belleğe koymak istediğimizi düşünelim.

Stack mimarisinde,

Push sayi_1
Push sayi_2
Add
Pop toplam

Akü mimarisinde,

load sayi_1
add sayi_2
store toplam

Register-bellek mimarisinde,

load Reg_1, sayi_1
Add Reg_2, Reg_1, sayi_2
Store Reg_2, toplam

Register-register mimarisinde,

load Reg_1, sayi_1
load Reg_2, sayi_2
Add Reg_3, Reg_1, Reg_2
store Reg_3, toplam

Adresleme Metotları

Bir komutun, opkod ve adres kısımlarından oluştuğundan bahsedilmişti. Komutun adres kısmı, üzerinde işlem yapılacak operandı gösterir. Operand bellekte ise operandın adresi, etkin adres (effective address) diye anılır.

1. İvedi (Immediate) Adresleme: Bu adresleme modunda operand, komutun bir parçasıdır. Yani operand, komutun adres kısmındaki veridir.

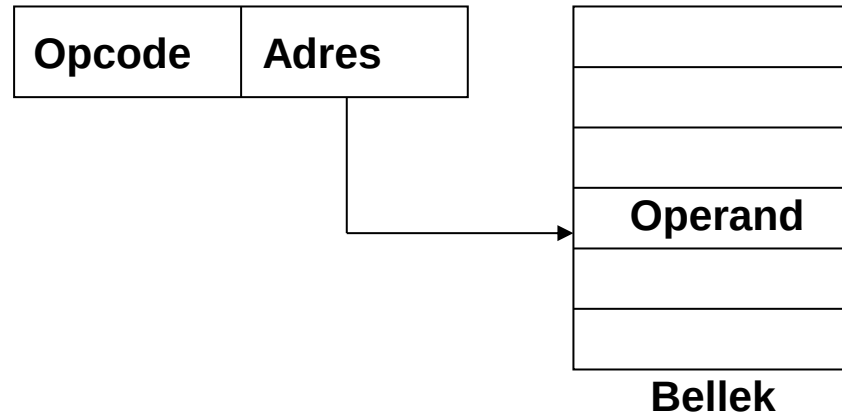
Opcode	Operand
--------	---------

Örnek: *Add #3* komutunda ‘3’ operanttır. Bu komut akünün içeriğine 3 değerini ekler ve sonucu aküye yazar. Belleğe erişim gerektirmeyen hızlı icra edilen bir komuttur. # sembolü operandın kendisini göstermek için kullanılmıştır

Yüksek seviyeli dillerde değişkenlere sabit bir başlangıç değeri atamak için de bu adresleme metodu kullanılabilir. (x=3)

Adresleme Metotları

2. Direkt (Direct) Adresleme: Bu adresleme metodunda komutun adres kısmı, operandın adresini gösterir.

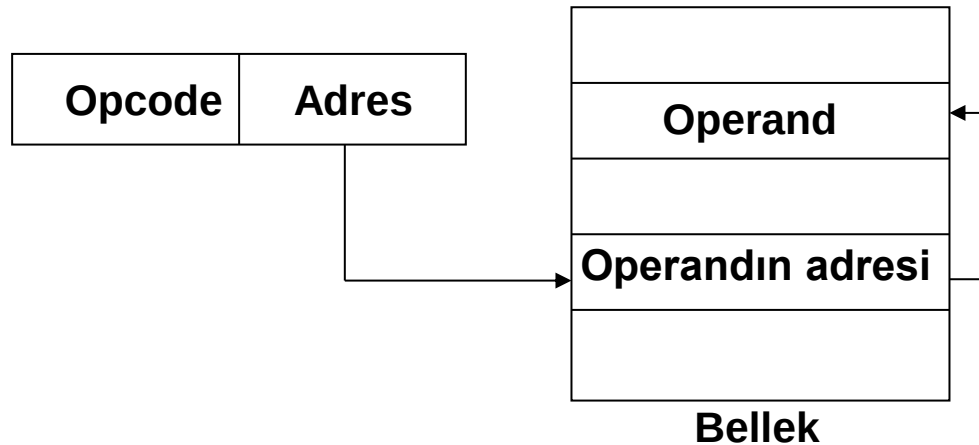


Örnek: *Add 100* komutu, akünün içeriğine 100 numaralı bellek bölgesinde tutulan değeri ekler ve sonucu aküye yazar.

- Bu adresleme metodunda bellek referans edildiğinden, komutların işletim zamanı ivedi moda göre daha uzundur
- Sınırlı adres uzayına erişim mümkündür (Adres kısmının içerdiği bit sayısı kadar).

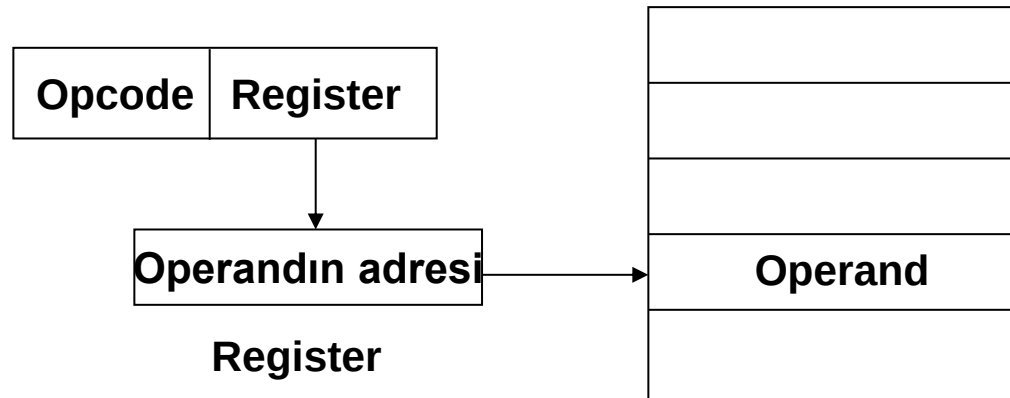
Adresleme Metotları

3. Dolaylı (Indirect) Adresleme: Bu adresleme metodunda komutun adres kısmı, operandın adresini göstermez; operandın etkin adresinin tutulduğu bir register'i ya da bellek bölgesini gösterir.



Örnek: *Add (100)* komutu akünün içeriğine, adresi 100 numaralı bellek bölgesinde olan operandın değerini ekler ve sonucu aküye yazar. ‘()’ sembolleri dolaylı adreslemeyi belirtmek için kullanılmıştır.

Register - dolaylı adresleme modunda, operandın etkin adresi register'da tutulur.



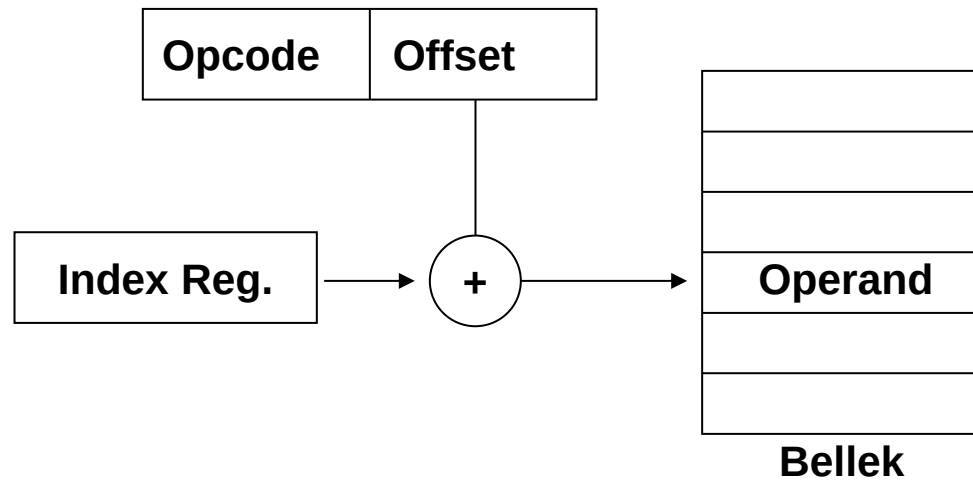
Örnek: *Add (R1)*

Özetle;

- Bu adresleme metodunu kullanan komutların işletim zamanı, direkt adreslemeye göre daha uzundur
- Operanda erişim için ekstra efektif adres hesabı gerektirir.
- Adres uzayını genişletmek mümkündür.

Adresleme Metotları

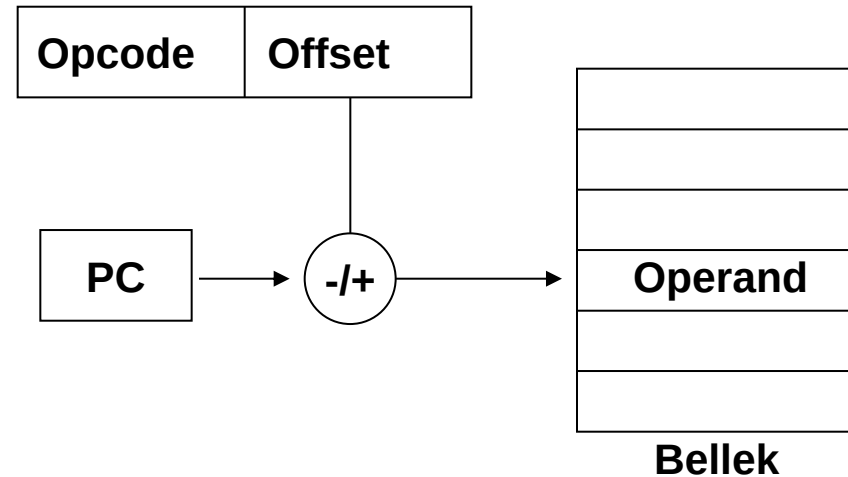
4. İndis (Index) Adresleme: Bu adresleme modunda bir index registerinden faydalanılır. Özellikle bir dizinin elemanlarına sıralı erişim söz konusu ise kullanılabilir. Ulaşılmak istenen dizinin başlangıç adresi index registerine atanır. İstenen dizi elemanının ofset değeri, komutun operand kısmında yer alır.



Örnek: `main {
 k=A[2];
}`

Adresleme Metotları

5. Göreceli (Relative) Adresleme: Program sayacının (PC) mevcut değerinin yakınlarında bir yere dallanma yapar.



Örnek: `main{
 if (i==1) i++;
 else i--;
}`

Adresleme Metotları

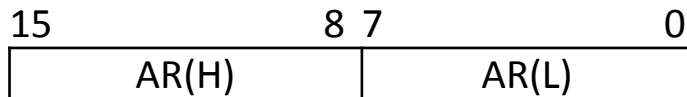
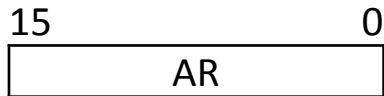
6. Doğal (Inherent) Adresleme: Operand bilgisi içermeyen adresleme modudur.

Örnek: Clr (Akünün içeriğini temizler)
Inc (Akünün içeriğini 1 arttırır)

Register Transfer Dili

Bir bilgisayar sistemindeki en temel işlem, registerlar arası bilgi transferleridir. Bu dil, bilgisayar sisteminin donanımsal yapısını ve davranışını tam bir biçimde ifade eder. Donanım tasarım dili (Hardware Description Language-HDL) olarak da bilinir.

- Registerlar genellikle büyük harflerle ifade edilirler. Ayrıca harflerden sonra rakam bilgileri de içerebilirler. n bitlik bir kaydedici içerisindeki flip floplar 0'dan (n-1)'e kadar numaralandırılırlar. Temel bilgisayar sistemimizdeki kaydediciler 8 ya da 16 bitlidir.



- Yüksek anlamlı kısım AR(8-15) ya da AR(H)
- Düşük anlamlı kısım AR(0-7) ya da AR(L)
- Belli bir bit grubu, AR(0-3) ya da AR_{0-3} şeklinde ifade edilir.

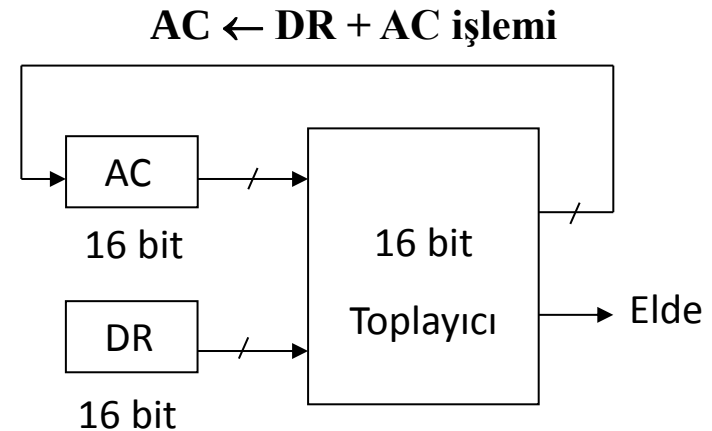
Register Transfer Dili

- Register transfer işlemi ' $\leftarrow$ ' sembolü ile gösterilir ve aktarımın yönünü belirtir.

Örneğin $AR \leftarrow PC$ ifadesi PC'nin içeriğinin AR'ye aktarılacağını belirtir (Bit sayılarının uyumlu olması gerektiği unutulmamalıdır). Bu aktarım sonucunda PC'nin içeriği korunur.

- Aritmetik ya da mantık işlemlerinin sonuçları da kaydedicilere aktarılabilir;

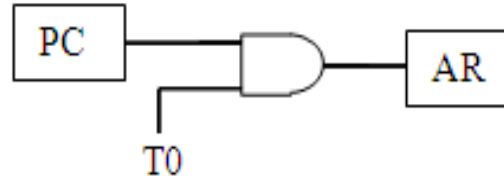
$AC \leftarrow DR + AC$	Toplama
$AC \leftarrow DR - AC$	Çıkarma
$AC \leftarrow AC'$	Tümleyen
$AC \leftarrow AC' + 1$	2'ye tümleyen
$AC \leftarrow AC \cap DR$	VE işlemi
$AC \leftarrow AC \cup DR$	VEYA işlemi
$AC \leftarrow SHR(AC)$	Sağa kaydırma
$AC \leftarrow SHL(AC)$	Sola kaydırma



Register Transfer Dili

- Transfer işlemi bir kontrol sinyali ile olacaksa, bu şart “Kontrol Sinyali :” şeklinde ifade edilir.

Örneğin T0 : $AR \leftarrow PC$



- Belli bir kontrol sinyaline bağlı olarak, birden fazla aktarım (mikro işlem) eş zamanlı olarak yerine getirilmek isteniyorsa “,” ile belirtilir;

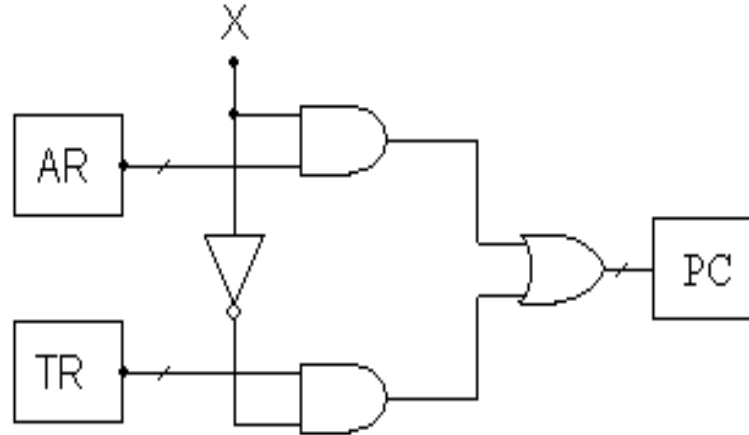
Örneğin X: $AR \leftarrow PC, PC \leftarrow PC + 1$

Register Transfer Dili

Şayet X kontrol sinyalinin durumuna göre değişik atamalar yapılacaksa (if-else yapısı) aşağıdaki notasyon kullanılır;

$X : PC \leftarrow AR$

$X' : PC \leftarrow TR$



BÖLÜM 5. KOMUT SETİ MİMARİSİ

Temel Bilgisayar Sistemimizin Bileşenleri

Düşük ve Yüksek Anlamlı Veri Yolunun Tahsisi

Komut Tasarımı

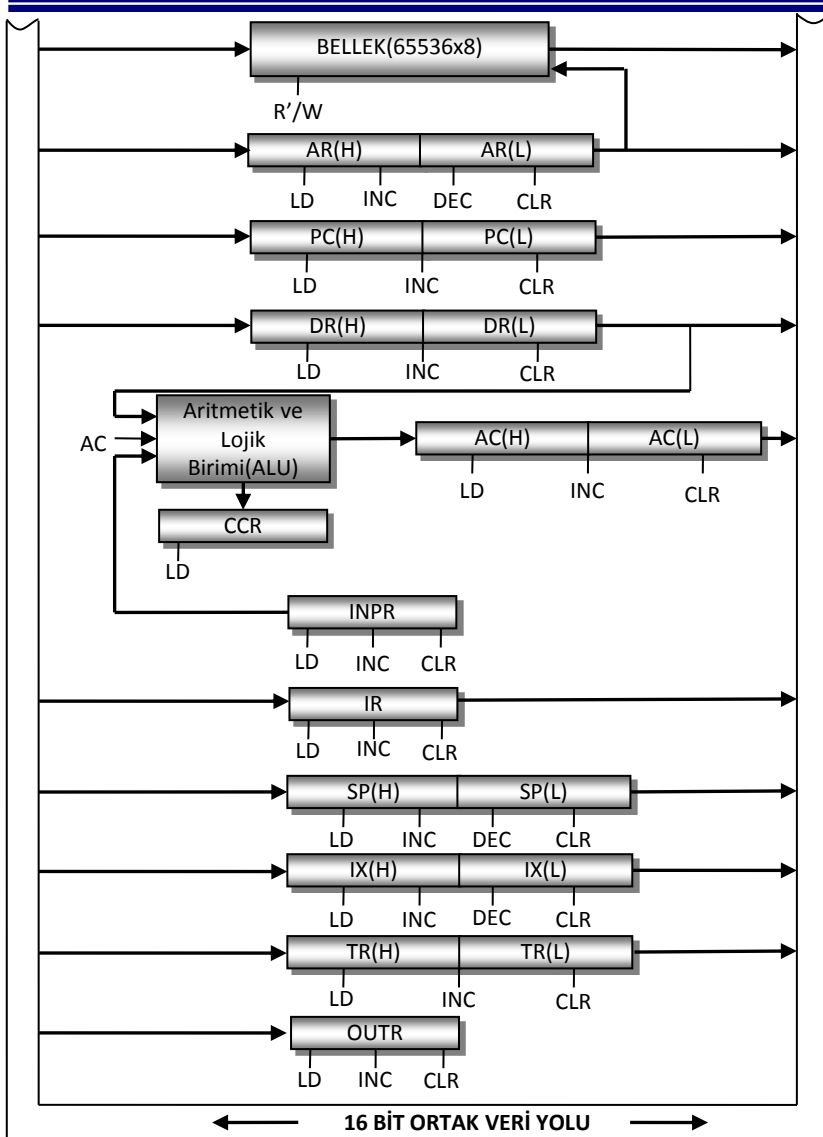
Adresleme Modu Seçim Bitleri ve OPCODE

Adresleme Modu Bilgisinin Çözümlemesi

Komutun İşlem Tipi Bilgisinin Çözümlemesi

Komutun Çözümlemesi

Temel Bilgisayar Sistemimizin Bileşenleri



Bellek, 64 KB olarak düşünülmüştür. Belleğin her gözü de 8 bitlik veriyi tutmaktadır.

Ayrıca komut tasarımında bazı komutların 1 byte yer kaplayabileceği de düşünülerek, belleğin her bir gözünün 8 bitlik olmasına karar verilmiştir (Ayrıca komutların OPCODE kısmı da 8 bitlik olduğundan).

11 adet kaydedici ve bellek ortak veri yoluna bağlanmışlardır. Hangi elemanın hangi durumda veri yoluna bilgi aktaracağına, 4×16 'lık kod çözücülerin girişlerine gelen veri ile karar verilir.

ALU'da gerçekleştirilecek işlemler 16 bitlik olacağından ve bellek de 8 bitlik olduğundan, veri yolu 2 kısım olarak düzenlenmiştir; veri yolunun düşük ve yüksek anlamlı kısmı diye. Bu sebepten ötürü 2 kod çözücü kullanılmıştır.

Düşük ve Yüksek Anlamlı Veri Yolunun Tahsisi

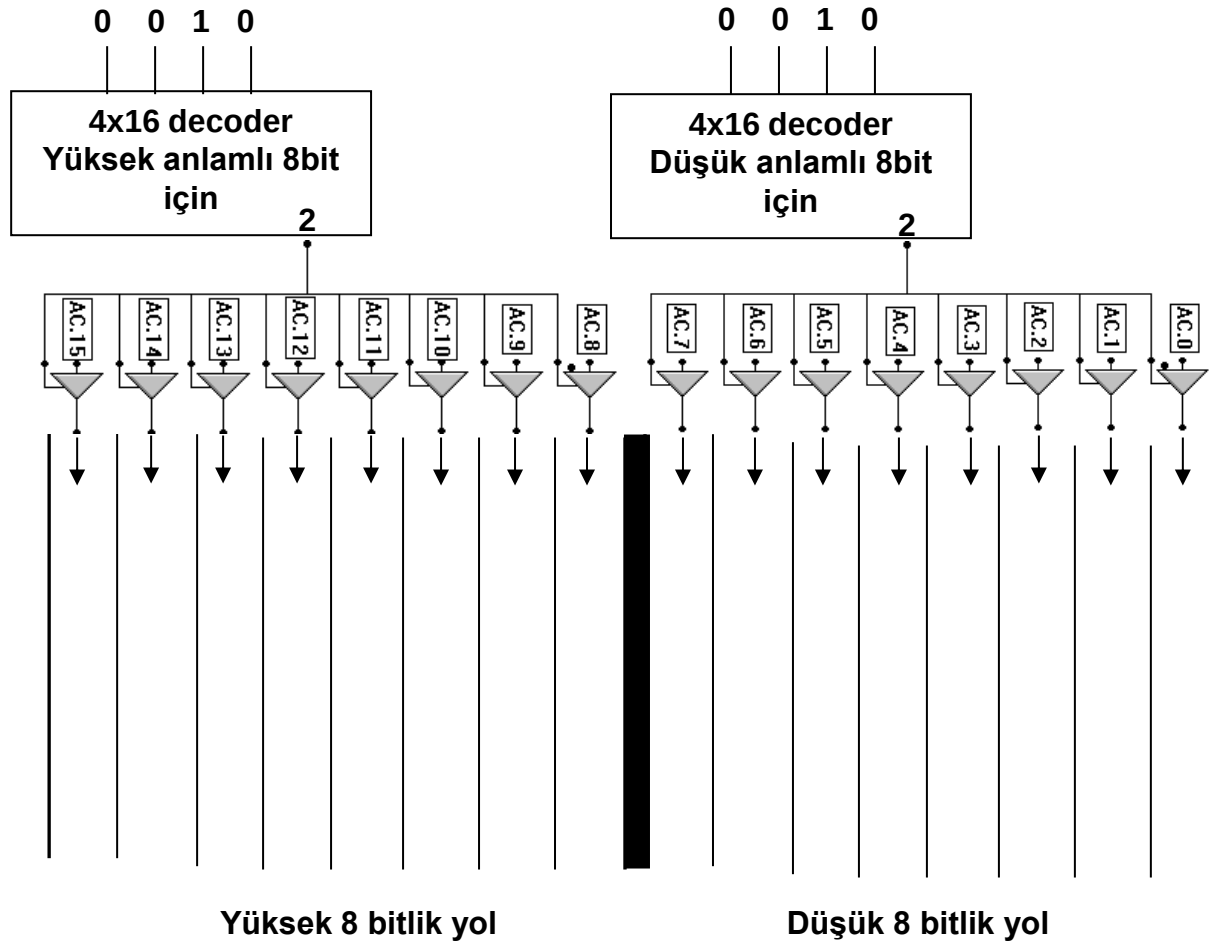
Bir kaydedici veya belleğin verisini yola aktarmak için kod çözücülerin girişlerine uygulanması gereken ikili değerler aşağıdaki tablodaki gibi seçilmiştir.

Kod Çözücü Girişi	Yolu Kullanan Birim
0001	Yığın göstericisi (SP)
0010	Akümülatör (AC)
0011	Program sayacı (PC)
0100	Komut kaydedicisi (IR)
0101	Veri kaydedicisi (DR)
0110	İndis (Index) kaydedicisi (IX)
0111	Geçici Kaydedici (TR)
1000	Adres Kaydedicisi (AR)
1001	Bellek (MEM)
1010	İndis ve Göreceli Adreslemede kullanılır
1100	Durum Kod Kaydedicisi (CCR)

Durum kod kaydedicisi ve komut kaydedicisi dışındaki kaydediciler 16 bitlidir. 16 bitlik kaydediciler için ikinci bir kod çözücü daha kullanılmıştır (Yüksek anlamlı veri yolu).

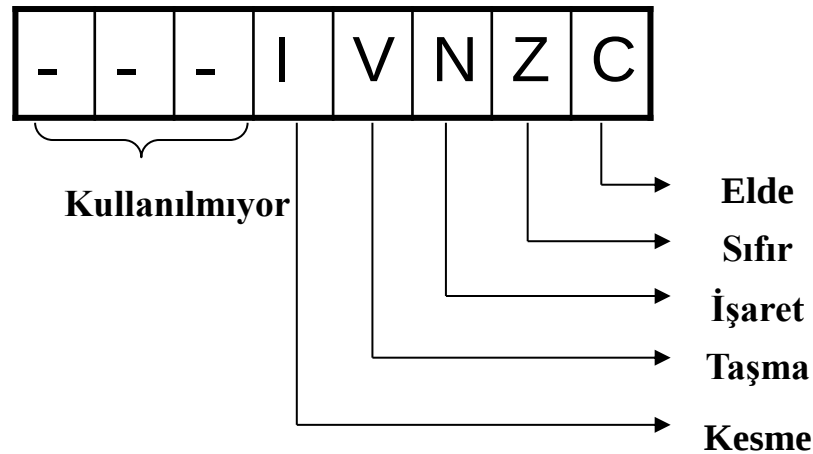
Düşük anlamlı yol tahsisini yapan kod çözücü giriş bitleri ile yüksek anlamlı yol tahsisini yapan kod çözücünün giriş bitleri aynıdır.

Örnek Gerçekleştirim: Akünün İçeriğinin Yola Aktarılması



Durum Kod Kaydedicisi (CCR)

ALU ile birlikte çalışır. Bayrak kaydedicisi olarak da anılır. Bayrak kaydedicisi, bütün mikroişlemcilerde olduğu gibi, bir işlemin sonucuyla ilgili bazı bilgileri içerir ve bu bilgiler daha sonra işletilecek komutlar üzerinde etkiye sahiptirler. Özellikle karar vermeye dayalı komutların yürütülmesinde sıklıkla kullanılırlar.



Komut Tasarımı

Program, CPU'nun yapması gereken işlemleri tanımlayan komutlar dizisidir. Ayrıca işlemlerin hangi sıra dahilinde yapılacağını da gösterir. Bir bilgisayar komutu, ikili (binary) bir koddur ve bir dizi mikro işlem adımını tanımlar.

Komutlar verilerle birlikte bellekte bulunurlar. Bilgisayar her komutu bellekten okur bunu komut kaydedicisine yazar. Kontrol birimi bu kodu çözer ve gereken mikro işlemleri sırasıyla icra eder.

Komut kodu bir grup bitten ibarettir. Bunlar bilgisayara belli bir işlemi yapmasını söylerler. Bir komut birkaç kısımdan meydana gelir. Komutun en önemli parçası *işlem* parçasıdır. İşlem kısmını oluşturan bitler toplama, çıkarma, çarpma, kaydırma ve tümleme gibi bazı işlemleri tanımlarlar. Bir komuttaki işlem bitlerinin sayısı, bilgisayarda kullanılacak işlem sayısına bağlıdır. n tane bit ile 2^n (veya daha az) işlem tanımlanabilir.

Komut kodunda yer alan diğer önemli kısım ise adresleme modunu belirleyen kısımdır. Temel bilgisayar sistemimizde komut kodunun 5 biti işlem koduna ayrılmıştır. Beş bitlik bir işlem koduyla 32 adet işlem yerine getirilebilmesine rağmen, bazı komutların bazı adresleme modlarına gereksinimi olmamasından ötürü, kullanılmayan durumlar başka komutlara tahsis edilmiş ve yaklaşık 60 farklı işlem tanımlanabilmiştir.

Komut Tasarımı

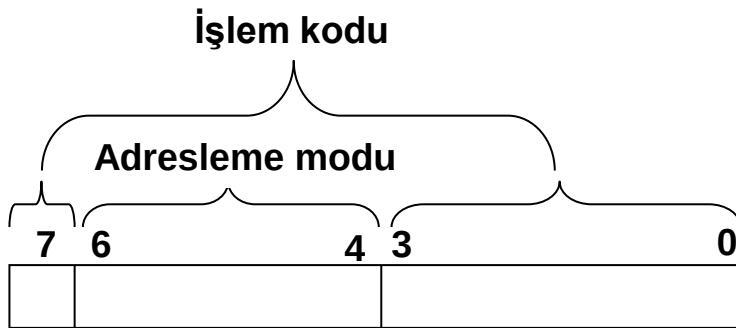
6 tip adresleme modu kullanıldığından dolayı komut kodunun 3 biti, adres alanı için tahsis edilmiştir.

Temel bilgisayar sistemimizde ivedi, direkt ve dolaylı adresleme modlarına sahip komutlar 3 byte,

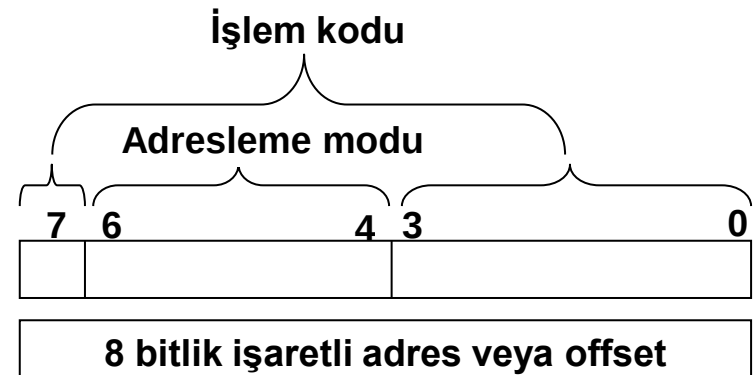
indis ve göreceli adresleme modlarına sahip komutlar 2 byte,

doğal adresleme moduna sahip komutlar ise 1 byte yer kaplamaktadır.

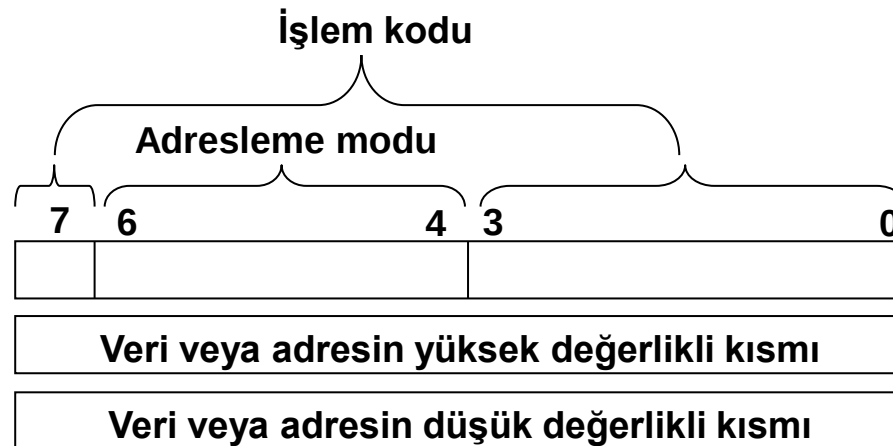
Komut Formatı



Doğal adreslemede



İndis ve göreceli adreslemede



İvedi, direkt ve dolaylı adreslemede

Adresleme Modu Seçim Bitleri

Adresleme modunun seçimi için 3 bit gerekmektedir.

Seçim Bitleri	Adresleme Modu
000	Doğal Mod
001	İvedi Mod
010	Direkt Mod
011	Dolaylı Mod
100	İndis Mod
101	Göreceli Mod

Adresleme modu değiştiğinde opcode'un da sıra ile değişmesini temin etmek için, işlem tipinin 1 biti opcode'un baş tarafına alınmıştır

Örnek: Temel bilgisayar sistemimizde toplama (ADD) komutunun işlem tipi 00000_2 olarak seçilmiştir.

İvedi adresleme için opcode kısmı;

0	001	0000
---	-----	------

Opcode'un karşılığı 10h

Direkt adresleme için opcode kısmı;

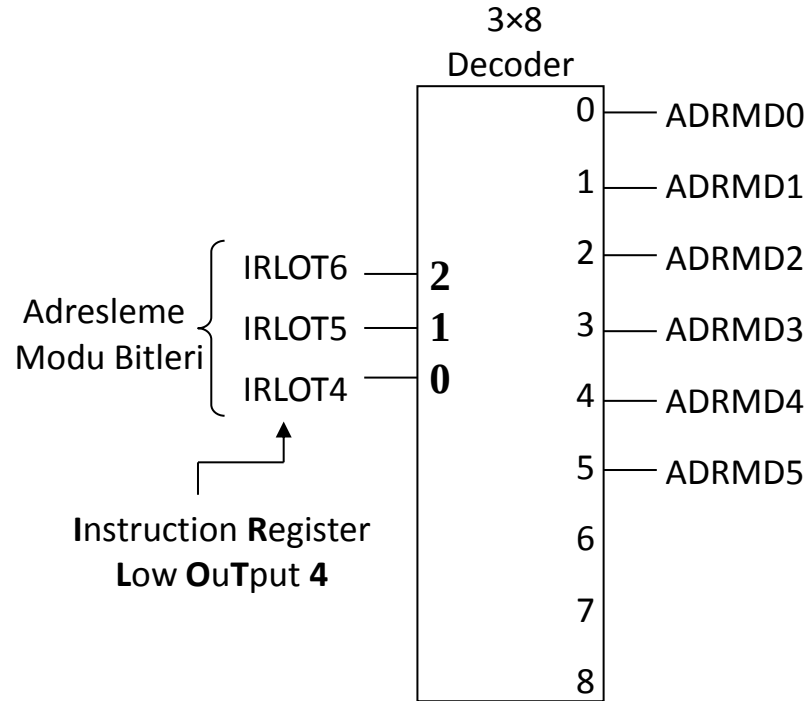
0	010	0000
---	-----	------

Opcode'un karşılığı 20h

Adresleme Modu Bilgisinin Çözümlemesi

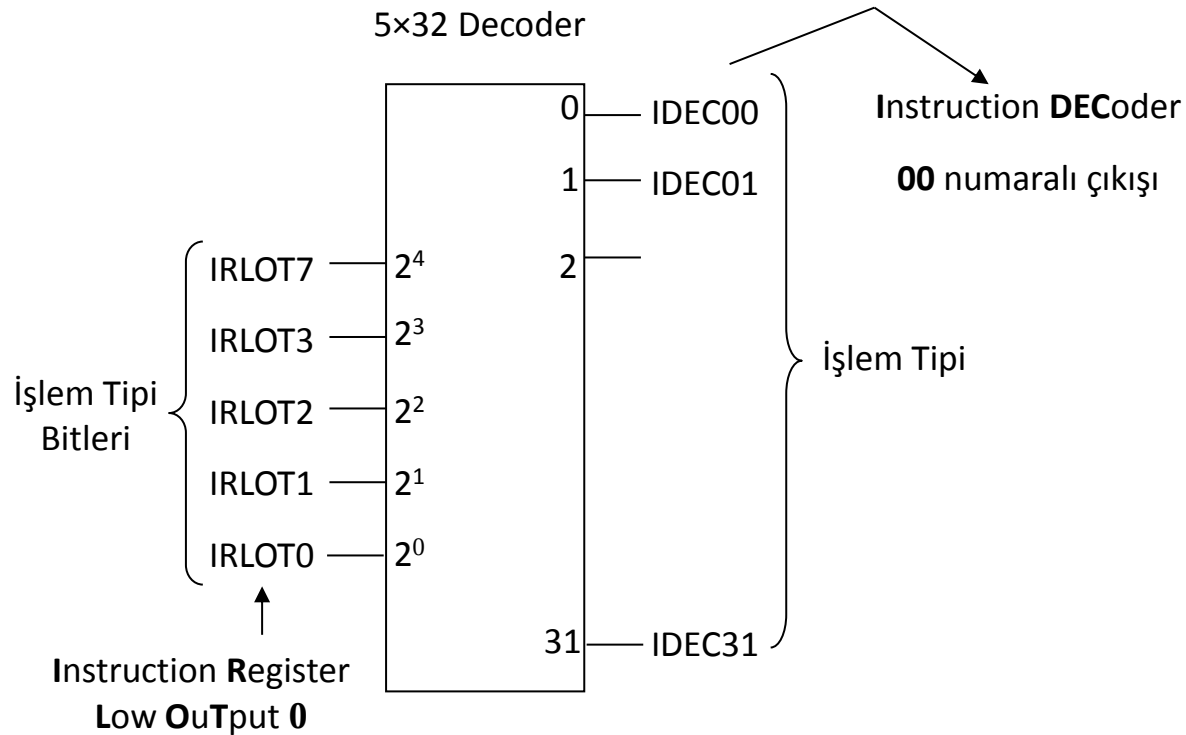
Bellekteki komutlar işletilirken, opcode kısımları komut kaydedicisine aktarılmaktadır. Opcode'un 4, 5 ve 6. bitleri adresleme modunu tayin etmeye yaramaktadır. Bu üç bit, 3×8 'lik bir kod çözücünden geçirilerek 8 adet çıkış elde edilmiştir. Temel bilgisayar sistemimizde bunların 6 tanesi kullanılmıştır. Adresleme modları aşağıdaki kısaltmalarla isimlendirilmiştir;

Kontrol Adı	Adresleme Modu Tipi
ADRMD0	Doğal Mod
ADRMD1	İvedi Mod
ADRMD2	Direkt Mod
ADRMD3	Dolaylı Mod
ADRMD4	İndis Mod
ADRMD5	Göreceli Mod



Komutun İşlem Tipi Bilgisinin Çözülmesi

Komut kaydedicisindeki opcode'un 0, 1, 2, 3 ve 7. bitleri işlem tipini tayin etmeye yaramaktadır. Bu beş bit, 5×32 'lik bir kod çözücünden geçirilerek 32 adet çıkış elde edilmiştir.



Komutun Çözülmesi

Adresleme modunu ve işlem tipini çözen donanımsal düzeneklerden gelen sinyallere göre komut çözülmektedir.

Örneğin komut kaydedicisindeki işlem tipini gösteren 5 bitin 00101_2 olduğunu varsayalım. Bu durumda işlem tipini çözen kod çözücünün 5 numaralı çıkışı yani IDEC05 sinyali aktif olacaktır.

Bu sinyal, tek başına komutun tipini belirlemeye yeterli olmayacaktır. Çünkü aynı sinyali üreten başka komutlar da olabilir. Temel bilgisayar sistemimizin komut setini oluştururken, hem bölme komutu (DIV) hem de ikiye tümleyen alma komutu (NEG) için aynı işlem tipi seçilmiştir. Çünkü bölme komutu doğal adresleme modunu kullanmamaktadır (her zaman operand bilgisi aldığından). Yani işlem tipinin 5 ve adresleme modunun 0 olduğu kombinasyon ile NEG komutu tanımlanmıştır.

Komutun Çözümlemesi

Komut	Komutun Opcode'u				
	Doğal	İvedi	Direkt	Dolaylı	İndis
DIV	-	15h	25h	35h	45h
NEG	05h	-	-	-	-

IDEC05.ADRMD0 şartı NEG komutunu aktive edecek,

IDEC05.ADRMD1
IDEC05.ADRMD2
IDEC05.ADRMD3
IDEC05.ADRMD4

} şartlarından herhangi biri ise DIV komutunu aktive edecektir.

Bu aktivasyon sinyalleri, komutların icrası ile ilgili donanımsal düzenekleri harekete geçirmek için kullanılacaktır.

BÖLÜM 5. KOMUT SETİ MİMARİSİ

Komut Saykılı

Zamanlama Düzenegi

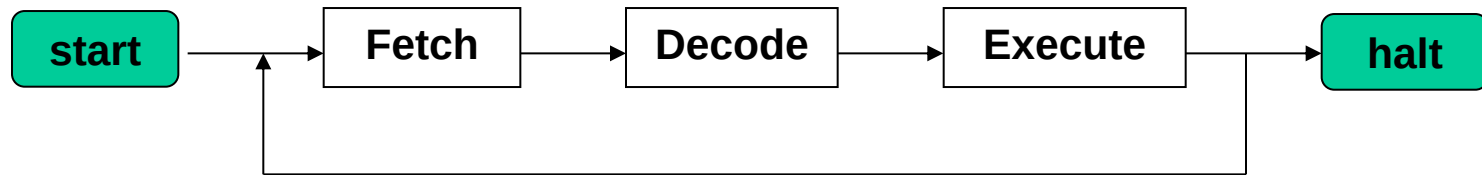
Komut Saykılıının Fetch ve Decode Safhaları

Komut Saykılıının Execute Safhası

ADD Komutunun İvedi, Direkt ve Dolaylı Mod için Mikroişlem Adımları

Komut Saykılı

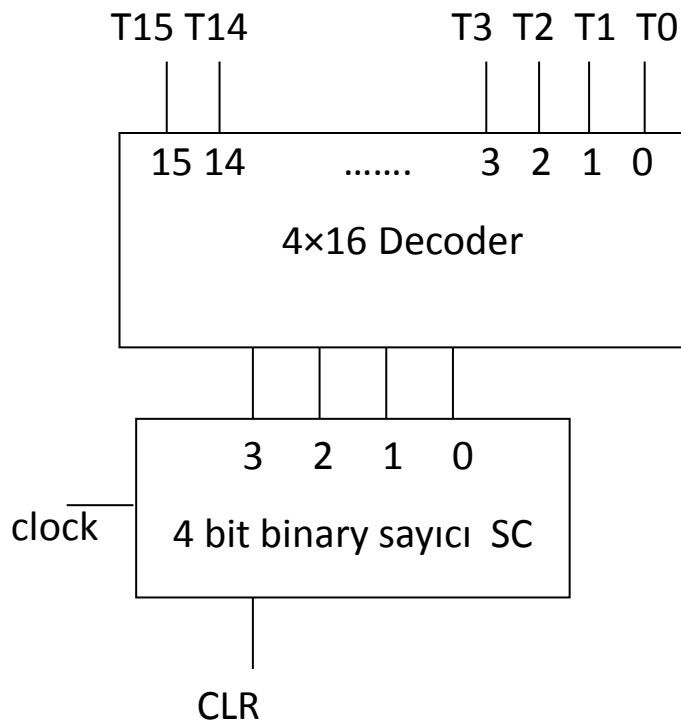
Bilgisayarın bellekte bulunan komutları sırayla işlemesi beklenir. Bir komutun yerine getirmesi gereken işlemlerin bütününe komut saykılı adı verilir. Komut saykılı 3 aşamadan oluşur bunlar; fetch (al-getir), decode (çöz) ve execute (icra et) safhalarıdır. Komutun opcode kısmının bellekten alınıp komut kaydedicisine (IR) koyulması birlikte, adresleme modu ile ilgili kısmı adresleme modu kod çözücüsüne, işlem tipi ile ilgili kısmı da komut kod çözücüsüne giriş olarak uygulanır. Daha sonra bu kod çözücülerden alınan sinyaller, ilgili komut için gerekli alt işlemlerin (mikroişlem) başlatılmasını sağlayan düzeneği aktif hale getirir.



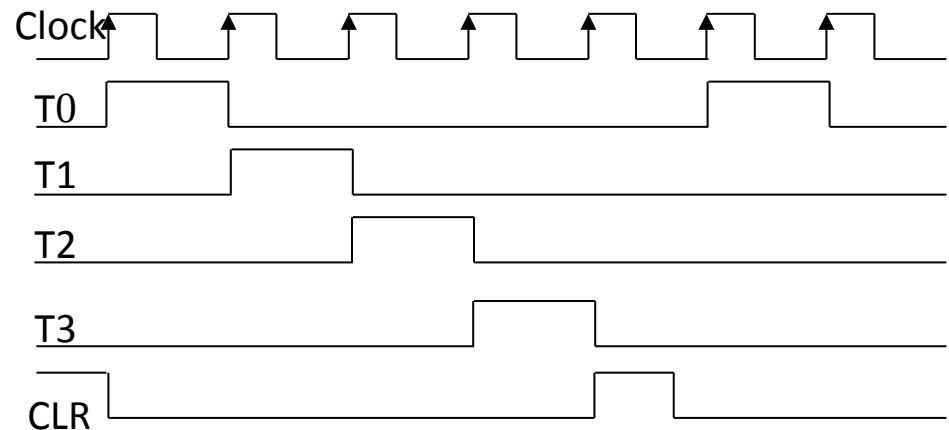
- Komut saykılı -

Zamanlama Düzeneđi

Bir bilgisayar sisteminden, bellekteki komutları ve her bir komutun da sahip olduđu mikroişlemleri sıra ile işletmesi beklendiğinden dolayı donanımsal bir düzeneğre ihtiyaç vardır.



- Zamanlama düzeneđi -



- Zamanlama Sinyalleri -

Komut Saykılıının Fetch ve Decode Safhaları

Komuta ait alt işlemler, komutun tipine bağlı olarak farklı sayıda adımlardan oluşur. Ama tüm komutların fetch ve decode safhaları aynı yapıda olup 3 adımdan oluşmaktadır. Bu adımların zamanlaması bir sayıcı vasıtasıyla yerine getirilir. Fetch safhası iki adımda yerine getirilir (T0 ve T1). Decode safhası tek adım olup, T2 zaman diliminde yerine getirilir. Execute safhası ise T3 zaman diliminden başlayıp, komutun tipine bağlı olarak çeşitli sayıda adımdan oluşur. Ancak, doğal adresleme moduna sahip komutlar, 1 byte'lık komutlar olduğundan T2 zaman dilimindeki işlemler yapılmayacaktır.

T0 : $AR \leftarrow PC$

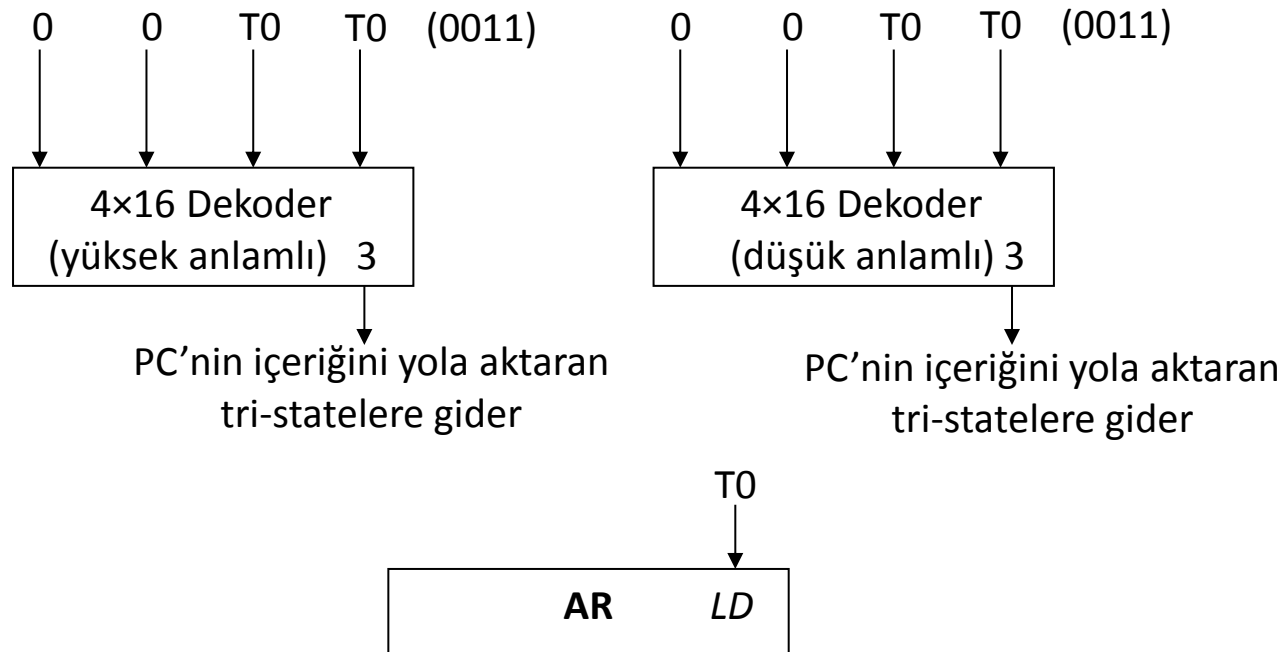
T1 : $IR \leftarrow M[AR], PC \leftarrow PC+1$

T2. **ADRMDO**: $AR \leftarrow PC, PC \leftarrow PC+1$, kod çözme aşaması

T0 Zaman Dilimi

T0 zaman diliminde PC aracılığıyla bir sonraki komutun adresi, adres kaydedicisine yüklenir. Bu işlem için, adres kaydedicisinin *LD* girişi aktif edilir, PC'nin içeriği ise ortak yola aktarılır.

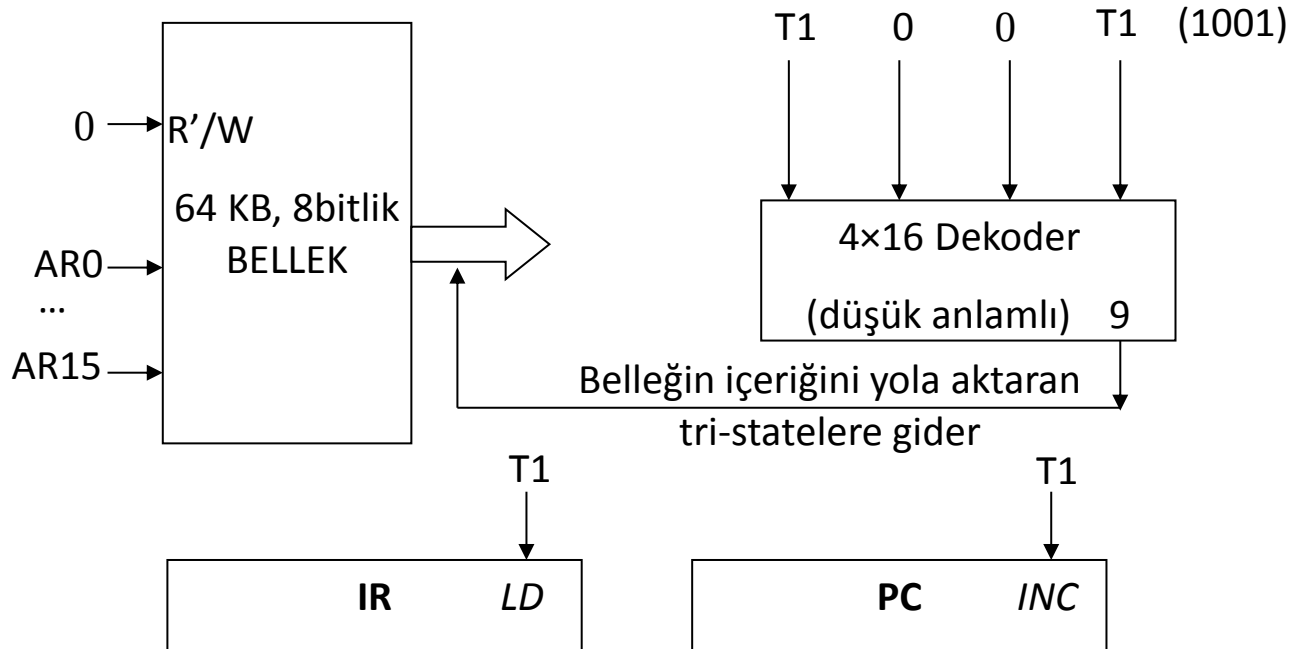
T0: AR $\leftarrow$ PC



T1 Zaman Dilimi

T1 zaman diliminde, adres kaydedicisi ile belirtilen bellek bölgesindeki komutun opcode'u ortak yola aktarılır. Komut kaydedicisinin *LD* girişi aktif edilerek opcode, komut kaydedicisine alınır. Program sayacının içeriği 1 arttırılır (Doğal adresleme modu kullanılıyorsa bir sonraki komutu veya diğer adresleme modlarında operandın adres bilgisini göstermek için).

T1: $IR \leftarrow M[AR], PC \leftarrow PC+1$

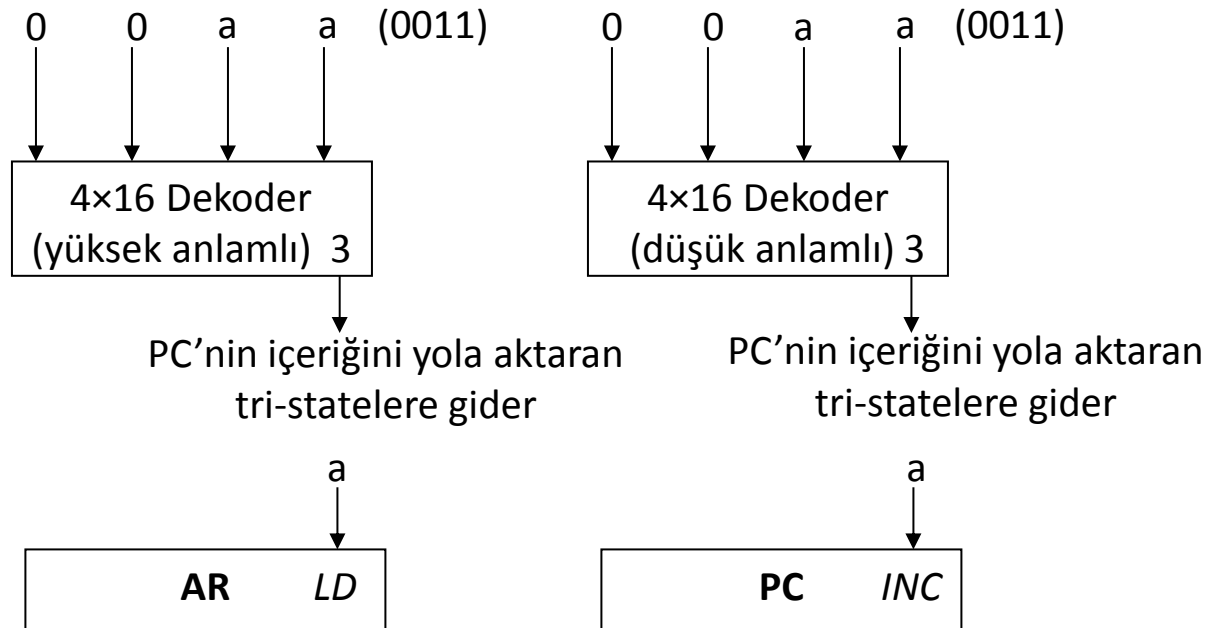


T2 Zaman Dilimi

T2 zaman diliminde, PC'nin içeriği AR'ye aktarılır. PC ise yine 1 arttırılır; şayet 2 byte lık bir komut ise bir sonraki komutu göstermek için, ya da 3 byte lık bir komutsa, komutun adres kısmındaki düşük anlamlı adres bilgisini göstermek için.

T2. ADRMDO: $AR \leftarrow PC, PC \leftarrow PC+1$, kod çözme aşaması

a



Komut Saykılının Execute Safhası

Daha önce de bahsedildiği gibi, execute safhasının kaç adım süreceği komutun tipine ve adresleme moduna bağlı olarak değişiklik göstermektedir.

Örnek: İvedi adresleme moduna sahip ADD komutunun mikroişlem adımları.

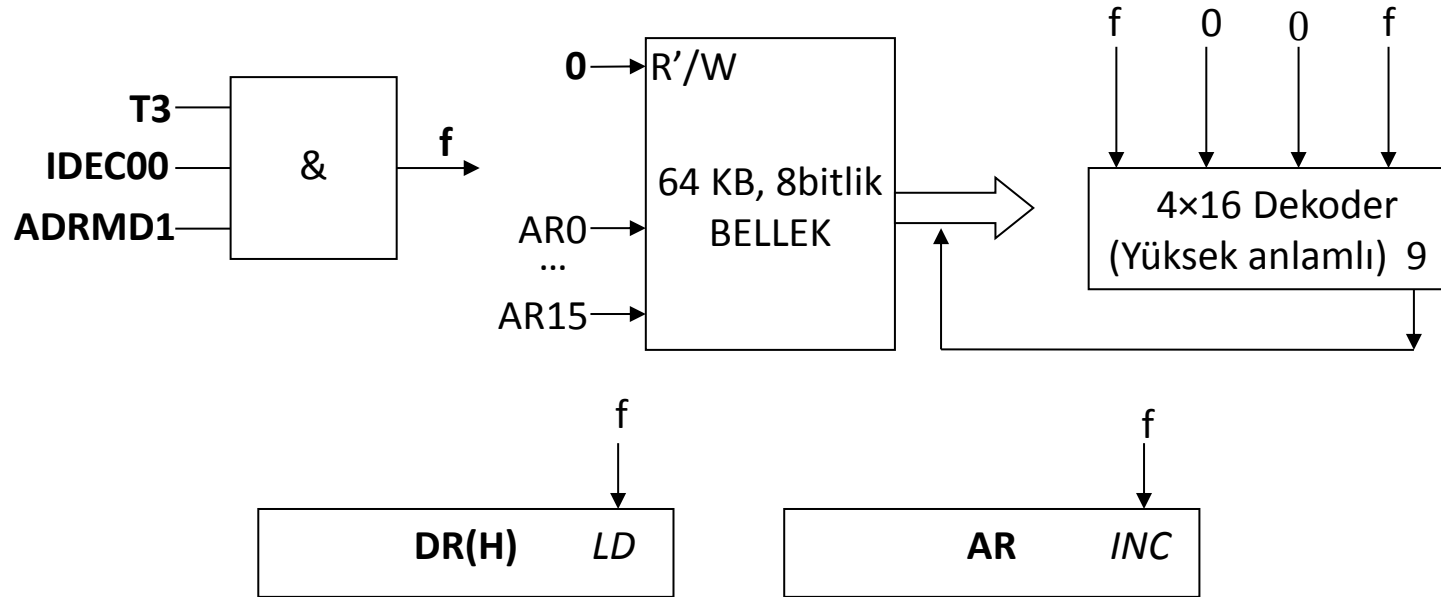
ADD #1234h komutu gibi. Burada # işareti ivedi modu göstermektedir. 3 byte'lık bir komuttur. ADD komutunun opcode'u $0\ 001\ 0000_2 = 10h$

	10h
T2:AR →	12h
T2:PC →	34h

İvedi adresleme modunda akü, sabit bir sayı (1234h) ile işleme giriyordu. Akü ile bellekteki sayı toplanacak ve sonuç aküye aktarılacaktır. Şu anda AR kaydedicisi bellekteki sayımızın yüksek anlamlı kısmını göstermektedir. Bu değeri, DR kaydedicimizin yüksek anlamlı kısmına koymamız gerekecektir. Daha sonra da sayımızın düşük anlamlı kısmına erişebilmemiz için AR kaydedicisini 1 arttırmamız gerekecektir.

İvedi Mod için T3 Adımı

T3. IDEC00.ADRMD1: $DR_H \leftarrow M[AR]$, $AR \leftarrow AR+1$



T3:PC,AR →

10h
12h
34h

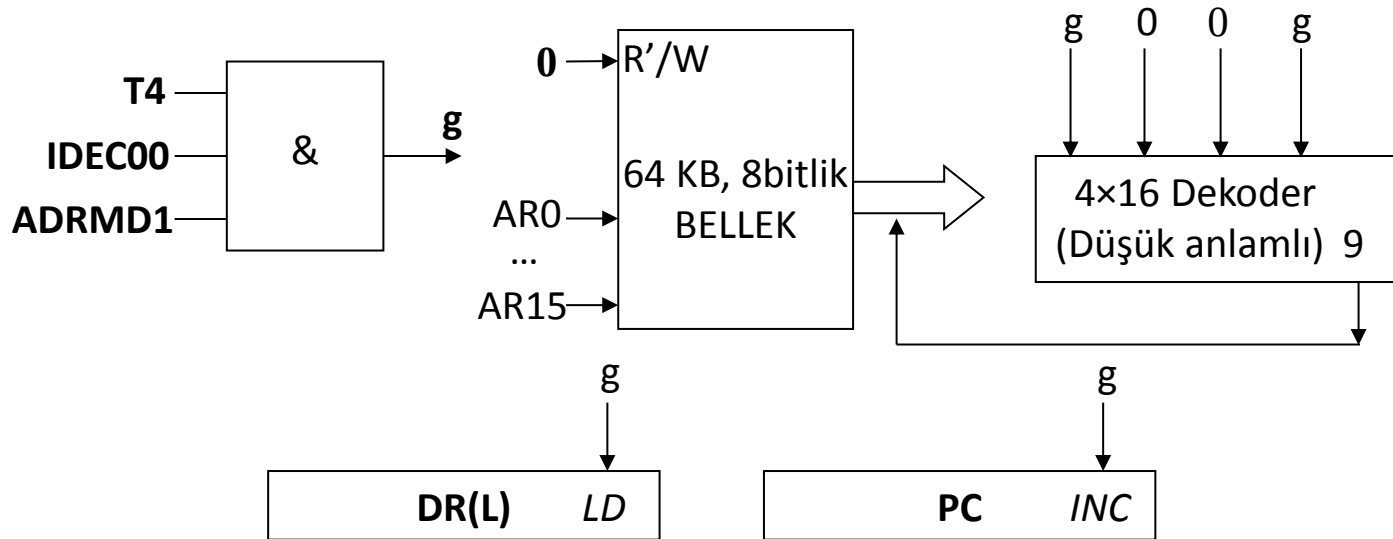
T3 adımımda $AR \leftarrow AR+1$ işlemi yerine $AR \leftarrow PC$ kullanılması düşünülebilirdi ancak, veri yolunu aynı anda iki cihaz kullanamaz.

İvedi Mod için T4 Adımı

T4. IDEC00.ADRMD1: $DR_L \leftarrow M[AR], PC \leftarrow PC+1$

	10h
	12h
T3:AR →	34h
T4:PC →	

İvedi moda sahip ADD komutunun önceki adımlarında PC, 2 kere arttırılmıştır. Bu komut 3 byte'lık olduğundan, 1 kere daha arttırılarak PC'nin bir sonraki komutu göstermesi sağlanmıştır

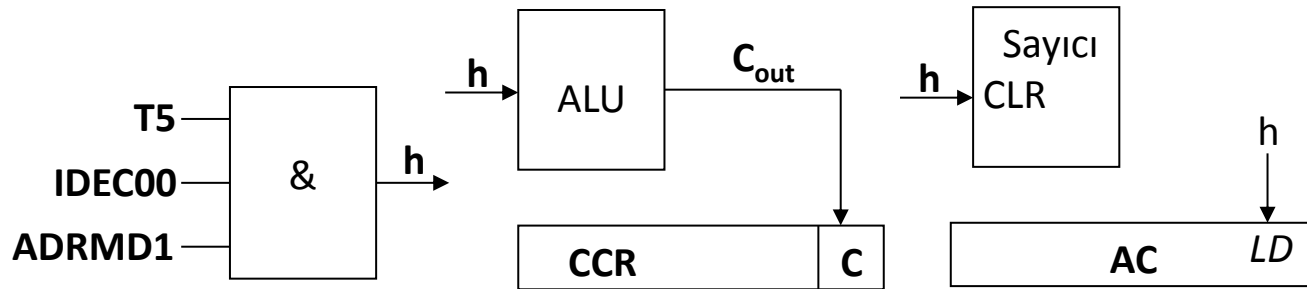


İvedi Mod için T5 Adımı

T5 adımı da ALU'da gerekli işlem yapılacak ve sayıcı sıfırlanacaktır:

T5. IDEC00.ADRMD1: $AC \leftarrow AC + DR$, $C \leftarrow C_{out}$, $SC \leftarrow 0$

T5 zamanlama diliminde ALU'da gerekli toplama işlemi yapılmış, işlemin sonucu AC'nin *LD* girişi aktive edilerek AC'ye aktarılmıştır. Toplama işlemi sonucunda elde oluşabileceğinden elde bayrağı güncellenmiştir. Ayrıca sayıcı da, bir sonraki komutun fetch işlemi için sıfırlanmıştır.



Örnek: Direkt adresleme moduna sahip ADD komutunun mikroişlem adımları.

ADD 1000h komutu gibi. Burada adres kısmının önünde bir işaret kullanılmamıştır, direkt adresleme modunu göstermektedir. 3 byte'lık bir komuttur.

Direkt adresleme modunu kullanan ADD komutunun opcode'u $0\ 010\ 0000_2 = 20h$. 1000h adresindeki değerle akü toplanacak ve sonuç tekrar aküye aktarılacaktır.

Diğer komutlarda olduğu gibi bu komutun da T0, T1 ve T2 adımları aynıdır. Execute saykılı ise aşağıdaki gibidir.

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3.IDEC00.ADRMD2:	$TR_H \leftarrow M[AR], AR \leftarrow AR+1$
T4.IDEC00.ADRMD2:	$TR_L \leftarrow M[AR], PC \leftarrow PC+1$
T5.IDEC00.ADRMD2:	$AR \leftarrow TR$
T6.IDEC00.ADRMD2:	$DR_H \leftarrow M[AR], AR \leftarrow AR+1$
T7.IDEC00.ADRMD2:	$DR_L \leftarrow M[AR]$
T8.IDEC00.ADRMD2:	$AC \leftarrow AC+DR, C \leftarrow C_{out}, SC \leftarrow 0$

Direkt Mod için T3 ve T4 Adımları

T2 saykılının sonunda belleğin ve kaydedicilerin gösterdiği değerler:

	20h
T2:AR →	10h
T2:PC →	00h
	...
1000h	12h
1001h	34h

T3. IDEC00.ADRMD2: $TR_H \leftarrow M[AR], AR \leftarrow AR+1$

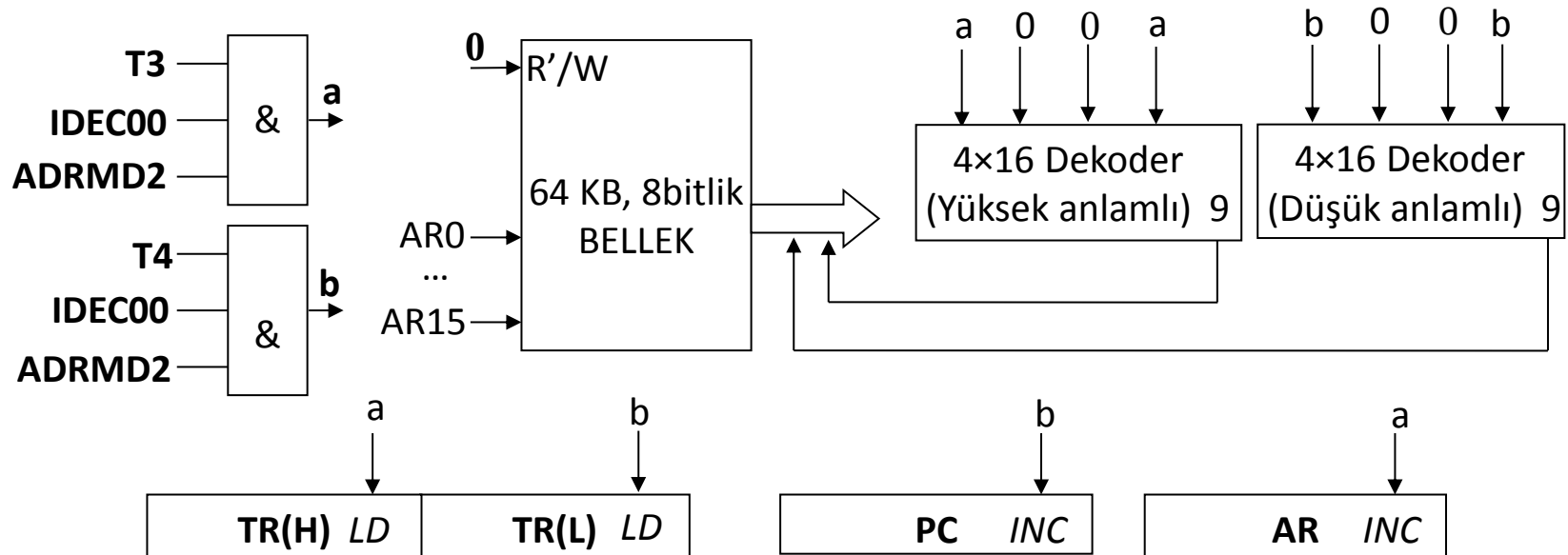
T4. IDEC00.ADRMD2: $TR_L \leftarrow M[AR], PC \leftarrow PC+1$

	20h
T2:AR →	10h
T3:PC,AR →	00h
T4:PC →	...
1000h	12h
1001h	34h

Direkt Mod için T3 ve T4 Adımları

T3. IDEC00.ADRMD2: $TR_H \leftarrow M[AR]$, $AR \leftarrow AR+1$

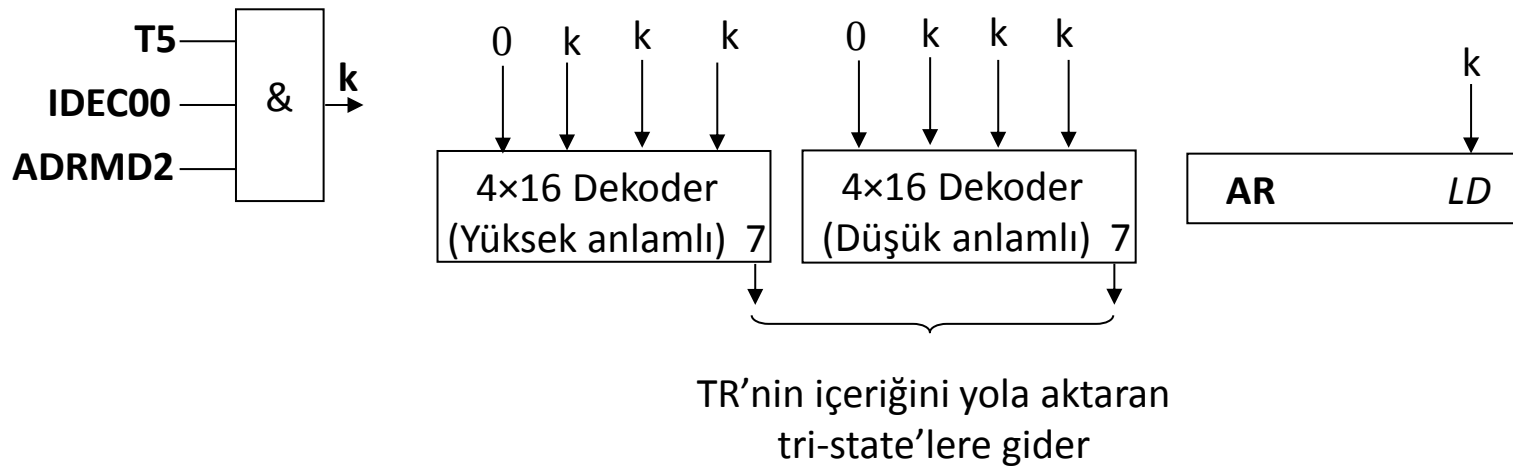
T4. IDEC00.ADRMD2: $TR_L \leftarrow M[AR]$, $PC \leftarrow PC+1$



Direkt Mod için T5 Adımı

T5. IDEC00.ADRMD2: $AR \leftarrow TR$

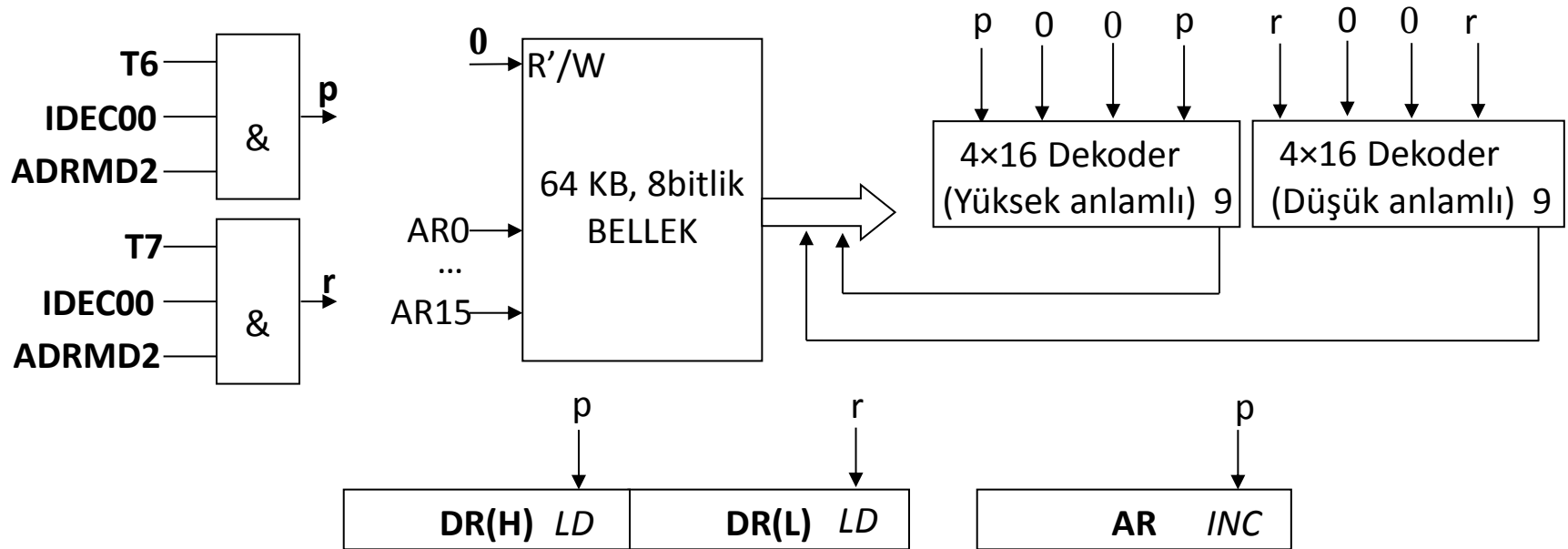
(TR'nin gösterdiği yerdeki veriye erişmek için aktarım yapılmıştır)



Direkt Mod için T6 ve T7 Adımları

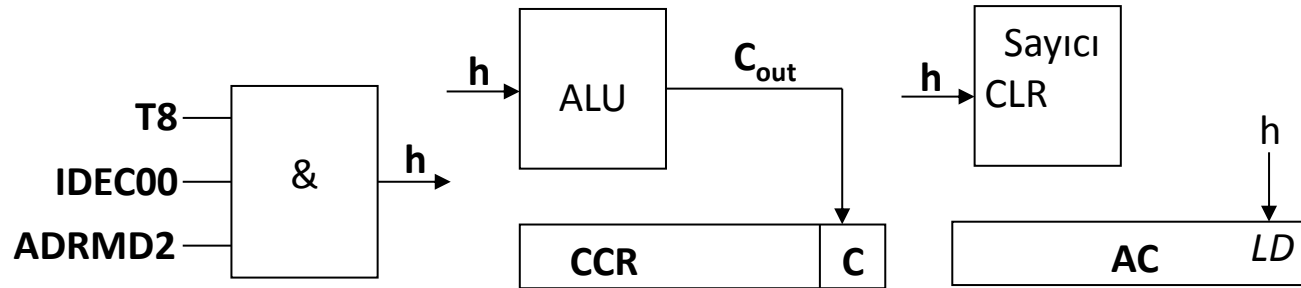
T6. IDEC00.ADRMD2 : $DR_H \leftarrow M[AR], AR \leftarrow AR+1$

T7. IDEC00.ADRMD2 : $DR_L \leftarrow M[AR]$



Direkt Mod için T8 Adımı

T8. IDEC00.ADRMD2: $AC \leftarrow AC + DR$, $C \leftarrow C_{out}$, $SC \leftarrow 0$



Örnek: Dolaylı adresleme moduna sahip ADD komutunun mikroişlem adımları

ADD (1000h) komutu gibi. Burada adres kısmında parantezler kullanılmıştır, dolaylı adresleme modunu göstermektedir. 3 byte'lık bir komuttur.

Dolaylı adresleme modunu kullanan ADD komutunun opcode'u $0\ 011\ 0000_2 = 30h$. 1000h adresinde, operandın adresi bulunmaktadır. Belirtilen adresteki değerle (0123h) akü toplanacak ve sonuç tekrar aküye aktarılacaktır.

	30h	
	10h	
	00h	
	...	
1000h	12h	} Operand'ın adresi
1001h	34h	
	...	
1234h	01h	} Operand'ın kendisi
1235h	23h	

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3. IDEC00.ADRMD3:	$TR_H \leftarrow M[AR], AR \leftarrow AR+1$
T4. IDEC00.ADRMD3:	$TR_L \leftarrow M[AR], PC \leftarrow PC+1$
T5. IDEC00.ADRMD3:	$AR \leftarrow TR$
T6. IDEC00.ADRMD3:	$TR_H \leftarrow M[AR], AR \leftarrow AR+1$
T7. IDEC00.ADRMD3:	$TR_L \leftarrow M[AR]$
T8. IDEC00.ADRMD3:	$AR \leftarrow TR$
T9. IDEC00.ADRMD3:	$DR_H \leftarrow M[AR], AR \leftarrow AR+1$
T10. IDEC00.ADRMD3:	$DR_L \leftarrow M[AR]$
T11. IDEC00.ADRMD3:	$AC \leftarrow AC+DR, C \leftarrow C_{out}, SC \leftarrow 0$

BÖLÜM 5. KOMUT SETİ MİMARİSİ

❖ **Göreceli ve İndis Adresleme Modları için Adres Hesaplama Birimi**

❖ **Yığın (Stack) ve Yığın Göstercisi (Stack Pointer-SP)**

Yığınla İlgili Komutlar

Göreceli ve İndis Adresleme Modları için

Adres Hesaplama Birimi

Göreceli ve indis adresleme modlarına sahip komutlardan sonra gelen bir byte'lık ofset değerinin, program sayıcısıyla (PC) veya indeks kaydedicisiyle (IX) toplanarak etkin adresin hesaplanıldığına değinilmişti.

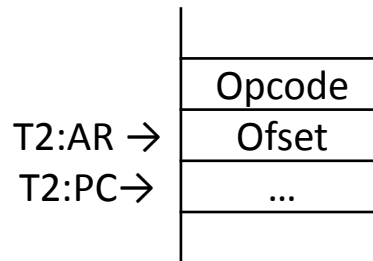
Bu toplama işlemini gerçekleştirmek için elimizde olan aritmetik mantık birimini (ALU) kullanabiliriz. Ancak, bu toplama işleminden önce, akümülatördeki ve data kaydedicisindeki verileri kaybetmememiz için yığına atmamız ve etkin adresi hesapladıktan sonra da tekrar yığından bu bilgileri geri yüklememiz gerekecektir.

Bu işlemler, bu adresleme modlarına sahip komutların icra edilme sürelerini uzatacaktır. Etkin adres hesabının, ALU yerine ayrı bir birimde yapılması, bu süre kaybını önleyecektir. Yol seçiminde kullanılan kod çözücülerin girişlerine 1010_2 verilerek, bu birimin hesapladığı etkin adres bilgisinin yola aktarılması sağlanır.

Göreceli ve İndis Adresleme Modları için Adres Hesaplama Birimi

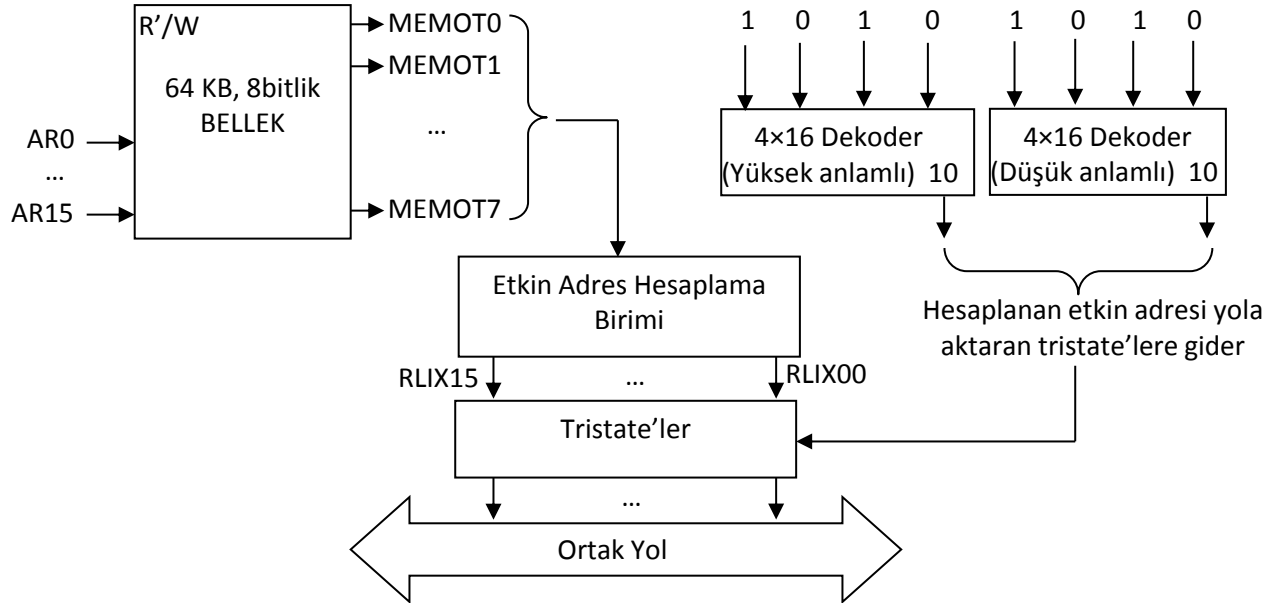
PC ve IX 16 bitlik kaydedicilerdir, bellek ise 8 bitlidir. Etkin adresin hesaplanabilmesi için bu kaydedicilerin komutun ofset kısmıyla toplanması gerekmektedir. İndis adresleme modunu kullanan komutların ofset kısmı işaretli sayı olarak, göreceli moda sahip komutların ofset kısmı ise işaretli sayı (2'ye tümleyen formunda) olarak düşünülmüştür.

Bu adresleme modlarına sahip komutlar işletilirken, T2 zaman diliminde AR, bellekteki ofset değerine işaret ettiğinden ve bellek de AR ile direkt bağlantılı olduğundan dolayı belleğin çıkışındaki ofset değeri, etkin adres hesaplama birimine iletilecektir.



Göreceli ve İndis Adresleme Modları için Adres Hesaplama Birimi

Belleğin çıkışındaki 1 byte'lık ofset değeri MEMOT0 ... MEMOT7 ile isimlendirilmiştir.



Aslında göreceli ve indis adresleme modlarına sahip komutlar olmasa bile, her zaman etkin adres hesaplama birimine bir bilgi gitmektedir. Örneğin direkt adresleme moduna sahip bir komutun T2 zaman diliminde, operandın adres bilgisinin yüksek anlamlı kısmı bu birime iletilir. Her zaman bir etkin adres hesaplanılmasına rağmen, sadece indis ve göreceli adresleme modlarında, etkin adres bilgisi yola aktarılmaktadır.

Etkin Adresin Hesabı

Etkin adres hesaplama biriminde 16 bitlik toplama işlemleri yapılacağından dolayı 8 bitlik ofset değerinin 16 bit olarak ifade edilmesi gerekmektedir.

İndis adresleme modunda, ofset değeri işaretsiz sayı olduğundan dolayı, ofseti 16 bite genişletmek için 8 adet 0 ilave edilir.

Göreceli adresleme modunda ise ofset değeri işaretli sayı olduğundan, işaret bitine bağlı olarak, ofset değeri genişletilir. Yani bellekten okunan ofset değerinin en yüksek anlamlı kısmı (MEMOT7) ile genişletme yapılır. Bu işlemler neticesinde ofset değerinin asıl değerinde hiç bir değişme meydana gelmez.

Etkin Adresin Hesabı

Örnek: Ofset değeri 85h olsun. Etkin adresi her iki adresleme modu için hesaplayalım.

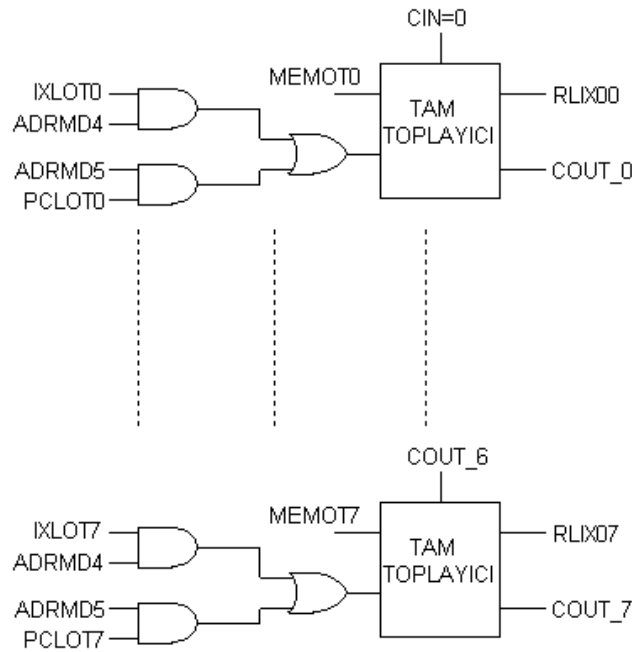
Göreceli adresleme moduna göre bu sayı negatif bir sayıdır. Değeri;
 $85h = 1000\ 0101_2 \rightarrow 2$ 'ye tümleyenini alıp önüne '-' işaretini koyarsak
 $-(0111\ 1011_2) = -123$ 'tür.

PC'nin 1234h olduğunu düşünelim. Bu durumda etkin adres şu şekilde hesaplanır; ofset değerinin işaret biti 1 olduğundan, ofset değerinin soluna 8 adet 1 konulur. Bu durumda ofset değerinin 16 bitlik hali, FF85h olur. Bunun da decimal karşılığı -123'e tekabül etmektedir.

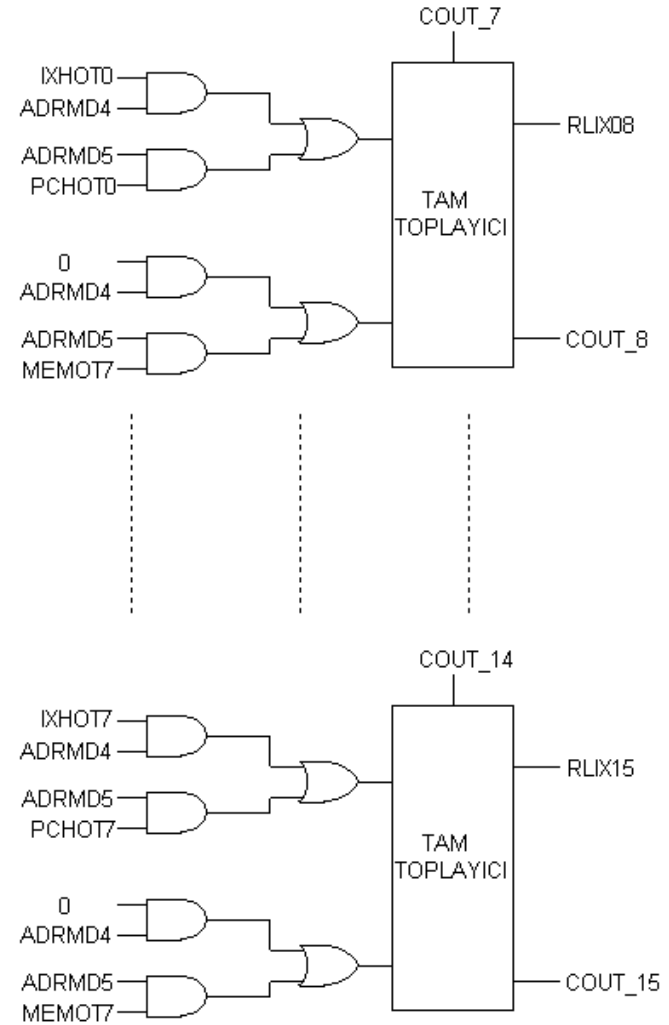
Etkin adres = $1234h + FF85h = 11B9h$ olarak bulunur.

İndis adresleme moduna göre bu sayı işaretsiz düşünülmüş olup decimal 133 değerine karşılık gelmektedir. Etkin adres hesaplanmadan önce ofset değerinin soluna 8 adet 0 konulur. Bu durumda ofset değerinin 16 bitlik hali 0085h olur. IX'in değerini 1234h alırsak, Etkin adres = $1234h + 0085h = 12B9h$ olur.

Etkin Adres Hesaplama Biriminin Donanımsal Yapısı



Düşük anlamlı kısmı



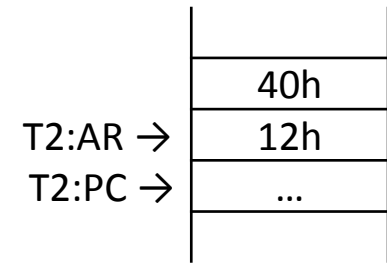
Yüksek anlamlı kısmı

Örnek: İndis moda sahip ADD komutunun işlem adımları

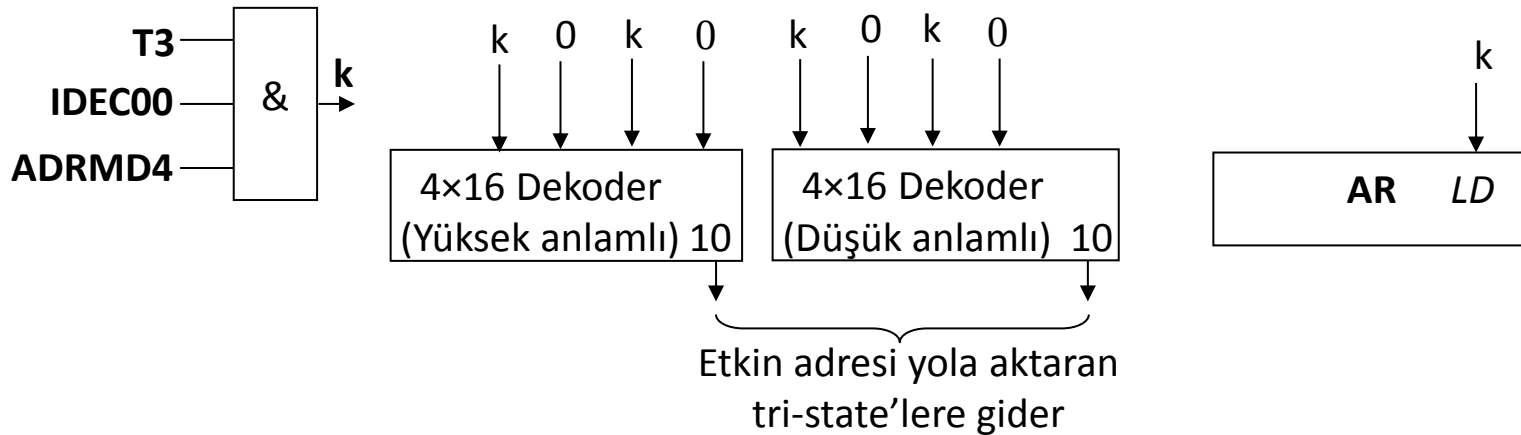
ADD *12h komutu gibi. ‘*’ sembolü indis adreslemeyi temsil etmek için kullanılmıştır. 2 byte’lık bir komuttur. Komuttaki 12h, ofset değeridir. Index kaydedicisinin içeriğine 12h eklenecek ve etkin adres hesaplanacaktır.

İndis mod için ADD komutunun opcode’u $0\ 100\ 0000_2 = 40h$.

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3. IDEC00.ADRMD4:	$AR \leftarrow \text{Etkin Adres}$
T4. IDEC00.ADRMD4:	$DR_H \leftarrow M[AR], AR \leftarrow AR+1$
T5. IDEC00.ADRMD4:	$DR_L \leftarrow M[AR]$
T6. IDEC00.ADRMD4:	$AC \leftarrow AC+DR, C \leftarrow C_{out}, SC \leftarrow 0$



T3 zamanlama diliminde yapılan mikroişlem adımının donanımsal gerçekleştirimi



Yığın (Stack) ve Yığın Göstericisi (Stack Pointer-SP)

Herhangi bir dallanma olduğunda PC'nin değeri, alt programdan dönüşte kullanılmak üzere yığına atılmalıdır. Kesme geldiğinde ise PC, IX, AC ve CCR kaydedicilerinin içerikleri yığına kaydedilmeli ve kesme programı işletildikten sonra da bu değerler yığından alınarak ilgili kaydedicilere geri yüklenmelidir

Bu işlem için, hafızada ilk olarak bir yığın bölgesi belirlenir. SP ise verinin koyulacağı adresi gösterir. Veri, yığına eklendikçe SP'nin değeri 1 azaltılır. Benzer şekilde yığından veri alındıkça da 1 arttırılır. Yani temel bilgisayar sistemimizde kullanılan yığın türü LIFO'dur (Last-In First-Out).

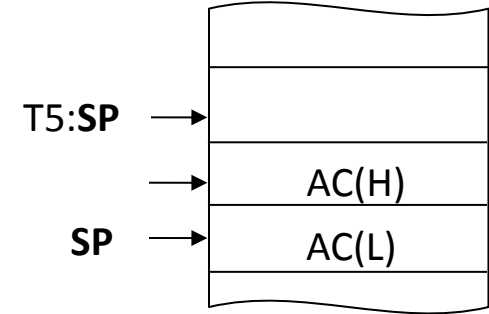
Yüksek seviyeli programların derleyicileri, yerel ve global değişkenleri tutmak için yığını kullanabilirler. Temel bilgisayar sistemimizde bu işleri yerine getirebilmek için PSH (yığına koyma) ve PUL (yığından alma) komutları vardır. Bir de matematiksel işlemlerde, işlem önceliğinin temini için kullanılmaktadır.

Özetle yığın, özellikle çok sayıda alt programa dallanma olanağı sunar.

Yığınla İlgili Komutlar

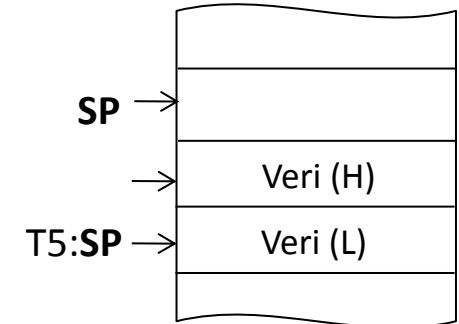
PSH (Push): Akümülatördeki veriyi, yığın göstericisinin gösterdiği yere kaydeder. Doğal adresleme moduna sahiptir. Akünün önce düşük daha sonra da yüksek anlamlı kısmı yığına konulur.

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3. IDEC06.ADRMD0:	$AR \leftarrow SP$
T4. IDEC06.ADRMD0:	$M[AR] \leftarrow AC_L, SP \leftarrow SP-1, AR \leftarrow AR-1$
T5. IDEC06.ADRMD0:	$M[AR] \leftarrow AC_H, SP \leftarrow SP-1, SC \leftarrow 0$



PUL (Pop): Yığın göstericisinin gösterdiği yerdeki veriyi akümülatöre kopyalar. Doğal adresleme moduna sahiptir. Yığındaki değer, aküye direkt olarak yüklenemeyeceğinden dolayı önce DR'ye, daha sonra da aküye yüklenir.

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3. IDEC07.ADRMD0:	$SP \leftarrow SP+1$
T4. IDEC07.ADRMD0:	$AR \leftarrow SP$
T5. IDEC07.ADRMD0:	$DR_H \leftarrow M[AR], SP \leftarrow SP+1, AR \leftarrow AR+1$
T6. IDEC07.ADRMD0:	$DR_L \leftarrow M[AR]$
T7. IDEC07.ADRMD0:	$AC \leftarrow DR, SC \leftarrow 0$



Yığınla İlgili Komutlar

BSR (Branch to SubRoutine): Şartsız olarak hesaplanan etkin adresteki alt programa, kısa dallanma sağlar. Göreceli adresleme modunu kullanır. PC'nin içeriğinin yığına atılmasını gerektirir.

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3. IDEC15.ADRMD5:	$TR \leftarrow AR$ (ofsetin adresi kaydedildi)
T4. IDEC15.ADRMD5:	$AR \leftarrow SP$
T5. IDEC15.ADRMD5:	$M[AR] \leftarrow PC_L$, $AR \leftarrow AR-1$, $SP \leftarrow SP-1$
T6. IDEC15.ADRMD5:	$M[AR] \leftarrow PC_H$, $SP \leftarrow SP-1$
T7. IDEC15.ADRMD5:	$AR \leftarrow TR$ (ofsetin adresi geri yüklendi)
T8. IDEC15.ADRMD5:	$PC \leftarrow$ Etkin adres, $SC \leftarrow 0$

T3 ve T7 adımlarındaki mikroişlemlerin sebebi; etkin adres hesaplama ünitesinin ofset değerine ihtiyaç duymasıdır. AR, direkt olarak belleğe bağlantılı olduğundan ofset değeri, T7 saykılıyla birlikte belleğin çıkışında hazır olacaktır ve bu değer etkin adres hesaplama ünitesine gidecektir.

Yığınla İlgili Komutlar

JSR (Jump to SubRoutine): Şartsız olarak, alt programa uzun dallanma sağlar. Direkt ve indis adresleme modlarını kullanır.

Direkt adresleme modu için mikroişlem adımları;

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3. IDEC24.ADRMD2:	$TR \leftarrow AR, PC \leftarrow PC+1$ (adres bilgisi kaydedildi)
T4. IDEC24.ADRMD2:	$AR \leftarrow SP$
T5. IDEC24.ADRMD2:	$M[AR] \leftarrow PC_L, AR \leftarrow AR-1, SP \leftarrow SP-1$
T6. IDEC24.ADRMD2:	$M[AR] \leftarrow PC_H, SP \leftarrow SP-1$
T7. IDEC24.ADRMD2:	$AR \leftarrow TR$ (adres bilgisinin başlangıcı geri yüklendi)
T8. IDEC24.ADRMD2:	$TR_H \leftarrow M[AR], AR \leftarrow AR+1$
T9. IDEC24.ADRMD2:	$TR_L \leftarrow M[AR]$
T10. IDEC24.ADRMD2:	$PC \leftarrow TR, SC \leftarrow 0$

İndis adresleme modunu kullanan JSR ile göreceli adresleme modunu kullanan BSR komutlarının mikro işlem adımları aynıdır. Farklılık, etkin adreslerin hesabındadır.

Yığınla İlgili Komutlar

RTS (ReTurn to Subroutine): Alt programdan, program akışının kaldığı yere döner. RTS komutu, PUL komutuna benzemektedir. Aralarındaki fark, yığından okunan bilginin AC yerine PC'ye konulmasıdır.

KOMUTUN MİKRO İŞLEM ADIMLARI	
T3. IDEC26.ADRMD0:	$SP \leftarrow SP+1$
T4. IDEC26.ADRMD0:	$AR \leftarrow SP$
T5. IDEC26.ADRMD0:	$PC_H \leftarrow M[AR], AR \leftarrow AR+1, SP \leftarrow SP+1$
T6. IDEC26.ADRMD0:	$PC_L \leftarrow M[AR], SC \leftarrow 0$

BÖLÜM 5. KOMUT SETİ MİMARİSİ

❖ Aritmetik Mantık Ünitesi (ALU)

Aritmetik İşlemler Kısmı

Mantıksal İşlemler Kısmı

Çarpma Kısmı

Bölme Kısmı

❖ Temel Bilgisayar Sistemimiz için Örnek Bir Uygulama

Aritmetik Mantık Ünitesi (ALU)

Bir mikroişlemcide bulunan ALU birimi, bütün aritmetik ve mantıksal işlemlerin yapıldığı yerdir. Temel bilgisayar sistemimizde tasarlanan ALU, çarpma ve giriş/çıkış işlemleri haricinde, 16 bitlik veriler üzerinde işlem yapabilecek şekilde tasarlanmıştır. Kullanılan kaydediciler 16 bitlik olduğundan, çarpma işleminde iki tane 8 bitlik sayı kullanılmıştır.

Tasarlanan ALU dört bölümden oluşmaktadır;

1. Çarpma ve bölme işlemleri hariç, diğer aritmetik işlemlerin yapıldığı bölüm
2. Mantıksal işlemlerin yapıldığı bölüm
3. Çarpma işleminin yapıldığı bölüm
4. Bölme işleminin yapıldığı bölüm

Aritmetik İşlemler Kısımı

Bu kısımda 16 adet tam toplayıcı kullanılmıştır. Tam toplayıcının tasarım adımları aşağıda gösterilmiştir.

Tam Toplayıcı

Tam toplayıcı, 2 adet 1 bitlik veriyi ve elde bitini (c_i) giriş olarak alır, toplam ve elde çıkışı (c_o) üretir.

Girişler			Çıkışlar	
AC	DR	c_i	Toplam	c_o
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

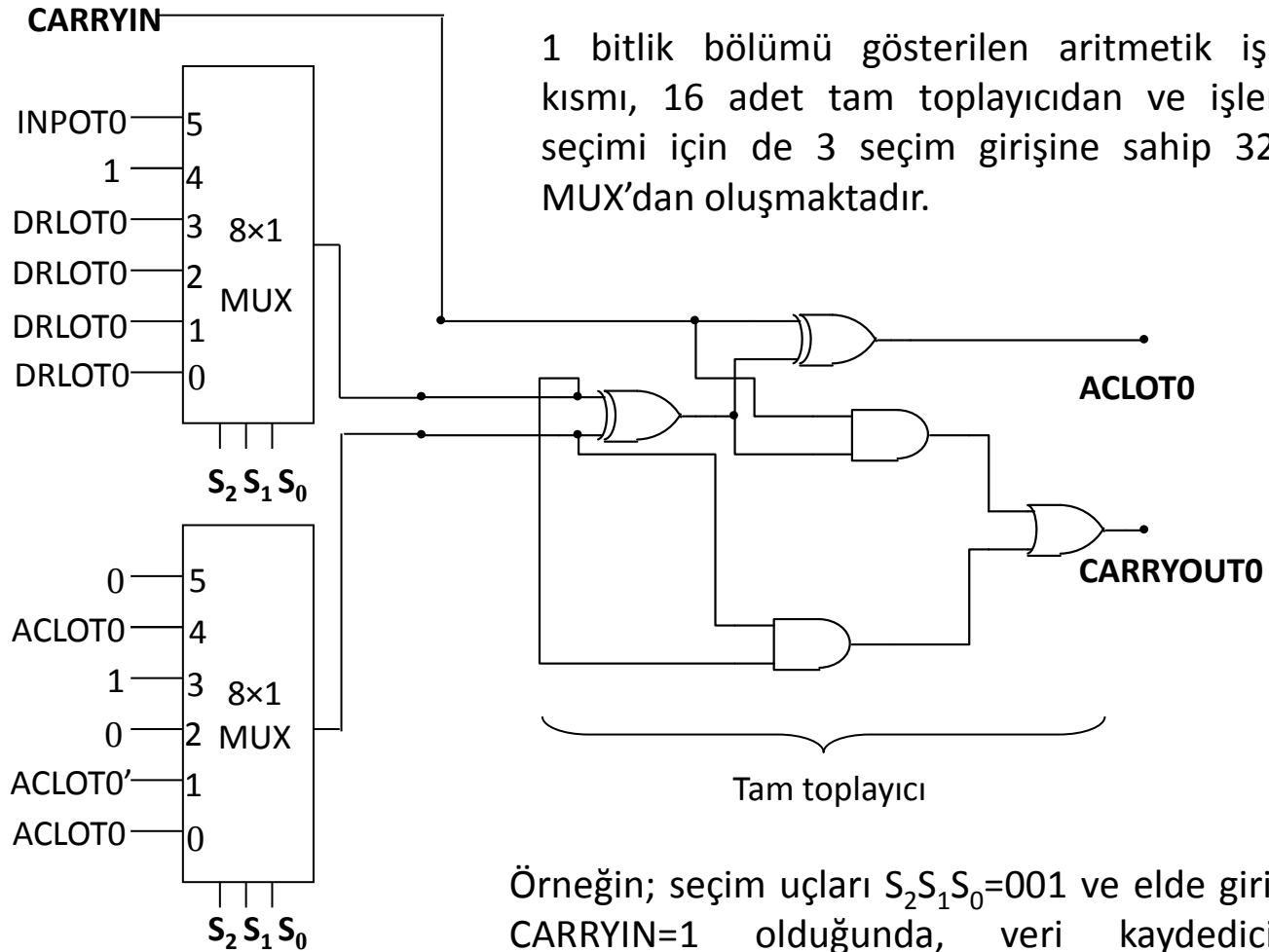
$$\begin{aligned}\text{Toplam} &= AC'.DR'.c_i + AC'.DR.c_i' + AC.DR'.c_i' + AC.DR.c_i \\ &= c_i.(AC'.DR' + AC.DR) + c_i'.(AC'.DR + AC.DR') \\ &= c_i.(AC \oplus DR)' + c_i'.(AC \oplus DR) \\ &= AC \oplus DR \oplus c_i\end{aligned}$$

$$\begin{aligned}c_o &= AC'.DR.c_i + AC.DR'.c_i + AC.DR.c_i' + AC.DR.c_i \\ &= c_i.(AC \oplus DR) + AC.DR\end{aligned}$$

ALU'da Gerçekleştirilen Aritmetik İşlemler

Seçim			Girişler			Çıkış	Açıklaması
S_2	S_1	S_0	CARRYIN	Y_0	Y_1	Q	
0	0	0	0	DR	AC	$AC \leftarrow DR + AC$	Toplama
0	0	0	1	DR	AC	$AC \leftarrow DR + AC + 1$	Elde ile toplama
0	0	1	0	DR	AC'	$AC \leftarrow DR + AC'$	Borç ile çıkarma
0	0	1	1	DR	AC'	$AC \leftarrow DR + AC' + 1$	Çıkarma
0	1	0	0	DR	0..0	$AC \leftarrow DR$	DR'in aktarımı
0	1	0	1	DR	0..0	$AC \leftarrow DR + 1$	DR'nin 1 fazlasını aktarma
0	1	1	0	DR	1..1	$AC \leftarrow DR - 1$	DR'nin 1 eksiğini aktarma
0	1	1	1	DR	1..1	$AC \leftarrow DR$	DR'nin aktarımı
1	0	0	0	1..1	AC	$AC \leftarrow AC - 1$	AC'yi 1 azaltma
1	0	0	1	1..1	AC	$AC \leftarrow AC$	AC'nin aktarımı
1	0	1	0	INPR	0..0	$AC \leftarrow INPR$	Giriş Kaydedicisinin aktarımı
1	0	1	1	INPR	0..0	$AC \leftarrow INPR + 1$	Giriş Kaydedicisinin 1 fazlasının aktarımı

Aritmetik İşlemler Kısımının 1 Bitlik Bölümünün Gerçekleştirimi



Örneğin; seçim uçları $S_2 S_1 S_0 = 001$ ve elde girişi de $CARRYIN = 1$ olduğunda, veri kaydedicisinin içeriğinden akümülatörün içeriği çıkartılır.

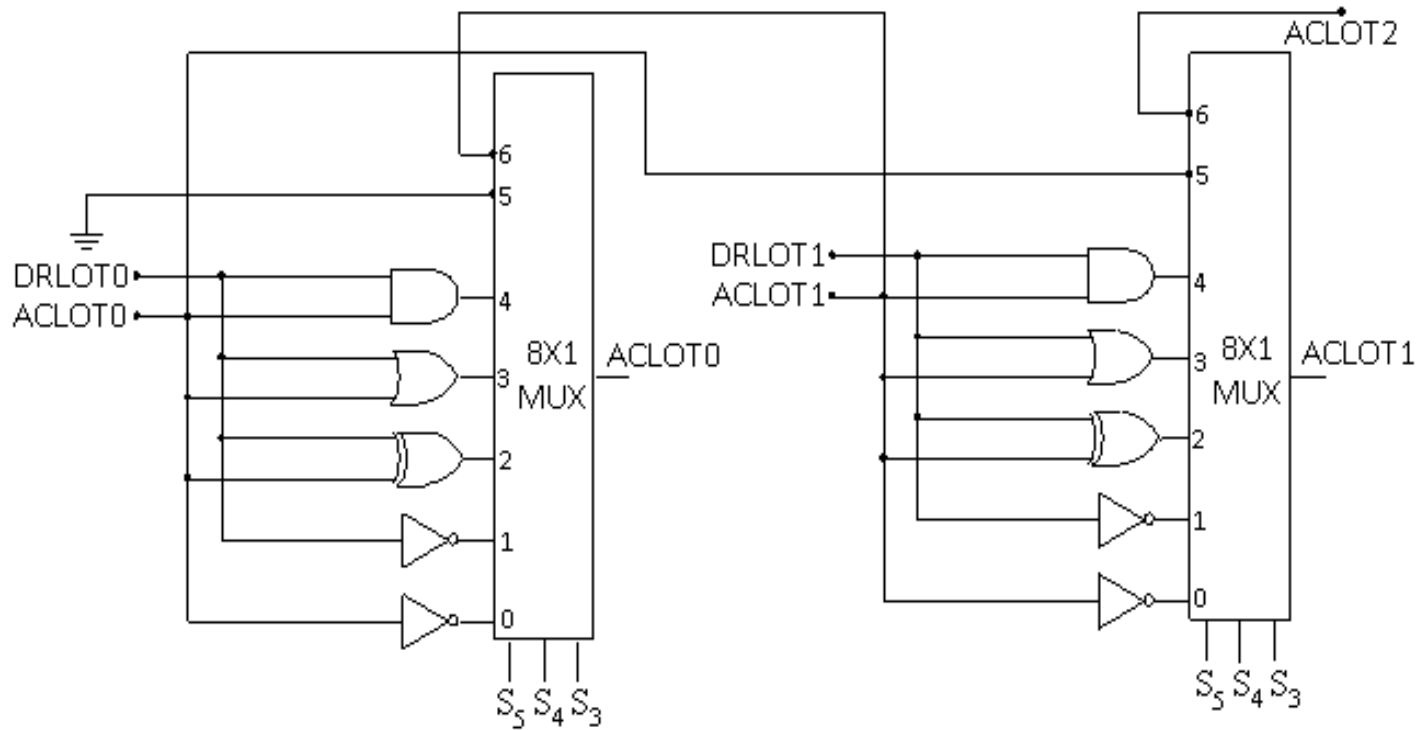
Mantıksal İşlemler Kısmı

Bu kısım, 7 farklı işlemi yerine getirebilmektedir. Aşağıdaki tabloda yerine getirilen işlemler verilmiştir;

S_5	S_4	S_3	Q	Açıklaması
0	0	0	$AC \leftarrow AC'$	AC'nin tümleyeni
0	0	1	$AC \leftarrow DR'$	DR'nin tümleyeni
0	1	0	$AC \leftarrow AC \oplus DR$	Lojik "XOR" işlemi
0	1	1	$AC \leftarrow AC \vee DR$	Lojik "VEYA" işlemi
1	0	0	$AC \leftarrow AC \text{ DR}$	Lojik "VE" işlemi
1	0	1	$AC \leftarrow \text{shl}(AC)$	AC'yi sola kaydırma
1	1	0	$AC \leftarrow \text{shr}(AC)$	AC'yi sağa kaydırma

Mantıksal İşlemler Kısımının 2 Bitlik Bölümünün Gerçekleştirimi

ALU'nun bu bölümünde yapılan işlemlerin yerine getirilebilmesi için 16 adet 8×1'lik MUX kullanılmıştır.

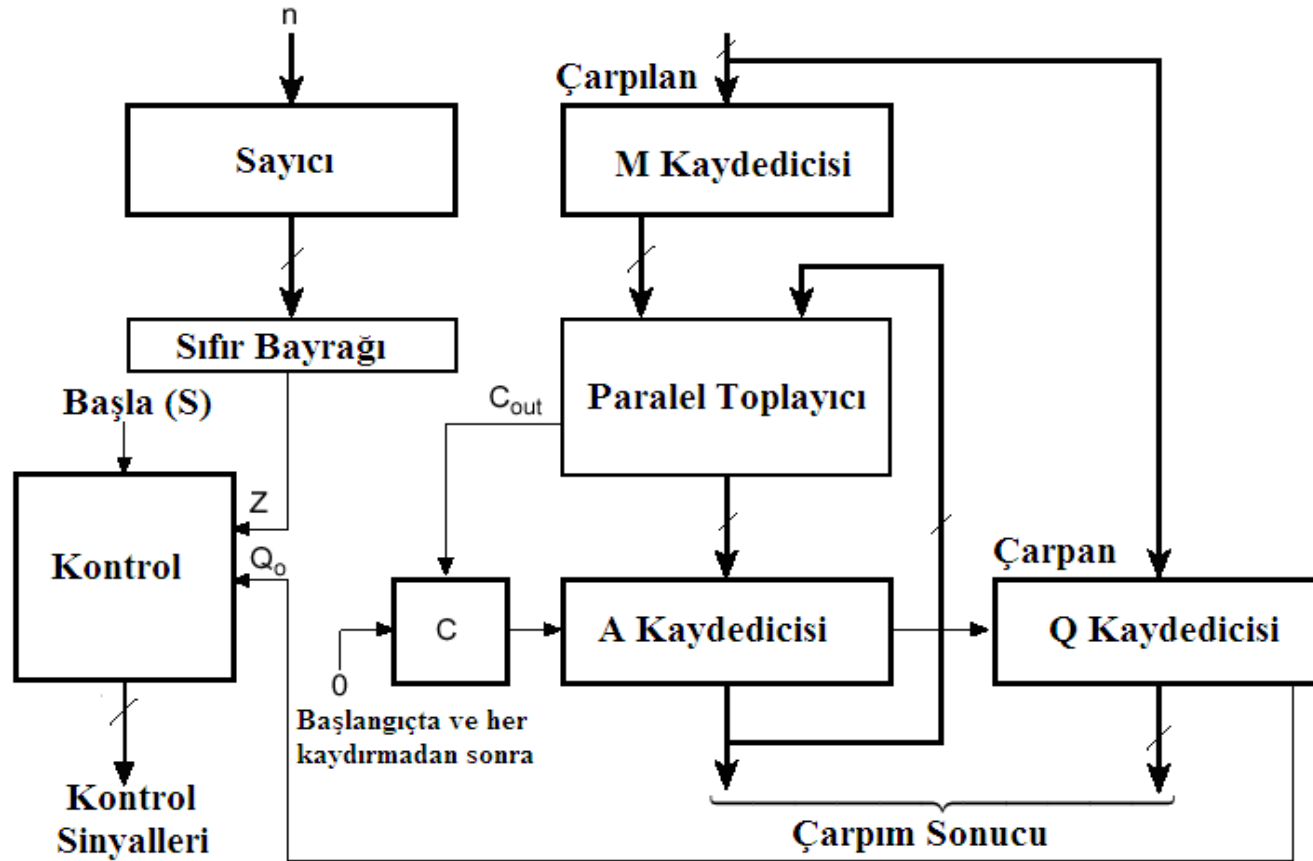


Örneğin; seçim uçları $S_5S_4S_3 = 011$ olduğunda, veri kaydedicisindeki veri ile akümülatördeki veri lojik “VEYA” işlemine tabi tutularak akümülatöre aktarılmaktadır.

Çarpma Kısım

- İki adet ikili sayının çarpımı, kalem kâğıtla yapılacak olursa bir dizi ardışıl kaydırma ve toplama işlemleriyle gerçekleştirilir. İşlem, çarpanın en önemsiz bitinden başlayarak sırayla bütün bitlerine bakılarak yapılır. Eğer çarpanın biti 1 ise çarpılan aşağıya aynen alınır. Aksi taktirde aşağıya sıfırlar yazılır. Daha sonra da sola bir bit kaydırılır. Kısmi çarpımlar toplanarak, çarpım elde edilmiş olur.
- Donanımsal olarak çarpma işlemi yapılacağı zaman, tüm kısmi çarpımları kaydedicilere yazmak yerine, kısmi çarpımların toplamı bir kaydediciye yazılır. Daha sonra da kısmi toplam sağa kaydırılır. Çarpanın biti 0 ise sıfırla toplamaya gerek yoktur sadece sağa kaydırma işlemi yapılır.

Çarpma İşleminin Prensiş Şeması



Örnek: 1001×0011 İşlemi için Algoritmanın Gerçekleştirilmesi

- M kaydedicisine çarpılanın, Q kaydedicisine çarpanın değeri konulur.
- A kaydedicisi ve elde biti (c) sıfırlanır, sayıcıya 4 değeri atanır.
- Çarpan'ın Q_0 bitine bakılarak kısmi çarpımlar toplamı elde edilir.
- İşlemin sonucunun yüksek anlamlı kısmı A, düşük anlamlı kısmı Q kaydedicisindedir.

Başlangıç Değerleri	C(0)	A (0000)	Q (0011) Q0	İşlem
1.Adım	0	1001	0011	Toplama
	0	0100	1001	Kaydırma
2.Adım	0	1101	1001	Toplama
	0	0110	1100	Kaydırma
3.Adım	0	0011	0110	Kaydırma
4.Adım	0	0001	1011	Kaydırma

M = 1001

Çarpım=A Q = 0001 1011 = $1B_h = 27$

Bölme Kısımı

- İki adet ikili sayının bölme işlemi, karşılaştırma, kaydırma ve çıkarma işlemleri gerektirir. İkili bölme, ondalık bölmeden daha basittir; işleme giren sayıların basamaklarında sadece 0 veya 1 vardır. Ayrıca bölenin, bölünen veya kalan içinde kaç defa olduğunu bulmak kolaydır.
- Sayısal bir bilgisayarda bölme işlemini donanımsal olarak gerçekleştirmek istersek, işlemler biraz değişecektir. Bölünen veya kısmi kalan bir sola kaydırılır. Bölünende bölen varsa gerekli çıkarma işlemi, bölünene bölenin 2'ye göre tümleyeninin eklenmesiyle yapılır.

Bölme Kısımının Algoritması

Bölen M ve bölünen Q kaydedicilerine konulur.

Sıra sayıcıya başlangıçta bölünenin bit sayısı atanır.

Bölünen, sola 1 kaydırılarak bölünenin en anlamlı kısmı A kaydedicisine aktarılır (Başlangıçta $A=0$ 'dır) ve bölenin 2'ye göre tümleyeni A 'ya eklenir. Bu işlemin sonucu pozitifse bölüm hanesine 1 yazılır (Q 'nun en düşük anlamlı kısmına 1 konulur) ve sıra sayıcı 1 azaltılır. Şayet kalan negatifse bölünende bölen yok demektir (Q 'nun en düşük anlamlı kısmına 0 konulur). Bölünen 1 sola kaydırılır ve kısmi kalana bölen eklenerek eski haline getirilir. Yine bölünen de bölenin olup olmadığı araştırılır. Bu işlemler, sayıcı sıfırlanana kadar devam eder.

Örnek: 010110/000011 İşlemi için Algoritmanın Gerçekleştirilmesi

- M kaydedicisine bölünen, Q kaydedicisine bölünenin değeri konulur.
- A kaydedicisi, bölme işlemi sonucu oluşan kalanı tutar ve başlangıçta sıfır değeri atanır, sayıcıya da 6 değeri yüklenir.
- Q kaydedicisi, A kaydedicisine doğru 1 sola kaydırılır, A'da bölen varsa Q'nun 0.bitine 1, yoksa 0 yüklenir. Bu işlemler 6 kere yapılır.

Başlangıç Değerleri	A (000000)	Q (010110) Q0	İşlem
1.adım	000000	10110 0	Kaydırma
2.Adım	000001	01100 0	Kaydırma
3.adım	000010	11000 0	Kaydırma
4.adım	000101	100000	Kaydırma
	000010	10000 1	Çıkartma
5.adım	000101	000010	Kaydırma
	000010	00001 1	Çıkartma
6.adım	000100	000110	Kaydırma
	000001	00011 1	Çıkartma

M= 000011

Bölüm = Q = 000111₂ = 7

Kalan = A = 000001₂ = 1

Örnek Uygulama

Üst seviyeli bir dil ile yazılan aşağıdaki programın, temel bilgisayar sistemimize göre assembly ve makine dili karşılıklarının bulunması.

```
a = 5;  
for i = 1 to 10  
{  
    a = a+i;  
}
```

Değişkenler 0030h adresinden itibaren belleğe konulmuştur;

0030h-0031h: a değişkeni

0032h-0033h: i değişkeni

0034h-0035h: i değişkeninin maksimum değeri

Programın Assembly Dili Karşılığının Bulunması

- 0000h:** LDA #0005h / Akümülatöre, a değişkenine konulacak olan decimal 5 değeri atanıyor.
- 0003h:** STA 0030h / a değişkeni için, belleğin 0030h ile 0031h adresleri tahsis edilmiştir.
- 0006h:** LDA #0001h / Akümülatöre, i değişkenine konulacak olan decimal 1 değeri atanıyor.
- 0009h:** STA 0032h / i değişkeni için, belleğin 0032h ile 0033h adresleri tahsis edilmiştir.
- 000Ch:** LDA #000Ah / Akümülatöre i değişkeninin maksimum değeri olan decimal 10 atanıyor.
- 000Fh:** STA 0034h / i'nin maksimum değeri için, belleğin 0034h ile 0035h adresleri tahsis edilmiştir.
- 0012h:** LDA 0030h / a değişkeni bellekten alınıp akümülatöre konuluyor.
- 0015h:** ADD 0032h / a değişkeni ile i değişkeni toplanıyor.
- 0018h:** STA 0030h / toplam sonucu, a değişkeninin tutulduğu bellek bölgesine kaydediliyor
- 001Bh:** LDA 0032h / i değişkeni bellekten alınıyor.
- 001Eh:** SUB 0034h / i'nin maksimum değerinden (decimal 10) i çıkartılıyor
- 0021h:** BZR ~09h / Eğer i değişkeni decimal 10 değerine ulaşmışsa, 002Ch adresine dallanılıyor
- 0023h:** LDA 0032h / i değişkeni bellekten alınıyor
- 0026h:** INCR / i değişkeninin değerinin aktarıldığı akümülatör, 1 artırılıyor
- 0027h:** STA 0032h / i değişkeni belleğe kaydediliyor
- 002Ah:** BRA ~E6h / 0012h adresine dallanılıyor
- 002Ch:** LDA 0030h / toplam sonucu akümülatöre aktarılıyor (Sonucu görebilmek için)
- 002Fh:** HLT / Program sonlandırılıyor.

Programın Makine Dili Karşılığının Bulunması

BELLEK ADRESLERİ	MAKİNE DİLİ	ASSEMBLY DİLİ
0000	1A	LDA #0005h
0001	00	
0002	05	
0003	A0	STA 0030h
0004	00	
0005	30	
0006	1A	LDA #0001h
0007	00	
0008	01	
0009	A0	STA 0032h
000A	00	
000B	32	
000C	1A	LDA #000Ah
000D	00	
000E	0A	
000F	A0	STA 0034h
0010	00	
0011	34	
0012	2A	LDA 0030h
0013	00	
0014	30	

0015	20	ADD 0032h
0016	00	
0017	32	
0018	A0	STA 0030h
0019	00	
001A	30	
001B	2A	LDA 0032h
001C	00	
001D	32	
001E	2E	SUB 0034h
001F	00	
0020	34	
0021	53	BZR ~09h
0022	09	
0023	2A	
0024	00	LDA 0032h
0025	32	
0026	03	
0027	A0	INCR
0028	00	
0029	32	
002A	50	STA 0032h
002B	E6	
002C	2A	
002D	00	BRA ~E6h
002E	30	
002F	0E	
002F	0E	HLT

Örnek 2

```
k=5;  
m=10;  
s=0;  
if (k>=m)  
    s=k+m;  
else  
    s=k-m;
```

Değişkenler DE00h adresinden itibaren belleğe konulmuştur;

DE00h-DE01h: k değişkeni

DE02h-DE03h: m değişkeni

DE04h-DE05h: s değişkeni

Örnek 2

```
0000h:LDA  #0005H;
0003h:STA  DE00H;
0006h:LDA  #000AH;
0009h:STA  DE02H;
000Ch:LDA  #0000H;
000Fh:STA  DE04H;
0012h:LDA  DE02H;
0015h:SUB  DE00H;
0018h:BCS  ~0AH;
001Ah:LDA  DE02H;
001Dh:SUB  DE00H;
0020h:STA  DE04H;
0023h:HLT
0024h:LDA  DE00H;
0027h:ADD  DE02H
002Ah:STA  DE04H
002Dh: HLT
```

```
k=5;
m=10;
s=0;
if (k>=m)
    s=k+m;
else
    s=k-m;
```

DE00h-DE01h: k değişkeni
DE02h-DE03h: m değişkeni
DE04h-DE05h: s değişkeni

Örnek 2

BELLEK ADRESLERİ	MAKİNE DİLİ	ASSEMBLY DİLİ
0000	1A	LDA #0005h
0001	00	
0002	05	
0003	A0	STA DE00h
0004	DE	
0005	00	
0006	1A	LDA #000Ah
0007	00	
0008	0A	
0009	A0	STA DE02h
000A	DE	
000B	02	
000C	1A	LDA #0000h
000D	00	
000E	00	
000F	A0	STA DE04h
0010	DE	
0011	04	
0012	2A	LDA DE02h
0013	DE	
0014	02	

0015	2E	SUB DE00h
0016	DE	
0017	00	
0018	52	BCS ~0AH
0019	0A	
001A	2A	LDA DE02h
001B	DE	
001C	02	
001D	2E	SUB DE00h
001E	DE	
001F	00	
0020	A0	STA DE04H
0021	DE	
0022	04	
0023	0E	HLT
0024	2A	LDA DE00H
0025	DE	
0026	00	
0027	20	ADD DE02H
0028	DE	
0029	02	
002A	A0	STA DE04H
002B	DE	
002C	04	
002D	0E	HLT